

飞腾矢量信号图像处理库用户指南

V1.7.0(2025-12-30)

版权所有 © 飞腾信息技术有限公司 2022。保留所有权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium和其他飞腾商标均为飞腾信息技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

本文档的内容视为飞腾的保密信息，您应当严格遵守保密义务；未经飞腾事先书面同意，您不得向任何第三方披露本文档内容或提供给任何第三方使用。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，飞腾在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但飞腾在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

本文档中所有内容，包括但不限于图片、架构设计、页面布局、文字描述，均由飞腾和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权等。非经飞腾和/或其关联公司书面同意，任何人不得擅自使用、修改、复制上述内容。
如有技术问题，可联系 support@phytium.com.cn 获取支持。

飞腾信息技术有限公司

地址：天津市滨海新区海缘路1号信安创业广场5号楼

网址：www.phytium.com.cn

版本历史

文档版本	发布日期	更新说明
V1.0.0	2024-03-22	首次发布
V1.2.0	2024-06-30	版本更新
V1.3.0	2024-10-30	版本更新
V1.4.0	2024-12-19	版本更新
V1.5.0	2025-03-26	版本更新
V1.6.0	2025-06-19	版本更新
V1.7.0	2025-08-15	版本更新

目 录

1 引言	1
2 安装和使用	1
2.1 环境要求	1
2.1.1 硬件环境	1
2.1.2 软件环境	1
2.2 安装 VSIPL-FT	1
2.3 批量测试	2
2.3.1 vsipl-ft/testing 正确性测试集	2
2.3.2 VSIPL-FT_testing 正确性测试集	2
2.4 使用 VSIPL-FT	2
2.4.1 接口调用示例 1	2
2.4.2 接口调用示例 2	4
3 数据类型	1
3.1 类型介绍	1
3.2 标量	3
3.2.1 实标量	3
3.2.2 复标量	4
3.2.3 Boolean	4
3.2.4 索引	5
3.2.5 Offset, Stride, Length	5
3.3 随机数生成器	5
3.4 块对象	6
3.5 块布局	6
3.5.1 存储格式	6
3.5.2 排列方式	7

3.5.3 维度顺序	7
3.5.4 布局属性	8
3.6 向量视图	9
3.6.1 实向量视图	9
3.6.2 复向量视图	11
3.7 矩阵视图	11
3.7.1 实矩阵视图对象	12
3.7.2 复矩阵视图对象	13
3.8 张量视图	14
3.8.1 实张量视图对象	15
3.8.2 复张量视图对象	16
3.9 信号处理	17
3.9.1 快速傅里叶变换 (FFT)	19
3.9.2 滤波 (Filtering)	20
3.9.3 卷积 (Convolution)	21
3.9.4 相关性 (Correlation)	22
3.9.5 直方图 (Histogram)	23
3.10 线性代数	23
3.10.1 LU 分解	24
3.10.2 Cholesky 分解	24
3.10.3 QR 分解	25
3.10.4 SVD 分解	26
3.11 直接数据访问 (Direct Data Access)	27
3.11.1 同步策略 (Synchronization Policy)	27
4 函数说明	29
4.1 命名规则	29
4.2 函数介绍	30
4.2.1 VSIPL-FT 支撑函数 (新增 18 个)	31
4.2.2 VSIPL-FT 标量函数 (新增 23 个)	80
4.2.3 VSIPL-FT 随机数字生成函数	100
4.2.4 VSIPL-FT 向量&元素运算函数 (新增 114 个)	102
4.2.5 VSIPL-FT 线性代数函数 (新增 63 个)	168
4.2.6 VSIPL-FT 信号处理函数 (新增 45 个)	194
4.2.7 VSIPL-FT 插值函数 (新增 9 个)	217

4.2.8 VSIPL-FT 置换函数（新增 7 个）	220
4.2.9 VSIPL-FT 直接数据访问（DDA）函数（新增 8 个）	222
5 总结	226

表 目 录

表 2-1 软件环境1

表 4-1 数据结构类型的说明与命名规则29

表 4-2 VSIPL-FT 函数分类 30

1 引言

VS IPL 是跨平台的标准应用编程接口，涵盖标量、向量操作，随机数生成、信号处理、基本线性代数库、数据块操作、内存管理等。飞腾矢量信号图像处理库 VS IPL-FT (Vector Signal Image Processing Library for Phytium) 提供高性能计算接口，相比开源同类型产品，VS IPL-FT 基于飞腾 CPU 体系结构和指令集特点进行了优化，实现了在飞腾平台上的性能提升。

2 安装和使用

本章节介绍了如何在飞腾平台上安装使用 VSIPL-FT。

2.1 环境要求

2.1.1 硬件环境

飞腾矢量信号图像处理库紧密支持飞腾处理器，已在 FT-2000/4、飞腾腾锐 D2000、FT-2000+/64、飞腾腾云 S2500、飞腾腾云 S5000C 处理器上成熟运用。

2.1.2 软件环境

VSIPL-FT 函数库已经验证的软件环境如表 2-1 所示。

表 2-1 软件环境

项目	版本
OS	CentOS8.2、Kylin V10、Ubuntu19.10、 OpenEuler22.03
GCC	7.3.0、8.3.1、9.3.0、10.3.1、11.3.0
GNU Binutils	2.32、2.33、2.34、2.35、2.37
Glibc	2.28、2.23、2.31、2.30、2.34

2.2 安装 VSIPL-FT

步骤 1 获取并解压 vsipl-ft.zip，解压缩目录下包含 VSIPL-FT 的库文件以及头文件。

步骤 2 设置库路径。

生成的库在解压文件夹中的 lib 目录下。

```
export LD_LIBRARY_PATH=$HOME/vsimpl-ft/lib:$LD_LIBRARY_PATH
```

2.3 批量测试

2.3.1 vsimpl-ft/testing 正确性测试集

步骤 1 进入测试目录

```
cd $HOME/vsimpl-ft/testing
```

步骤 2 编译运行

编译运行之前，确保已设置好库路径：

```
export  
LD_LIBRARY_PATH=$HOME/vsimpl-ft/lib:$LD_LIBRARY_PATH
```

编译运行：

```
make  
./test_all
```

2.3.2 VSIPL-FT_testing 正确性测试集

步骤 1 进入测试目录

```
cd $HOME/VSIPL-FT_testing
```

步骤 2 编译运行

编译运行之前，确保已设置好库路径：

```
export  
LD_LIBRARY_PATH=$HOME/vsimpl-ft/lib:$LD_LIBRARY_PATH
```

编译运行：

```
make  
./test_all
```

2.4 使用 VSIPL-FT

2.4.1 接口调用示例 1

步骤 1 创建 test.c 文件。

步骤 2 test.c 文件中添加以下代码内容。

```
#include<vsip.h>  
#include <stdlib.h>  
  
int main(int argc, char *argv[]){vsip_init((void*)0);  
{  
    int i = atoi(argv[1]);
```

```
vsip_length M = i;
vsip_length N = M;
vsip_length L = M;
vsip_major major = VSIP_ROW;
vsip_vview_f *a = vsip_vcreate_f(i,0);
vsip_cmview_f *B = vsip_cmcreate_f(M,L,0,0),
    *R = vsip_cmcreate_f(M,L,0,0);

for(int j=0;j<i;j++){
    float num_random_1 = ((float) (rand() % 1000+1)) / 10.0f;
    vsip_vput_f(a,j,num_random_1);
}
for (int i=0;i<M;i++){
    for(int j=0;j<L;j++){
        vsip_cscalar_f y;
        y.r=((rand() % 10000)) / 100.0f;
        y.i=((rand() % 10000)) / 100.0f;
        vsip_cmput_f(B,i,j,y);
    }
}

vsip_rvcmmul_f(a,B,major,R);

vsip_valldestroy_f(a);
vsip_cmalldestroy_f(B);
vsip_cmalldestroy_f(R);

} vsip_finalize((void*)0); return 1;
}
```

步骤 3 编写 MakeFile 文件如下。

```
RDIR=..
CC=cc
INCLUDEDIR=-I$(RDIR)/include
LIBDIR=-L$(RDIR)/lib
LIBS=-IVU -lvsip -lfftw3 -lfftw3f -llapacke -llapack -lgfortran -llapackex -lm
-fopenmp
OPTIONS=-O3

example: test.c
    $(CC) -o test test.c $(OPTIONS) $(INCLUDEDIR) $(LIBDIR) $(LIBS)

clean:
    rm -f test
```

步骤 4 编译运行如下。

```
export LD_LIBRARY_PATH=$PWD/../lib:$LD_LIBRARY_PATH
make
./test 5
```

2.4.2 接口调用示例 2

步骤 1 创建 test.c 文件。

步骤 2 test.c 文件中添加以下代码内容。

```
#include "vsip.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int main(int argc, char *argv[]) {vsip_init((void*)0);
{
    int testlen = atoi(argv[1]);
    int32_t row = testlen;
    int32_t col = testlen;
    int64_t rowStride = 1;
    int64_t colStride = row * rowStride;
    uint64_t offset = 0;
    vsip_major major = VSIP_ROW;
    uint64_t cmlen = 2 * (col * colStride + offset);
    float *src1 = (float *)malloc(testlen * sizeof(float));
    float *src2 = (float *)malloc(cmlen * sizeof(float));
    float *dst = (float *)malloc(cmlen * sizeof(float));

    for (int32_t i = 0; i < testlen; ++i) {
        src1[i] = ((float) (rand() % 1000+1)) / 10.0f;
    }
    for (int32_t i = 0; i < cmlen; ++i) {
        src2[i] = ((float) (rand() % 1000+1)) / 10.0f;
    }

    vsip_block_f *block_a = vsip_blockbind_f(src1, testlen, VSIP_MEM_NONE);
    vsip_cblock_f *block_b = vsip_cblockbind_f(src2, NULL, cmlen,
    VSIP_MEM_NONE);
    vsip_cblock_f *block_c = vsip_cblockbind_f(dst, NULL, cmlen,
    VSIP_MEM_NONE);

    vsip_vview_f *a = vsip_vbind_f(block_a, 0, 1, testlen);
```

```
vsip_blockadmit_f(block_a, VSIP_TRUE);
vsip_cmview_f *b = vsip_cmbind_f(block_b, offset, colStride, col, rowStride,
row);
vsip_cblockadmit_f(block_b, VSIP_TRUE);
vsip_cmview_f *c = vsip_cmbind_f(block_c, offset, colStride, col, rowStride,
row);
vsip_cblockadmit_f(block_c, VSIP_TRUE);

vsip_rvcmmul_f(a,b,major,c);

vsip_valldestroy_f(a);
vsip_cmalldestroy_f(b);
vsip_cmalldestroy_f(c);

free(src1);free(src2);free(dst);

} vsip_finalize((void*)0); return 0;
}
```

步骤 3 编写 MakeFile 文件如下。

```
RDIR=..
CC=cc
INCLUDEDIR=-I$(RDIR)/include
LIBDIR=-L$(RDIR)/lib
LIBS=-IVU -lvsip -lfft3 -lfft3f -llapacke -llapack -lgfortran -llapackex -lm
-fopenmp
OPTIONS=-O3

example: test.c
    $(CC) -o test test.c $(OPTIONS) $(INCLUDEDIR) $(LIBDIR) $(LIBS)

clean:
    rm -f test
```

步骤 4 编译运行如下。

```
export LD_LIBRARY_PATH=$PWD/../lib:$LD_LIBRARY_PATH
make
./test 5
```

3 数据类型

3.1 类型介绍

本节提到的基本标量类型数据类型定义对应 ANSI C 中的宏定义（float.h 和 limits.h），包括：

1. 浮点数：

这是使用后缀 “_f” 表示的类型：

- ◆ `_f(float)`：单精度浮点数。
- ◆ `_d(double)`：双精度浮点数。
- ◆ `_l(implementation-dependent)`：这取决于具体的实现，可能是某种长双精度或其他类型的浮点数。

可移植精度浮点类型：

- ◆ `_f6`：至少有 6 位十进制精度的浮点数类型。IEEE 754 单精度（32 位）具有 6 位十进制精度。
- ◆ `_f15`：至少有 15 位十进制精度的浮点数类型。IEEE 754 双精度（64 位）具有 15 位十进制精度。
- ◆ `_fn`：具有至少 n 位十进制精度的浮点数类型。如果系统支持这种精度，它根据 FLT_MANT_DIG、DBL_MANT_DIG 或 LDBL_MANT_DIG 的值解析为最小的 C 类型。

2. 整数：

整数类型通常用于表示不带小数点的数值。整数类型使用后缀 “_i” 来表示：

- ◆ `_c(char)`：字符类型，通常占用一个字节。
- ◆ `_uc(unsigned char)`：无符号字符类型，通常占用一个字节。
- ◆ `_si(short int)`：短整型，通常占用 2 个字节。

- ◆ `_us(unsigned short int)`: 无符号短整型, 通常占用 2 个字节。
- ◆ `_i(int)`: 整型, 通常占用 4 个字节。
- ◆ `_u(unsigned int)`: 无符号整型, 通常占用 4 个字节。
- ◆ `_li(long int)`: 长整型, 通常占用 8 个字节。
- ◆ `_ul(unsigned long int)`: 无符号长整型, 通常占用 8 个字节。
- ◆ `_ll`: 依赖于实现的整数类型, 可能占用更多或更少的字节。
- ◆ `_ull`: 未定义的、无符号的整数类型, 依赖于实现。
- ◆ `_vi`: 未定义的、有符号的整数类型, 依赖于实现。

以下类型定义了不同精度和大小的整数:

Portable precision types of at least n bits:

- ◆ `_il8`: 整型, 至少 8 位。
- ◆ `_il16`: 整型, 至少 16 位。
- ◆ `_il32`: 整型, 至少 32 位。
- ◆ `_il64`: 整型, 至少 64 位。
- ◆ `_iln`: 整型, 至少 n 位。
- ◆ `_ul8`: 无符号整型, 至少 8 位。
- ◆ `_ul16`: 无符号整型, 至少 16 位。
- ◆ `_ul32`: 无符号整型, 至少 32 位。
- ◆ `_ul64`: 无符号整型, 至少 64 位。
- ◆ `_uln`: 无符号整型, 至少 n 位。

Portable precision types of exactly n bits:

- ◆ `_ie8`: 整型, 恰好 8 位。
- ◆ `_ie16`: 整型, 恰好 16 位。
- ◆ `_ie32`: 整型, 恰好 32 位。
- ◆ `_ie64`: 整型, 恰好 64 位。
- ◆ `_ien`: 整型, 恰好 n 位。
- ◆ `_ue8`: 无符号整型, 恰好 8 位。
- ◆ `_ue16`: 无符号整型, 恰好 16 位。
- ◆ `_ue32`: 无符号整型, 恰好 32 位。
- ◆ `_ue64`: 无符号整型, 恰好 64 位。
- ◆ `_uen`: 无符号整型, 恰好 n 位。

Fastest type of at least n bits:

- ◆ `_if8`: 最快速整型, 至少 8 位。
- ◆ `_if16`: 最快速整型, 至少 16 位。

- ◆ `_if32`: 最快速整型, 至少 32 位。
- ◆ `_if64`: 最快速整型, 至少 64 位。
- ◆ `_ifn`: 最快速整型, 至少 n 位。

Unsigned Fastest Types of at least n bits:

- ◆ `_uf8`: 无符号最快速整型, 至少 8 位。
- ◆ `_uf16`: 无符号最快速整型, 至少 16 位。
- ◆ `_uf32`: 无符号最快速整型, 至少 32 位。
- ◆ `_uf64`: 无符号最快速整型, 至少 64 位。
- ◆ `_ufn`: 无符号最快速整型, 至少 n 位。

3.Integer or Floating Point 整数或浮点数:

用后缀 “`_p`” 表示, 这个后缀是上述 “`_f`” 和 “`_i`” 的并集。

4.Boolean 布尔类型:

值为 0 逻辑上为假, 值为非 0 逻辑上为真。

用后缀 “`_b1`” 表示。

5.Vector Index 向量索引:

无符号整数, 用于表示向量元素的索引。向量元素 x_i 的向量索引为 i。

用后缀 “`_vi`” 表示。

6.Matrix Index 矩阵索引:

无符号整数, 用于表示矩阵元素的索引。矩阵元素 x_{ij} 的索引为(i,j)。

用后缀 “`_mi`” 表示。

7.Tensor Index 张量索引:

无符号整数, 用于表示 3 阶张量 (三维数组) 的元素的索引。张量元素 $x_{i,j,k}$ 的索引为(i,j,k)。

用后缀 “`_ti`” 表示。

3.2 标量

3.2.1 实标量

```
#define VSIP_PI implementation-defined
#define VSIP_MAX_SCALAR_L implementation-defined
#define VSIP_MAX_SCALAR_D DBL_MAX
#define VSIP_MAX_SCALAR_F FLT_MAX
#define VSIP_MAX_SCALAR_LI LONG_MAX
#define VSIP_MAX_SCALAR_UL ULONG_MAX
#define VSIP_MAX_SCALAR_I INT_MAX
#define VSIP_MAX_SCALAR_U UINT_MAX
```



```
#define VSIP_MAX_SCALAR_SI SHRT_MAX
#define VSIP_MAX_SCALAR_US USHRT_MAX
#define VSIP_MAX_SCALAR_C CHAR_MAX
#define VSIP_MAX_SCALAR_UC UCHAR_MAX
#define VSIP_MIN_SCALAR_L implementation-defined
#define VSIP_MIN_SCALAR_D DBL_MIN
#define VSIP_MIN_SCALAR_F FLT_MIN
#define VSIP_MIN_SCALAR_LI LONG_MIN
#define VSIP_MIN_SCALAR_UL 0
#define VSIP_MIN_SCALAR_I INT_MIN
#define VSIP_MIN_SCALAR_U 0
#define VSIP_MIN_SCALAR_SI SHRT_MIN
#define VSIP_MIN_SCALAR_US 0
#define VSIP_MIN_SCALAR_C CHAR_MIN
#define VSIP_MIN_SCALAR_UC 0
#ifdef LLONG_MAX
#define VSIP_MAX_SCALAR_LL LLONG_MAX
#define VSIP_MAX_SCALAR_ULL ULLONG_MAX
#define VSIP_MIN_SCALAR_LL LLONG_MIN
#define VSIP_MIN_SCALAR_ULL 0
#endif /* LLONG_MAX */
typedef long double vsip_scalar_l;
typedef double vsip_scalar_d;
typedef float vsip_scalar_f;
typedef signed long int vsip_scalar_li;
typedef unsigned long int vsip_scalar_ul;
typedef signed int vsip_scalar_i;
typedef unsigned int vsip_scalar_u;
typedef signed short int vsip_scalar_si;
typedef unsigned short int vsip_scalar_us;
typedef signed char vsip_scalar_c;
typedef unsigned char vsip_scalar_uc;
#ifdef LLONG_MAX typedef signed long long vsip_scalar_ll; /* Non-ANSI C */
typedef unsigned long long vsip_scalar_ull; /* Non-ANSI C */
#endif /* LLONG_MAX */
```

3.2.2 复标量

```
typedef struct{ vsip_scalar_p r, i; }vsip_cscalar_p;
```

3.2.3 Boolean

```
typedef implementation-defined vsip_scalar_bl;
```

```
typedef vsip_scalar_bl vsip_bool;
#define VSIP_FALSE 0
#define VSIP_TRUE 1
```

3.2.4 索引

向量索引 (Vector index) :

```
typedef unsigned implementation-defined vsip_scalar_vi;
```

矩阵索引 (Matrix index) :

```
typedef struct{ vsip_scalar_vi r,c;}vsip_scalar_mi;
```

张量索引 (Tensor index) :

```
typedef struct{ vsip_scalar_vi z,y,x;}vsip_scalar_ti;
```

“Element index” 和 “vector index” 在数学和编程语境中经常被用作同义词，都表示向量或数组中特定位置的标识符。

```
typedef vsip_scalar_vi vsip_index;
#define VSIP_MAX_SCALAR_VI implementation-defined
#define VSIP_MIN_SCALAR_VI 0
```

3.2.5 Offset, Stride, Length

Offset: 偏移量，指的是元素之间的位置差。例如，在数组或向量中，第一个元素和第二个元素之间的偏移量是 1。

```
typedef vsip_scalar_vi vsip_offset;
```

Stride: 步长或步幅，描述了在内存中沿着某一维度（如行或列）连续元素之间的距离。例如，对于一个矩阵，步长可能表示在连续行之间的距离。

```
typedef signed implementation-defined vsip_stride;
```

Length: 长度，指的是某个维度或向量的元素数量。

```
typedef vsip_scalar_vi vsip_length;
```

3.3 随机数生成器

用于存储随机数生成器状态信息的对象类型:

```
struct vsip_randomstate;
typedef struct vsip_randomstate vsip_randstate;
```

随机数生成器的枚举类型定义了一个枚举类型 vsip_rng，它有两个可能的值：VSIP_PRNG 和 VSIP_NPRNG。这个枚举类型用于表示想要使用的随机数生成器类型。其中，VSIP_PRNG 表示一个可移植的随机数生成器，而 VSIP_NPRNG 表示一个非可移植的随机数生成器。定义如下：

```
typedef enum
{
```

```
VSIP_PRNG = 0, /* Portable random number generator */
VSIP_NPRNG = 1 /* Non-portable random number generator */
}vsip_rng;
```

3.4 块对象

枚举类型 `vsip_memory_hint`, 它用于为数据内存的块对象分配内存时提供提示。这些提示可以帮助优化内存分配和访问方式, 以提高性能或满足特定的数据访问要求。类型定义如下:

```
typedef enum
{
    VSIP_MEM_NONE = 0, /* 没有提示 */
    VSIP_MEM_RDONLY = 1, /* 只读 */
    VSIP_MEM_CONST = 2, /* 常量 */
    VSIP_MEM_SHARED = 3, /* 共享 */
    VSIP_MEM_SHARED_RDONLY = 4, /* 共享且只读 */
    VSIP_MEM_SHARED_CONST = 5 /* 共享且常量 */
}vsip_memory_hint;
```

枚举类型 `vsip_cmplx_mem`, 用于表示复数数据的首选内存存储布局。这个枚举类型用于指定如何存储复数数据。类型定义如下:

```
typedef enum
{
    VSIP_CMPLX_INTERLEAVED = 0, /* 交错的, 即实部和虚部连续存储 */
    VSIP_CMPLX_SPLIT = 1, /* 分离的, 即实部和虚部分开存储 */
    VSIP_CMPLX_NONE = 2 /* 没有首选的存储布局 */
}vsip_cmplx_mem;
```

3.5 块布局

3.5.1 存储格式

枚举类型 `vsip_storage_format` 用于描述如何在内存中存储数据或如何处理数据。具体来说, 这个枚举类型描述了以下四种存储格式:

`VSIP_STORAGE_FORMAT_ANY`: 这个格式表示实现可以选择任何存储格式。一旦实现决定了使用的存储格式, 相应的运行时布局属性将被更新为实际使用的存储格式。

`VSIP_STORAGE_FORMAT_ARRAY`: 对于复数数据, 这个格式表示复数数据被保存在一个 `vsip_cscalar_p[]` 类型的数组中。

`VSIP_STORAGE_FORMAT_SPLIT_COMPLEX`: 这个格式表示复数数据

被保存在两个独立的数组中，一个数组包含实部，另一个数组包含虚部。

VSIP_STORAGE_FORMAT_INTERLEAVED_COMPLEX: 这个格式表示复数数据被保存在一个数组中，实部和虚部交替存储。

```
typedef enum
{
    VSIP_STORAGE_FORMAT_ANY = 3,
    VSIP_STORAGE_FORMAT_ARRAY = 2, /* VSIP_CMPLX_NONE */
    VSIP_STORAGE_FORMAT_SPLIT_COMPLEX = 1, /*
VSIP_CMPLX_SPLIT */
    VSIP_STORAGE_FORMAT_INTERLEAVED_COMPLEX = 0 /*
VSIP_CMPLX_INTERLEAVED */
}vsip_storage_format;
```

3.5.2 排列方式

枚举类型 vsip_pack_type，用于指定数据在块中的排列方式。这些枚举值提供了不同的数据排列选项，以满足不同的需求，如性能优化、内存使用等。以下是该枚举类型的定义：

```
typedef enum
{
    VSIP_PACK_TYPE_ANY, /* 表示任何排列类型都可以 */
    VSIP_PACK_TYPE_UNIT_STRIDE, /* 表示每个元素之间有一个单位步
长 */
    VSIP_PACK_TYPE_DENSE, /* 表示数据密集排列，没有间隙 */
    VSIP_PACK_TYPE_ALIGNED, /* 表示数据对齐排列 */
    VSIP_PACK_TYPE_ALIGNED_8, /* 表示数据对齐到 8 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_16, /* 表示数据对齐到 16 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_32, /* 表示数据对齐到 32 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_64, /* 表示数据对齐到 64 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_128, /* 表示数据对齐到 128 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_256, /* 表示数据对齐到 256 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_512, /* 表示数据对齐到 512 的倍数 */
    VSIP_PACK_TYPE_ALIGNED_1024 /* 表示数据对齐到 1024 的倍数 */
}vsip_pack_type;
```

3.5.3 维度顺序

维度顺序类型用于指定内存地址中逻辑维度的使用顺序。逻辑维度从大到小编号，从 0 开始。与数据对象不相关的维度将被忽略。

视图 vsip_dim_order 包含三个 vsip_dim 类型的成员变量 dim0、dim1 和 dim2，用于表示三个维度的顺序。静态的 vsip_dim_order 常量，用于表示不同的维度顺序。

例如，VSIP_DIM_ORDER_VROW_MAJOR 表示垂直行主顺序，VSIP_DIM_ORDER_MROW_MAJOR 表示矩阵行主顺序，等等。这些常量的值都设置为{0,1,2}，这意味着所有三个维度的顺序都是从 0 开始编号的。具体的顺序取决于常量的名称，例如 VSIP_DIM_ORDER_VROW_MAJOR 表示垂直行主顺序，即第一个维度是最大的维度，第二个维度是次大的维度，第三个维度是最小的维度。这些常量可以用于指定数据对象的维度顺序，以满足特定的内存访问需求。定义如下：

```
typedef vsip_scalar_vi vsip_dim;
typedef struct
{
    vsip_dim dim0;
    vsip_dim dim1;
    vsip_dim dim2;
}vsip_dim_order;
static vsip_dim_order const VSIP_DIM_ORDER_VROW_MAJOR = {0, 1, 2};
static vsip_dim_order const VSIP_DIM_ORDER_MROW_MAJOR = {0, 1, 2};
static vsip_dim_order const VSIP_DIM_ORDER_TROW_MAJOR = {0, 1, 2};
static vsip_dim_order const VSIP_DIM_ORDER_VCOL_MAJOR = {0, 1, 2};
static vsip_dim_order const VSIP_DIM_ORDER_MCOL_MAJOR = {1, 0, 2};
static vsip_dim_order const VSIP_DIM_ORDER_TCOL_MAJOR = {2, 1, 0};
```

3.5.4 布局属性

视图 vsip_datalayout，用于指定数据在块或缓冲区中的布局属性。这些布局属性可以用于指定如何将数据存储在内存中，以便在后续的计算操作中进行高效的访问和操作。定义如下：

```
typedef struct
{
    vsip_length dim; /* 表示数据的维度大小 */
    vsip_storage_format storage_format; /* 表示数据的存储格式，使用之前定义的 vsip_storage_format 枚举类型 */
    vsip_pack_type packing; /* 表示数据的排列方式，使用之前定义的 vsip_pack_type 枚举类型 */
    vsip_dim_order order; /* 表示数据的维度顺序，使用之前定义的 vsip_dim_order 视图类型 */
}vsip_datalayout;
```

3.6 向量视图

3.6.1 实向量视图

向量索引的向量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride stride;
    vsip_length length;
    vsip_block_vi *block;
}vsip_vattr_vi;
```

向量索引的向量视图对象类型:

```
struct vsip_vviewobject_vi;
typedef struct vsip_vviewobject_vi vsip_vview_vi;
```

矩阵索引的向量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride stride;
    vsip_length length;
    vsip_block_mi *block;
}vsip_vattr_mi;
```

矩阵索引的向量视图对象类型:

```
struct vsip_vviewobject_mi;
typedef struct vsip_vviewobject_mi vsip_vview_mi;
```

张量索引的向量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride stride;
    vsip_length length;
    vsip_block_ti *block;
}vsip_vattr_ti;
```

张量索引的向量视图对象类型:

```
struct vsip_vviewobject_ti;
typedef struct vsip_vviewobject_ti vsip_vview_ti;
```

布尔值的向量视图属性结构:

```
typedef struct
{
```

```
    vsip_offset offset;  
    vsip_stride stride;  
    vsip_length length;  
    vsip_block_bl *block;  
}vsip_vattr_bl;
```

布尔值的向量视图对象类型:

```
struct vsip_vviewobject_bl;  
typedef struct vsip_vviewobject_bl vsip_vview_bl;
```

整数数据的向量视图属性结构:

```
typedef struct  
{  
    vsip_offset offset;  
    vsip_stride stride;  
    vsip_length length;  
    vsip_block_i *block;  
}vsip_vattr_i;
```

整数数据的向量视图对象类型:

```
struct vsip_vviewobject_i;  
typedef struct vsip_vviewobject_i vsip_vview_i;
```

浮点数据的向量视图属性结构:

```
typedef struct  
{  
    vsip_offset offset;  
    vsip_stride stride;  
    vsip_length length;  
    vsip_block_f *block;  
}vsip_vattr_f;
```

浮点数据的向量视图对象类型:

```
struct vsip_vviewobject_f;  
typedef struct vsip_vviewobject_f vsip_vview_f;
```

vsip_vattr_vi, vsip_vattr_mi, vsip_vattr_ti, vsip_vattr_bl, vsip_vattr_i 和 vsip_vattr_f 这些结构体定义了不同类型的向量视图的属性结构。每个结构体都包含以下成员:

- ◆ offset: 一个偏移量, 用于指定视图在原始数据中的起始位置。
- ◆ stride: 一个步长, 用于指定在原始数据中移动一个元素所需的步数。
- ◆ length: 一个长度, 用于指定视图的大小。
- ◆ block: 一个指向块 (block) 的指针, 块是原始数据的一个子集。

vsip_vviewobject_vi, vsip_vviewobject_mi, vsip_vviewobject_ti, vsip_vviewobject_bl, vsip_vviewobject_i 和 vsip_vviewobject_f 这些结构体定

义了不同类型的向量视图的对象类型。使用 typedef 关键字为这些结构体定义了别名，如 vsip_vview_vi， vsip_vview_mi， vsip_vview_ti， vsip_vview_bl， vsip_vview_i 和 vsip_vview_f。这些别名使得使用这些结构体更加方便。

通过使用向量视图，可以灵活地对数据进行切片、投影和变换等操作。由于视图不复制数据，因此它们可以有效地处理大规模数据集，并避免了不必要的内存开销。

3.6.2 复向量视图

复数整数数据的向量视图属性结构：

```
typedef struct
{
    vsip_offset offset;
    vsip_stride stride;
    vsip_length length;
    vsip_cblock_i *block;
}vsip_cvattr_i;
```

复数整数数据的向量视图对象类型：

```
struct vsip_cvviewobject_i;
typedef struct vsip_cvviewobject_i vsip_cvview_i;
```

复数浮点数据的向量视图属性结构：

```
typedef struct
{
    vsip_offset offset;
    vsip_stride stride;
    vsip_length length;
    vsip_cblock_f *block;
}vsip_cvattr_f;
```

复数浮点数据的向量视图对象类型：

```
struct vsip_cvviewobject_f;
typedef struct vsip_cvviewobject_f vsip_cvview_f;
```

与普通向量视图类似，复数向量视图也包含起始位置、步长、长度等属性，但这些属性用于处理复数数据。

3.7 矩阵视图

枚举类型 vsip_major 用于表示内存的方向。内存方向主要涉及多维数组在内存中的布局，对于多维数组，有两种主要的布局方式：行优先（Row-major）和列优先（Column-major）。

```
typedef enum
{
```



```
VSIP_ROW = 0, /* Row, C style */
VSIP_COL = 1 /* Column, FORTRAN style */
}vsip_major;
```

VSIP_ROW=0 表示行优先 (Row-major) 布局, 这是 C 语言风格的布局方式。在行优先布局中, 数组的元素按照行进行存储, 即先存储第 0 行的所有元素, 然后是第 1 行的所有元素, 以此类推。

VSIP_COL=1 表示列优先 (Column-major) 布局, 这是 FORTRAN 语言风格的布局方式。在列优先布局中, 数组的元素按照列进行存储, 即先存储第 0 列的所有元素, 然后是第 1 列的所有元素, 以此类推。

3.7.1 实矩阵视图对象

布尔值的矩阵视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride col_stride;
    vsip_length col_length;
    vsip_stride row_stride;
    vsip_length row_length;
    vsip_block_bl *block;
}vsip_mattr_bl;
```

布尔值的矩阵视图对象类型:

```
struct vsip_mviewobject_bl;
typedef struct vsip_mviewobject_bl vsip_mview_bl;
```

整数数据的矩阵视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride col_stride;
    vsip_length col_length;
    vsip_stride row_stride;
    vsip_length row_length;
    vsip_block_i *block;
}vsip_mattr_i;
```

整数数据的矩阵视图对象类型:

```
struct vsip_mviewobject_i;
typedef struct vsip_mviewobject_i vsip_mview_i;
```

浮点数据的矩阵视图属性结构:

```
typedef struct
```

```
{
    vsip_offset offset;
    vsip_stride col_stride;
    vsip_length col_length;
    vsip_stride row_stride;
    vsip_length row_length;
    vsip_block_f *block;
}vsip_mattr_f;
```

浮点数据的矩阵视图对象类型:

```
struct vsip_mviewobject_f;
typedef struct vsip_mviewobject_f vsip_mview_f;
```

3.7.2 复矩阵视图对象

复数整数或浮点数据的矩阵视图对象类型:

```
struct vsip_cmviewobject_p;
typedef struct vsip_cmviewobject_p vsip_cmview_p;
```

复数整数数据的矩阵视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride col_stride;
    vsip_length col_length;
    vsip_stride row_stride;
    vsip_length row_length;
    vsip_cblock_i *block;
}vsip_cmattr_i;
```

复数整数数据的矩阵视图对象类型:

```
struct vsip_cmviewobject_i;
typedef struct vsip_cmviewobject_i vsip_cmview_i;
```

复数浮点数据的矩阵视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride col_stride;
    vsip_length col_length;
    vsip_stride row_stride;
    vsip_length row_length;
    vsip_cblock_f *block;
}vsip_cmattr_f;
```

复数浮点数据的矩阵视图对象类型:

```
struct vsip_cmviewobject_f;
```

```
typedef struct vsip_cmviewobject_f vsip_cmview_f;
```

3.8 张量视图

以下定义了几个枚举类型，用于在处理张量时指定不同的内存布局和切片方式。

```
/* 指示张量主要的内存方向；C 风格以尾随维度，或者 FORTRAN 风格以领先维度。 */
typedef enum
{
    VSIP_TRAILING = 0, /* 尾随维度，C 风格 */
    VSIP_LEADING = 1 /* 领先维度，FORTRAN 风格 */
}vsip_tmajor;
```

vsip_tmajor 枚举用于指定张量数据在内存中的存储顺序。VSIP_TRAILING 表示 C 风格的尾随维度，而 VSIP_LEADING 表示 FORTRAN 风格的领先维度。

```
/* 指定张量的 2-D 切片。 */
typedef enum
{
    VSIP_TMYX = 0, /* Y - X 子矩阵 */
    VSIP_TMXZ = 1, /* Z - X 子矩阵 */
    VSIP_TMZY = 2 /* Z - Y 子矩阵 */
}vsip_tmslice;
```

vsip_tmslice 枚举用于指定张量的二维切片方式。VSIP_TMYX、VSIP_TMXZ 和 VSIP_TMZY 分别表示不同的子矩阵切片方式。

```
/* 指定张量的 1-D 切片。 */
typedef enum
{
    VSIP_TVX = 0, /* X 子向量 */
    VSIP_TVY = 1, /* Y 子向量 */
    VSIP_TVZ = 2 /* Z 子向量 */
}vsip_tvslice;
```

vsip_tvslice 枚举用于指定张量的一维切片方式。VSIP_TVX、VSIP_TVY 和 VSIP_TVZ 分别表示在不同的维度上取子向量。

```
typedef enum
{
    VSIP_TTRANS_NOP = 0, /* 无转置 */
    VSIP_TTRANS_YX = 1, /* Y - X 转置 */
    VSIP_TTRANS_ZY = 2, /* Z - Y 转置 */
    VSIP_TTRANS_ZX = 3, /* Z - X 转置 */
    VSIP_TTRANS_YXZY = 4, /* Y - X & Z - Y 转置 */
    VSIP_TTRANS_YXZX = 5 /* Y - X & Z - X 转置 */
}vsip_ttrans;
```

vsip_ttrans 枚举用于指定张量的转置方式：

- ◆ VSIP_TTRANS_NOP: 不执行任何转置操作。
- ◆ VSIP_TTRANS_YX: 将张量沿 Y 轴和 X 轴进行转置。
- ◆ VSIP_TTRANS_ZY: 将张量沿 Z 轴和 Y 轴进行转置。
- ◆ VSIP_TTRANS_ZX: 将张量沿 Z 轴和 X 轴进行转置。
- ◆ VSIP_TTRANS_YXZY: 首先沿 Y 轴和 X 轴转置, 然后沿 Z 轴和 Y 轴转置。
- ◆ VSIP_TTRANS_YXZX: 首先沿 Y 轴和 X 轴转置, 然后沿 Z 轴和 X 轴转置。

3.8.1 实张量视图对象

布尔值的张量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride z_length;
    vsip_length z_stride;
    vsip_stride y_length;
    vsip_length y_stride;
    vsip_stride x_length;
    vsip_length x_stride;
    vsip_block_bl *block;
}vsip_tattr_bl;
```

布尔值的张量视图对象类型:

```
struct vsip_tviewobject_bl;
typedef struct vsip_tviewobject_bl vsip_tview_bl;
```

整数数据的张量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride z_length;
    vsip_length z_stride;
    vsip_stride y_length;
    vsip_length y_stride;
    vsip_stride x_length;
    vsip_length x_stride;
    vsip_block_i *block;
}vsip_tattr_i;
```

整数数据的张量视图对象类型:

```
struct vsip_tviewobject_i;
typedef struct vsip_tviewobject_i vsip_tview_i;
```

浮点数据的张量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride z_length;
    vsip_length z_stride;
    vsip_stride y_length;
    vsip_length y_stride;
    vsip_stride x_length;
    vsip_length x_stride;
    vsip_block_f *block;
}vsip_tattr_f;
```

浮点数据的张量视图对象类型:

```
struct vsip_tviewobject_f;
typedef struct vsip_tviewobject_f vsip_tview_f;
```

这些结构体和类型定义了张量视图的属性和对象，其中包含了张量在三个维度（X,Y,Z）上的长度、步长和偏移量，以及指向数据块的指针。这些定义允许在处理多维数据时，可以灵活地定义数据的布局和访问方式。

3.8.2 复张量视图对象

复数整数或浮点数据的张量视图对象类型:

```
struct vsip_ctviewobject_p;
typedef struct vsip_ctviewobject_p vsip_ctview_p;
```

复数整数数据的张量视图属性结构:

```
typedef struct
{
    vsip_offset offset;
    vsip_stride z_length;
    vsip_length z_stride;
    vsip_stride y_length;
    vsip_length y_stride;
    vsip_stride x_length;
    vsip_length x_stride;
    vsip_cblock_i *block;
}vsip_ctattr_i;
```

复数整数数据的张量视图对象类型:

```
struct vsip_ctviewobject_i;
typedef struct vsip_ctviewobject_i vsip_ctview_i;
```

复数浮点数据的张量视图属性结构:

```
typedef struct
{
```

```
vsip_offset offset;
vsip_stride z_length;
vsip_length z_stride;
vsip_stride y_length;
vsip_length y_stride;
vsip_stride x_length;
vsip_length x_stride;
vsip_cblock_f *block;
}vsip_ctattr_f;
```

复数浮点数据的张量视图对象类型:

```
struct vsip_ctviewobject_f;
typedef struct vsip_ctviewobject_f vsip_ctview_f;
```

这些定义提供了复数张量数据的视图对象和属性结构，包括张量在三个维度（X,Y,Z）上的偏移量、长度、步长以及指向复数数据块的指针。这些结构允许开发者高效地操作和访问复数张量数据。

3.9 信号处理

◆ vsip_alg_hint 枚举用于提供优化对象的提示:

```
typedef enum
{
    VSIP_ALG_TIME = 0,
    VSIP_ALG_SPACE = 1,
    VSIP_ALG_NOISE = 2
}vsip_alg_hint;
```

➤ **VSIP_ALG_SPACE**: 优先最小化所需的内存。

VSIP 开源库在实现时，会优先选择那些内存占用最少的算法或实现策略。这可能意味着使用原地（in-place）计算、避免创建大的临时缓冲区，或者选择内存效率高但计算可能稍慢的算法。

VSIP-FT 库在调用 FFT 运算的接口中，这个值对应的是 FFTW 规划器标志（planner flags）的 **FFTW_MEASURE**: FFTW 会尝试多种不同的 FFT 算法和参数配置。对于每一种配置，它都会实际执行一次（或多次）FFT 计算，并精确测量其执行时间。然后，它选择其中执行时间最短的那个方案来创建最终的 plan。创建 plan 的时间较长，执行性能很好。

➤ **VSIP_ALG_TIME**: 优先最小化执行时间。

VSIP 开源库在实现时，会优先选择那些计算速度最快、执行时间最短的算法或实现策略，即使这可能会消耗更多的内存或在数值精度上做出轻微的妥协（只要在可接受范围内）。

VSIPL-FT 库在调用 FFT 运算的接口中，这个值对应的是 FFTW 规划器标志（planner flags）的 `FFTW_PATIENT`：这是 `FFTW_MEASURE` 的“更耐心”版本。它会尝试比 `MEASURE` 更多的算法和优化策略，进行更彻底的搜索和基准测试。创建 plan 的时间最长，执行性能最好。

➤ `VSIP_ALG_NOISE`：优先最小化计算噪声/最大化精度。

VSIPL 开源库在实现时，会优先选择那些数值精度最高、累积误差最小、数值最稳定的算法。这可能意味着使用更复杂的算法、避免可能导致精度损失的操作（如某些类型的缩放或舍入），即使这会牺牲一些速度或增加内存使用。

VSIPL-FT 库在调用 FFT 运算的接口中，这个值对应的是 FFTW 规划器标志（planner flags）的 `FFTW_ESTIMATE`：FFTW 不进行任何实际的性能测量。它基于一套内置的启发式规则（heuristics）和预设的模型，快速选择一个“合理”的算法来创建 plan。创建 plan 的时间较短，执行性能一般（可能较差）。

◆ `vsip_symmetry` 枚举用于指定滤波器和卷积核的对称性：

```
/* 对于滤波器和卷积核，指定核的对称性。 */
typedef enum
{
    VSIP_NONSYM = 0, /* 非对称 */
    VSIP_SYM_EVEN_LEN_ODD = 1, /* (偶数) 对称，奇数长度 */
    VSIP_SYM_EVEN_LEN_EVEN = 2 /* (偶数) 对称，偶数长度 */
} vsip_symmetry;
```

➤ `VSIP_NONSYM`：非对称。

➤ `VSIP_SYM_EVEN_LEN_ODD`：偶数对称，长度为奇数。

➤ `VSIP_SYM_EVEN_LEN_EVEN`：偶数对称，长度为偶数。

◆ `vsip_support_region` 枚举用于指定滤波器、卷积和相关性计算输出结果时考虑的区域：

```
/* 对于滤波器、卷积和相关性；指定输出结果计算的区域。 */
typedef enum
{
    VSIP_SUPPORT_FULL = 0, /* 最大区域 */
    VSIP_SUPPORT_SAME = 1, /* 输入和输出大小相同 */
    VSIP_SUPPORT_MIN = 2 /* 不对核进行零扩展的区域，参考 */
} vsip_support_region;
```

➤ `VSIP_SUPPORT_FULL`：最大区域，包括所有可能的输出。

➤ `VSIP_SUPPORT_SAME`：输入和输出大小相同，通常意味着输出大小与输入大小相同。

➤ `VSIP_SUPPORT_MIN`：最小区域，不包括通过零扩展核得到的输出区域，通

常作为参考实现。

3.9.1 快速傅里叶变换 (FFT)

- ◆ vsip_fft_dir 枚举用于指定是执行正向 FFT 还是逆向 FFT:

```
/* 枚举类型，用于指示正向或逆向 FFT。 */  
typedef enum  
{  
    VSIP_FFT_FWD = -1, /* 正向 */  
    VSIP_FFT_INV = 1 /* 逆向（或反向） */  
} vsip_fft_dir;
```

➤ VSIP_FFT_FWD: 正向 FFT。

➤ VSIP_FFT_INV: 逆向 FFT。

- ◆ vsip_fft_place 枚举用于指定 FFT 计算是原地 (In-Place) 还是非原地 (Out-of-Place) 进行:

```
/* 枚举类型，用于指示是否进行原地计算。 */  
typedef enum  
{  
    VSIP_FFT_IP = 0, /* 原地 */  
    VSIP_FFT_OP = 1 /* 非原地 */  
} vsip_fft_place;
```

➤ VSIP_FFT_IP: 原地计算，输出覆盖输入数据。

➤ VSIP_FFT_OP: 非原地计算，输出数据存储单独的数组中。

- ◆ vsip_fft_attr_p 结构用于定义 1-D FFT 对象的属性，包括输入和输出的长度、计算方式（原地或非原地）、缩放因子以及变换方向（正向或逆向）:

```
/* 1-D FFT 对象的属性结构。 */  
typedef struct  
{  
    vsip_scalar_vi input; /* 输入长度 */  
    vsip_scalar_vi output; /* 输出长度 */  
    vsip_fft_place place; /* 原地/非原地 */  
    vsip_scalar_p scale; /* 缩放因子 */  
    vsip_fft_dir dir; /* 正向或逆向 */  
} vsip_fft_attr_p;
```

- ◆ vsip_fftm_attr_p 结构用于定义多个 1-D FFT 对象的属性，包括输入和输出的多维大小、计算方式、缩放因子、变换方向以及主要维度（按行或按列）:

```
/* 多个 1-D FFT 的属性结构。 */  
typedef struct  
{  
    vsip_scalar_mi input; /* 输入大小，M x N */
```



```

vsip_scalar_mi output; /* 输出大小, P x Q */
vsip_fft_place place; /* 原地/非原地 */
vsip_scalar_p scale; /* 缩放因子 */
vsip_fft_dir dir; /* 正向或逆向 */
vsip_major major; /* 按行/列 */
}vsip_fftm_attr_p;

```

- ◆ vsip_fft2d_attr_p 结构用于定义 2-D FFT 对象的属性, 包括输入和输出的二维大小、计算方式、缩放因子以及变换方向:

```

/* 2-D FFT 对象的属性结构。 */
typedef struct
{
    vsip_scalar_mi input; /* 输入大小, M x N */
    vsip_scalar_mi output; /* 输出大小, P x Q */
    vsip_fft_place place; /* 原地/非原地 */
    vsip_scalar_p scale; /* 缩放因子 */
    vsip_fft_dir dir; /* 正向或逆向 */
}vsip_fft2d_attr_p;

```

- ◆ vsip_fft_p 类型用于表示 1-D FFT 对象:

```

/* 1-D FFT 的对象类型。 */
struct vsip_fftobject_p;
typedef struct vsip_fftobject_p vsip_fft_p;

```

- ◆ vsip_fftm_p 类型用于表示多个 1-D FFT 对象:

```

/* 多个 1-D FFT 的对象类型。 */
struct vsip_fftmobject_p;
typedef struct vsip_fftmobject_p vsip_fftm_p;

```

- ◆ vsip_fft2d_p 类型用于表示 2-D FFT 对象:

```

/* 2-D FFT 的对象类型。 */
struct vsip_fft2dobject_p;
typedef struct vsip_fft2d_object_p vsip_fft2d_p;

```

3.9.2 滤波 (Filtering)

```

/* FIR 滤波器的对象类型。 */
struct vsip_fiobject_p;
typedef struct vsip_fiobject_p vsip_fir_p;

/* 复数 FIR 滤波器的对象类型。 */
struct vsip_cfirobject_p;
typedef struct vsip_cfirobject_p vsip_cfir_p;

/* 指示滤波器对象是否应保存状态信息的枚举类型。 */
typedef enum

```

```
{
    VSIP_STATE_NO_SAVE = 1, /* 不保存状态, 单次调用滤波器 */
    VSIP_STATE_SAVE = 2 /* 保存状态以进行连续滤波 */
}vsip_obj_state;

/* 实数 FIR 滤波器的属性结构。 */
typedef struct
{
    vsip_scalar_vi kernel_len; /* 核长度 */
    vsip_symmetry symm; /* 核的对称性 */
    vsip_scalar_vi in_len; /* 滤波器输入段长度 */
    vsip_scalar_vi out_len; /* 滤波器输出段长度 */
    vsip_length decimation; /* 降采样因子 */
    vsip_obj_state state; /* 保存状态信息 */
}vsip_fir_attr_f;

/* 复数 FIR 滤波器的属性结构。 */
typedef struct
{
    vsip_scalar_vi kernel_len; /* 核长度 */
    vsip_symmetry symm; /* 核的对称性 */
    vsip_scalar_vi in_len; /* 滤波器输入段长度 */
    vsip_scalar_vi out_len; /* 滤波器输出段长度 */
    vsip_length decimation; /* 降采样因子 */
    vsip_obj_state state; /* 保存状态信息 */
} vsip_cfir_attr_f;
```

这些定义描述了不同类型的滤波器对象及其属性结构。vsip_fir_p 和 vsip_cfir_p 分别用于表示实数和复数 FIR 滤波器。每个滤波器对象都有一些共同的属性,如核长度、对称性、输入段长度、输出段长度和降采样因子。此外,还有一个 vsip_obj_state 枚举类型,用于指示是否保存滤波器的状态信息。实数和复数 FIR 滤波器的属性结构还包括了输入和输出段的长度。这些属性结构为滤波器的创建和操作提供了必要的参数。

3.9.3 卷积 (Convolution)

```
/* 1-D 卷积的对象类型。 */
struct vsip_conv1dobject_p;
typedef struct vsip_conv1dobject_p vsip_conv1d_p;

/* 2-D 卷积的对象类型。 */
struct vsip_conv2dobject_p;
typedef struct vsip_conv2dobject_p vsip_conv2d_p;
```

```
/* 1-D 卷积的属性结构。 */
typedef struct
{
    vsip_scalar_vi kernel_len; /* 核长度, M */
    vsip_symmetry symm; /* 核的对称性 */
    vsip_scalar_vi data_len; /* 数据输入长度 */
    vsip_support_region support; /* 支持输出区域 */
    vsip_scalar_vi out_len; /* 输出长度 */
    vsip_length decimation; /* 输出降采样因子, D */
} vsip_conv1d_attr_f;

/* 2-D 卷积的属性结构。 */
typedef struct
{
    vsip_scalar_mi kernel_size; /* 核大小, M by N */
    vsip_symmetry symm; /* 核的对称性 */
    vsip_scalar_mi in_size; /* 数据输入大小, P by Q */
    vsip_support_region support; /* 支持输出区域 */
    vsip_scalar_mi out_size; /* 输出大小, S by T */
    vsip_length decimation; /* 输出降采样因子, D */
} vsip_conv2d_attr_f;
```

这些定义描述了用于1-D和2-D卷积的对象类型及其属性结构。vsip_conv1d_p和vsip_conv2d_p分别用于表示一维和二维卷积操作。每个卷积操作都有一些共同的属性,如核长度、对称性、数据输入长度、输出长度和降采样因子。此外,vsip_conv1d_attr_f和vsip_conv2d_attr_f结构还包含了支持输出区域和核大小。这些属性结构为卷积操作的创建和执行提供了必要的参数。

3.9.4 相关性 (Correlation)

```
/* 1-D 相关性的对象类型。 */
struct vsip_corr1dobject_p;
typedef struct vsip_corr1dobject_p vsip_corr1d_p;

/* 复数 1-D 相关性的对象类型。 */
struct vsip_ccorr1dobject_p;
typedef struct vsip_ccorr1dobject_p vsip_ccorr1d_p;

/* 指示相关性估计是偏置的还是无偏的枚举类型。 */
typedef enum
{
    VSIP_BIASED = 0, /* 偏置 */
    VSIP_UNBIASED = 1 /* 无偏 */
}
```

```
}vsip_bias;

/* 1-D 相关性的属性结构。 */
typedef struct
{
    vsip_scalar_vi ref_len; /* 参考长度 */
    vsip_scalar_vi data_len; /* 数据输入长度 */
    vsip_support_region support; /* 支持输出区域 */
    vsip_scalar_vi lag_len; /* 输出（滞后）长度 */
}vsip_corr1d_attr_p;

/* 1-D 复数相关性的属性结构。 */
typedef struct
{
    vsip_scalar_vi ref_len; /* 参考长度 */
    vsip_scalar_vi data_len; /* 数据输入长度 */
    vsip_support_region support; /* 支持输出区域 */
    vsip_scalar_vi lag_len; /* 输出（滞后）长度 */
}vsip_ccorr1d_attr_p;
```

3.9.5 直方图（Histogram）

```
/* 直方图累加选项。 */
typedef enum
{
    VSIP_HIST_RESET = 1, /* 直方图重置 */
    VSIP_HIST_ACCUM = 2 /* 直方图累加 */
}vsip_hist_opt;
```

◆ vsip_hist_opt 枚举用于指定直方图操作的选项：

- VSIP_HIST_RESET：重置直方图，将所有计数设置为零。
- VSIP_HIST_ACCUM：累加直方图，将当前计数添加到累加器中。

这些选项通常用于图像处理和信号处理中，以便在直方图分析中根据需要重置或累加直方图数据。

3.10 线性代数

```
/* 矩阵变换操作。 */
typedef enum
{
    VSIP_MAT_NTRANS = 0, /* 无变换 */
    VSIP_MAT_TRANS = 1, /* 矩阵转置 */
    VSIP_MAT_HERM = 2, /* 矩阵厄米（共轭转置） */
}
```

```
VSIP_MAT_CONJ = 3 /* 矩阵共轭 */
}vsip_mat_op;
```

◆ vsip_mat_op 枚举用于指定矩阵变换操作：

- VSIP_MAT_NTRANS：表示没有变换。
- VSIP_MAT_TRANS：表示执行矩阵转置。
- VSIP_MAT_HERM：表示执行矩阵的厄米转置，即共轭转置。
- VSIP_MAT_CONJ：表示执行矩阵的共轭。

这些操作通常用于线性代数和数值分析中，以便根据需要对矩阵进行特定的变换。

3.10.1 LU 分解

```
/* LU 矩阵分解的对象属性结构。 */
typedef struct
{
    vsip_length n; /* 矩阵大小为 N by N */
}vsip_lu_attr_f;

/* 复数 LU 矩阵分解的对象属性结构。 */
typedef struct
{
    vsip_length n; /* 矩阵大小为 N by N */
}vsip_clu_attr_f;

/* LU 矩阵分解的对象类型。 */
struct vsip_luobject_f;
typedef struct vsip_luobject_f vsip_lu_f;

/* 复数 LU 矩阵分解的对象类型。 */
struct vsip_cluobject_f;
typedef struct vsip_cluobject_f vsip_clu_f;
```

3.10.2 Cholesky 分解

```
/* Cholesky 矩阵的对象属性结构。 */
typedef struct
{
    vsip_length n; /* 矩阵大小为 N by N */
}vsip_chol_attr_f;

/* 复数 Cholesky 矩阵的对象属性结构。 */
typedef struct
{

```

```
    vsip_length n; /* 矩阵大小为 N by N */
}vsip_cchol_attr_f;

/* Cholesky 矩阵系统的对象类型。 */
struct vsip_choldobject_f;
typedef struct vsip_choldobject_f vsip_chol_f;

/* 复数 Cholesky 矩阵系统的对象类型。 */
struct vsip_ccholdobject_f;
typedef struct vsip_ccholdobject_f vsip_cchol_f;
```

3.10.3 QR 分解

```
/* 指示 QR 分解中是否应用矩阵 Q 的枚举类型。 */
typedef enum
{
    VSIP_MAT_LSIDE = 0, /* 左侧 */
    VSIP_MAT_RSIDE = 1 /* 右侧 */
}vsip_mat_side;

/* 指示矩阵 Q 是否保留的枚举类型。 */
typedef enum
{
    VSIP_QRD_NOSAVEQ = 0, /* 不保存 Q */
    VSIP_QRD_SAVEQ = 1, /* 保存 Q */
    VSIP_QRD_SAVEQ1 = 2 /* 保存瘦 Q */
}vsip_qrd_qopt;

/* 选择是解决协方差问题还是线性最小二乘问题。 */
typedef enum
{
    VSIP_COV = 0, /* 解协方差线性系统问题 */
    VSIP_LLS = 1 /* 解线性最小二乘问题 */
}vsip_qrd_prob;

/* QR 分解的对象属性结构。 */
typedef struct
{
    vsip_length m; /* 输入矩阵是 M by N */
    vsip_length n; /* 输入矩阵是 M by N */
    vsip_qrd_qopt Qopt; /* 矩阵 Q 是否保存 */
}vsip_qr_attr_f;
```

```
/* 复数 QR 分解的对象属性结构。 */
typedef struct
{
    vsip_length m; /* 输入矩阵是 M by N */
    vsip_length n; /* 输入矩阵是 M by N */
    vsip_qrd_qopt Qopt; /* 矩阵 Q 是否保存 */
}vsip_cqr_attr_f;

/* QR 矩阵分解的对象类型。 */
struct vsip_qrobject_f;
typedef struct vsip_qrobject_f vsip_qr_f;

/* 复数 QR 矩阵分解的对象类型。 */
struct vsip_cqrobject_f;
typedef struct vsip_cqrobject_f vsip_cqr_f;
```

3.10.4 SVD 分解

```
/* 指示是否计算 U/V 的列/行的枚举类型。 */
typedef enum
{
    VSIP_SVD_UVNOS = 0, /* 不计算 U/V 的列/行 */
    VSIP_SVD_UVFULL = 1, /* 计算 U/V 的所有列/行 */
    VSIP_SVD_UVPART = 2 /* 计算 U/V 的前 min{M, N}列/行 */
}vsip_svd_uv;

/* 实数 SVD 矩阵的对象属性结构。 */
typedef struct
{
    vsip_length m; /* 输入矩阵是 M by N */
    vsip_length n; /* 输入矩阵是 M by N */
    vsip_svd_uv Usave; /* 计算的 U 列 */
    vsip_svd_uv Vsave; /* 计算的 V 列 */
}vsip_sv_attr_f;

/* 复数 SVD 矩阵的对象属性结构。 */
typedef struct
{
    vsip_length m; /* 输入矩阵是 M by N */
    vsip_length n; /* 输入矩阵是 M by N */
    vsip_svd_uv Usave; /* 计算的 U 列 */
    vsip_svd_uv Vsave; /* 计算的 V 列 */
}vsip_csv_attr_f;
```

```
/* SVD 矩阵的对象类型。 */
struct vsip_svector_f;
typedef struct vsip_svector_f vsip_sv_f;

/* 复数 SVD 矩阵的对象类型。 */
struct vsip_csvdobject_f;
typedef struct vsip_csvdobject_f vsip_csv_f;
```

这些定义描述了用于奇异值分解（SVD）的对象类型及其属性结构。vsip_sv_f 和 vsip_csv_f 分别用于表示实数和复数 SVD 矩阵。每个 SVD 分解都有一些共同的属性，如矩阵的大小（MxN）、计算的 U 列和 V 列的数量。vsip_svd_uv 枚举类型用于指示是否计算 U/V 的所有列/行，或者只计算前 min{M,N} 列/行。这些属性结构为 SVD 分解的创建和执行提供了必要的参数。

3.11 直接数据访问（Direct Data Access）

3.11.1 同步策略（Synchronization Policy）

如果 DDA 通过 DDA 缓冲区提供对视图（view）的访问，则必须在 DDA 缓冲区和视图内存之间进行数据同步。同步发生的时间以及数据从/向何处同步，由同步策略（synchronization policy）指定。

视图可以通过 DDA 以以下三种方式访问：

- 输入（只读，input/read-only）
- 输出（只写，output/write-only）
- 输入和输出（读写，input and output/read-write）

此外，可以明确指示实现通过 DDA 缓冲区访问视图，而不论其原本是否需要这样做。

同步策略的命名是从外部操作使用 DDA 指针的角度来定义的。例如，一个输入（只读）同步策略表示：外部操作应仅将 DDA 指针用作输入。

```
typedef unsigned int vsip_dda_sync_policy;
static vsip_dda_sync_policy const VSIP_DDA_SYNC_POLICY_IN = 0x01; //
输入（只读）
static vsip_dda_sync_policy const VSIP_DDA_SYNC_POLICY_OUT = 0x02;
// 输出（只写）
static vsip_dda_sync_policy const VSIP_DDA_SYNC_POLICY_INOUT =
0x03; // 输入输出（读写）
static vsip_dda_sync_policy const VSIP_DDA_SYNC_POLICY_COPY = 0x04;
// 强制复制
```

策略说明：

- 策略标志可以按位或（OR）组合使用。
- 一个既没有设置 VSIP_DDA_SYNC_POLICY_IN 也没有设置 VSIP_DDA_SYNC_POLICY_OUT 的策略是无效的。

各策略详细行为：

- VSIP_DDA_SYNC_POLICY_IN（输入/只读）：表示视图被当作输入使用。
 - 在数据对象创建时，数据会从视图内存 → DDA 缓冲区自动同步。
 - 可通过调用 vsip_dda_dksync_in_p() 函数按需手动同步（视图 → DDA 缓冲区）。
- VSIP_DDA_SYNC_POLICY_OUT（输出/只写）：表示视图被当作输出使用。
 - 在数据对象销毁时，数据会从 DDA 缓冲区 → 视图内存自动同步。
 - 可通过调用 vsip_dda_dksync_out_p() 函数按需手动同步（DDA 缓冲区 → 视图）。
 - 如果使用了 VSIP_DDA_SYNC_POLICY_OUT 策略，并且使用了 DDA 缓冲区（无论显式或隐式），则该 DDA 缓冲区可能未被初始化，应被视为只写区域。
- VSIP_DDA_SYNC_POLICY_INOUT（输入输出/读写）：表示视图被当作输入和输出使用。
 - 在数据对象创建时：视图内存 → DDA 缓冲区 同步。
 - 在数据对象销毁时：DDA 缓冲区 → 视图内存 同步。
 - 可通过以下方式按需同步：
 - ✓vsip_dda_dksync_in_p()：从视图内存 → DDA 缓冲区
 - ✓vsip_dda_dksync_out_p()：从 DDA 缓冲区 → 视图内存
- VSIP_DDA_SYNC_POLICY_COPY（强制复制）：表示应将视图复制到一个独立的 DDA 缓冲区中，即使实现本可以直接访问视图内存（即绕过缓冲区），也强制使用缓冲区进行复制。

4 函数说明

4.1 命名规则

函数命名规则

函数命名规则遵循 VSIPL 规范命名通用格式：

[vsip]_<name>_<datatype>(<parameters>)。

例如：

```
void vsip_cvadd_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r),
```

其中：

- ◆ 前缀为 vsip(信号库)；
- ◆ cvadd 是函数名，表示该函数实现的功能是复数向量加法运算；
- ◆ f 表示该函数所有入参的数据类型为 float；
- ◆ 圆括号里面为该接口传入的参数。

函数名<name>

函数名表明该函数的主要功能，格式为：<name>=[modifier]<operation>。其中：

- ◆ modifier 通常由处理的数据结构类型的缩写组成。
- ◆ operation 通常由一个或多个单词、首字母缩略词、缩写组成，描述该函数的基本功能。

函数数据结构类型的说明与命名规则部分如表 4-1 所示，详见[数据类型](#)。

表 4-1 数据结构类型的说明与命名规则

数据结构类型	说明	缩写规则
complex	复数	c

数据结构类型	说明	缩写规则
scalar	标量	s
vector	向量	v
matrices	矩阵	m

数据类型<datatype>

数据类型表明该函数处理的数据类型，通常为函数参数的数据类型。VSIPL-FT 中所使用的数据类型请参见[数据类型](#)。

例如，vsip_cvadd_f 计算两个数据类型 32 位浮点型复数的源向量的加法运算，并将计算结果存储在 32 位浮点型复数的目标向量中。

函数参数<parameters>

<parameters>元素指定该函数的所有参数。参数的排列顺序按如下规则：

- 1.源操作数，通常是标量、向量或者矩阵视图的对象指针。
- 2.目标操作数。

4.2 函数介绍

VSIPL-FT 函数分类如表 4-2 所示：

表 4-2 VSIPL-FT 函数分类

名称	说明
支撑函数	该模块包括初始化和完成函数，以及创建、销毁和操作 VSIPL-FT 块和各视图对象的函数。
标量函数	该模块包括实标量函数、复数标量函数。
随机数函数	该模块包括 VSIPL-FT 所有的随机数生成相关函数。
向量&元素运算函数	该模块包括对单个元素或元素对执行简单操作的函数，比如初等数学函数、逻辑函数等等。
线性代数函数	该模块包括矩阵与矢量运算、矩阵与矩阵运算函数。
信号处理函数	该模块包括 FFT 函数、卷积函数、窗函数以及滤波函数。
插值函数	该模块包括插值相关的函数。
置换函数	该模块包括置换相关的函数。
直接数据访问（DDA）函数	该模块包括直接数据访问(DDA)相关的函数。

4.2.1 VSIPL-FT 支撑函数（新增 18 个）

4.2.1.1 函数介绍

本节介绍 VSIPL-FT 所需的支撑函数。这些支撑函数包括初始化、创建、销毁以及操作 VSIPL-FT 块和多种视图对象的函数。支撑函数分为五个部分，分别描述库初始化和完成函数、数组和块对象函数、向量函数、矩阵函数和张量函数。

4.2.1.2 初始化/完成操作

4.2.1.2.1 vsip_init

提供初始化，允许实现分配和设置任何全局状态，并准备支持用户使用 VSIPL-FT 功能。

接口：

```
int vsip_init(void *r);
```

参数：

空。

返回值：

如果初始化成功，则返回 0，否则返回非零。

4.2.1.2.2 vsip_finalize

提供清理并释放 VSIPL-FT 使用的资源(如果是一系列嵌套调用的最后一个)，允许实现保证在调用完成后不再使用 vsip_init 分配的任何资源。

接口：

```
int vsip_finalize(void *r);
```

参数：

空。

返回值：

如果完成成功，则返回 0，否则返回非零。

4.2.1.3 数组和块对象函数（新增 4 个）

4.2.1.3.1 vsip_dblockadmit_p

接纳用于 VSIPL-FT 操作的 VSIPL-FT 块。

接口:

```
int vsip_blockadmit_vi(vsip_block_vi *block, vsip_scalar_bl update);
int vsip_blockadmit_mi(vsip_block_mi *block, vsip_scalar_bl update);
int vsip_blockadmit_bl(vsip_block_bl *block, vsip_scalar_bl update);
int vsip_blockadmit_d(vsip_block_d *block, vsip_scalar_bl update);
int vsip_blockadmit_f(vsip_block_f *block, vsip_scalar_bl update);
int vsip_cblockadmit_d(vsip_cblock_d *block, vsip_scalar_bl update);
int vsip_cblockadmit_f(vsip_cblock_f *block, vsip_scalar_bl update);
int vsip_blockadmit_i(vsip_block_i *block, vsip_scalar_bl update);
int vsip_blockadmit_si(vsip_block_si *block, vsip_scalar_bl update);
int vsip_blockadmit_uc(vsip_block_uc *block, vsip_scalar_bl update);
```

参数:

block: 指向块对象的指针。

update: Boolean 标志, 其中 true 指示在修改期间必须维护数据数组值。

返回值:

成功时返回 0, 失败时返回非 0。

4.2.1.3.2 vsip_blockbind_p (新增 2 个)

创建 VSIPL-FT 块并将其绑定到用户分配的(用户数据阵列)内存。

接口:

```
vsip_block_vi *vsip_blockbind_vi(vsip_scalar_vi *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_mi *vsip_blockbind_mi(vsip_scalar_vi *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_bl *vsip_blockbind_bl(vsip_scalar_bl *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_c *vsip_blockbind_c(vsip_scalar_c *data, vsip_length N,
vsip_memory_hint hint); (新增接口)
vsip_block_d *vsip_blockbind_d(vsip_scalar_d *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_f *vsip_blockbind_f(vsip_scalar_f *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_i *vsip_blockbind_i(vsip_scalar_i *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_si *vsip_blockbind_si(vsip_scalar_si *data, vsip_length N,
vsip_memory_hint hint);
vsip_block_uc *vsip_blockbind_uc(vsip_scalar_uc *data, vsip_length N,
```

```
vsip_memory_hint hint);
    vsip_block_u *vsip_blockbind_u(vsip_scalar_u *data, vsip_length N,
vsip_memory_hint hint); (新增接口)
```

参数:

data: 指向包含至少 N 个 vsip_scalar_p 元素的连续内存的数据数组的指针。

N: 用户块 vsip_block_p 绑定到的用户数据数组的元素数。

hint: 内存运用方式。

返回值:

返回 vsip_block_p 类型的指针, 如果块绑定失败, 则返回 NULL。

4.2.1.3.3 vsip_cblockbind_p

创建 VSIPL-FT 复数块并将其绑定到用户分配的(用户数据数组)内存。

接口:

```
vsip_cblock_d *vsip_cblockbind_d(vsip_scalar_d *data1, vsip_scalar_d *data2,
vsip_length N, vsip_memory_hint hint);
```

```
vsip_cblock_f *vsip_cblockbind_f(vsip_scalar_f *data1, vsip_scalar_f *data2,
vsip_length N, vsip_memory_hint hint);
```

参数:

data1: 1 如果 data2 为 NULL, 则 data1 是指向至少包含 2n 个 vsip_scalar_p 元素的连续内存的数据数组的指针。数据数组的偶数元素包含实部值, 奇数元素包含虚部值。数据以实部虚部交错形式存储。请注意, 第一个元素被认为是偶数, 因为索引值从零开始。如果 data2 不为 NULL, 则 data1 是指向包含至少 N 个 vsip_scalar_p 连续内存的数据数组的指针。

data2: 如果 data2 为 NULL, 则数据以交错复数形式存储。如果 data2 不为 NULL, 则它是指向至少包含 n 个 vsip_scalar_p 元素的连续内存的数据数组的指针。数据数组包含虚部值。数据以实部虚部分离的复数形式存储。

N: vsip_cblock_p 类型的用户块绑定到的用户数据数组的复数元素数。

hint: 内存运用方式。

返回值:

返回 vsip_cblock_p 类型的指针, 如果块绑定失败, 则返回 NULL。

4.2.1.3.4 vsip_dblockcreate_p (新增 1 个)

创建 VSIPL-FT 块并将(VSIPL-FT 分配的)数据数组(内存)绑定到它。

接口:

```
vsip_block_vi *vsip_blockcreate_vi(vsip_length N, vsip_memory_hint hint);
```

```
vsip_block_mi *vsip_blockcreate_mi(vsip_length N,vsip_memory_hint hint);
vsip_block_bl *vsip_blockcreate_bl(vsip_length N,vsip_memory_hint hint);
vsip_block_d *vsip_blockcreate_d(vsip_length N,vsip_memory_hint hint);
vsip_block_f *vsip_blockcreate_f(vsip_length N,vsip_memory_hint hint);
vsip_cblock_d *vsip_cblockcreate_d(vsip_length N,vsip_memory_hint hint);
vsip_cblock_f *vsip_cblockcreate_f(vsip_length N,vsip_memory_hint hint);
vsip_block_i *vsip_blockcreate_i(vsip_length N,vsip_memory_hint hint);
vsip_block_si *vsip_blockcreate_si(vsip_length N,vsip_memory_hint hint);
vsip_block_uc *vsip_blockcreate_uc(vsip_length N,vsip_memory_hint hint);
vsip_block_u *vsip_blockcreate_u(vsip_length N,vsip_memory_hint hint); ( 新增接口 )
```

参数:

N: 要为数据数组分配的元素数。

hint: 内存运用方式。

返回值:

返回 vsip_cblock_p 类型的指针, 如果块绑定失败, 则返回 NULL。

4.2.1.3.5 vsip_dblockdestroy_p (新增 1 个)

销毁(释放)VSIPL-FT 块对象以及 VSIPL-FT 为其分配的任何数据数组。

接口:

```
void vsip_blockdestroy_vi(vsip_block_vi *block);
void vsip_blockdestroy_mi(vsip_block_mi *block);
void vsip_blockdestroy_bl(vsip_block_bl *block);
void vsip_blockdestroy_d( vsip_block_d *block);
void vsip_blockdestroy_f( vsip_block_f *block);
void vsip_cblockdestroy_d(vsip_cblock_d *block);
void vsip_cblockdestroy_f(vsip_cblock_f *block);
void vsip_blockdestroy_i(vsip_block_i *block);
void vsip_blockdestroy_si(vsip_block_si *block);
void vsip_blockdestroy_uc(vsip_block_uc *block);
void vsip_blockdestroy_u(vsip_block_u *block); ( 新增接口 )
```

参数:

Block: 指向块对象的指针。

返回值:

空。

4.2.1.3.6 vsip_blockfind_p

查找指向绑定到 VSIPL-FT 释放的块对象的数据的指针。

接口：

```
vsip_scalar_vi *vsip_blockfind_vi(const vsip_block_vi *block);  
vsip_scalar_vi *vsip_blockfind_mi(const vsip_block_mi *block);  
vsip_scalar_bl *vsip_blockfind_bl(const vsip_block_bl *block);  
vsip_scalar_d *vsip_blockfind_d(const vsip_block_d *block);  
vsip_scalar_f *vsip_blockfind_f(const vsip_block_f *block);  
vsip_scalar_i *vsip_blockfind_i(const vsip_block_i *block);  
vsip_scalar_si *vsip_blockfind_si(const vsip_block_si *block);  
vsip_scalar_uc *vsip_blockfind_uc(const vsip_block_uc *block);
```

参数：

block：指向块对象的指针。

返回值：

返回指向已释放块的数据数组的 vsip_scalar_p 类型的指针；如果块对象未处于已释放状态，则返回 NULL。请注意，如果块被释放并绑定到 NULL，则还返回空指针。

4.2.1.3.7 vsip_cblockfind_p

返回指向绑定到 VSIPL-FT 释放的复数块对象 vsip_cblock_p 的用户数据数组的指针，如果复数块对象数据处于允许状态，则返回 NULL。

接口：

```
void vsip_cblockfind_d(const vsip_cblock_d *block, vsip_scalar_d **data1,  
vsip_scalar_d **data2);  
void vsip_cblockfind_f(const vsip_cblock_f *block, vsip_scalar_f **data1,  
vsip_scalar_f **data2);
```

参数：

block：指向块对象的指针。

data1：输出-指向指向数据数组或实际值的数据数组的指针。如果复数块处于允许状态，则返回 NULL。

data2：输出-指向空指针或虚值的前一个数据数组的指针。如果复数块处于允许状态或前一个绑定是复数实部虚部交错不分离形式，则返回 NULL。

返回值：

空。

4.2.1.3.8 vsip_blockrebind_p

将 VSIPL-FT 块重新绑定到用户指定的数据。

接口：

```
vsip_scalar_vi *vsip_blockrebind_vi(vsip_block_vi *block, vsip_scalar_vi *data);
vsip_scalar_vi *vsip_blockrebind_mi(vsip_block_mi *block, vsip_scalar_vi *data);
vsip_scalar_bl *vsip_blockrebind_bl(vsip_block_bl *block, vsip_scalar_bl *data);
vsip_scalar_d *vsip_blockrebind_d(vsip_block_d *block, vsip_scalar_d *data);
vsip_scalar_f *vsip_blockrebind_f(vsip_block_f *block, vsip_scalar_f *data);
vsip_scalar_i *vsip_blockrebind_i(vsip_block_i *block, vsip_scalar_i *data);
vsip_scalar_si *vsip_blockrebind_si(vsip_block_si *block, vsip_scalar_si *data);
vsip_scalar_uc *vsip_blockrebind_uc(vsip_block_uc *block, vsip_scalar_uc
*data);
```

参数：

block：指向块对象的指针。

data：指向包含至少 N 个 vsip_scalar_p 元素的连续内存的用户数据数组的指针，以绑定到块。

返回值：

返回指向绑定到块的上一个数据数组的指针，如果块处于允许状态，则返回 NULL。

4.2.1.3.9 vsip_cblockrebind_p

将 VSIPL-FT 复数块重新绑定到用户指定的数据。

接口：

```
void vsip_cblockrebind_d(vsip_cblock_d *block, vsip_scalar_d *data1,
vsip_scalar_d *data2, vsip_scalar_d **prevdata1, vsip_scalar_d **prevdata2);
void vsip_cblockrebind_f(vsip_cblock_f *block, vsip_scalar_f *data1,
vsip_scalar_f *data2, vsip_scalar_f **prevdata1, vsip_scalar_f **prevdata2);
```

参数：

block：指向块对象的指针。

data1: 1 如果 data2 为 NULL，则 data1 是指向至少包含 2n 个 vsip_scalar_p 元素的连续内存的数据数组的指针。数据数组的偶数元素包含实部值，奇数元素包含虚部值。数据以实部虚部交错形式存储。请注意，第一个元素被认为是偶数，因为索引值从零开始。如果 data2 不为 NULL，则 data1 是指向包含至少 N 个 vsip_scalar_p 的连续内存的数据数组的指针。

data2：如果 data2 为 NULL，则数据以交错复数形式存储。如果 data2 不为

NULL, 则它是指向至少包含 n 个 `vsip_scalar_p` 元素的连续内存的数据数组的指针。数据数组包含虚部值。数据以实部虚部分离的复数形式存储。

`prevdata1`: 输出-指向上一个用户数据数组或实部值的用户数据数组的指针。如果复数块处于允许状态, 则返回 NULL。

`prevdata2`: 输出-指向空指针或虚值的前一个用户数据数组的指针。如果复数块处于允许状态或前一个绑定是复数交错形式, 则返回 NULL。

返回值:

空。

4.2.1.3.10 vsip_blockrelease_p

释放用于直接用户访问的 VSIPL-FT 块。

接口:

```
vsip_scalar_vi *vsip_blockrelease_vi( vsip_block_vi *block, vsip_scalar_bl  
update);
```

```
vsip_scalar_vi *vsip_blockrelease_mi( vsip_block_mi *block, vsip_scalar_bl  
update);
```

```
vsip_scalar_bl *vsip_blockrelease_bl( vsip_block_bl *block, vsip_scalar_bl  
update);
```

```
vsip_scalar_d *vsip_blockrelease_d( vsip_block_d *block, vsip_scalar_bl  
update);
```

```
vsip_scalar_f *vsip_blockrelease_f( vsip_block_f *block, vsip_scalar_bl update);
```

```
vsip_scalar_i *vsip_blockrelease_i( vsip_block_i *block, vsip_scalar_bl update);
```

```
vsip_scalar_si *vsip_blockrelease_si( vsip_block_si *block, vsip_scalar_bl  
update);
```

```
vsip_scalar_uc *vsip_blockrelease_uc( vsip_block_uc *block, vsip_scalar_bl  
update);
```

参数:

`block`: 指向块对象的指针。

`update`: Boolean 标志, 其中 true 指示在修改期间必须维护数据数组值。

返回值:

如果块释放失败, 则返回 NULL, 否则返回指向数据数组的指针。

4.2.1.3.11 vsip_cblockrelease_p

从 VSIPL-FT 释放一个复数的块直接用户访问。

接口:

```
void vsip_cblockrelease_d(vsip_cblock_d *block, vsip_scalar_bl update,
vsip_scalar_d **data1, vsip_scalar_d **data2);
void vsip_cblockrelease_f(vsip_cblock_f *block, vsip_scalar_bl update,
vsip_scalar_f **data1, vsip_scalar_f **data2);
```

参数:

block: 指向块对象的指针。

update: Boolean 标志, 其中 true 指示在修改期间必须维护数据数组值。

data1: 1 如果 data2 为 NULL, 则 data1 是指向至少包含 2n 个 vsip_scalar_p 元素的连续内存的数据数组的指针。数据数组的偶数元素包含实部值, 奇数元素包含虚部值。数据以实部虚部交错形式存储。请注意, 第一个元素被认为是偶数, 因为索引值从零开始。如果 data2 不为 NULL, 则 data1 是指向包含至少 N 个 vsip_scalar_p 连续内存的数据数组的指针。

data2: 如果 data2 为 NULL, 则数据以交错复数形式存储。如果 data2 不为 NULL, 则它是指向至少包含 n 个 vsip_scalar_p 元素的连续内存的数据数组的指针。数据数组包含虚部值。数据以实部虚部分离的复数形式存储。

返回值:

空。

4.2.1.3.12 vsip_cstorage_p

返回复数存储格式, 即实部虚部交错、实部虚部分离或 none 用于此简化的精度类型。

接口:

```
vsip_cmplx_mem vsip_cstorage(void);
```

参数:

空。

返回值:

对应于存储格式的枚举类型。

4.2.1.4 矢量视图对象函数 (新增 14 个)

4.2.1.4.1 vsip_dvalldestroy_p (新增 2 个)

销毁(释放)向量、其关联块以及绑定到该块的任何 VSIPL-FT 数据数组。

接口:

```
void vsip_valldestroy_u(vsip_vview_u *v); (新增接口)
```

```
void vsip_valldestroy_c(vsip_vview_c *v); (新增接口)
void vsip_valldestroy_vi(vsip_vview_vi *v);
void vsip_valldestroy_mi(vsip_vview_mi *v);
void vsip_valldestroy_bl(vsip_vview_bl *v);
void vsip_valldestroy_d(vsip_vview_d *v);
void vsip_valldestroy_f(vsip_vview_f *v);
void vsip_valldestroy_i(vsip_vview_i *v);
void vsip_valldestroy_si(vsip_vview_si *v);
void vsip_valldestroy_uc(vsip_vview_uc *v);
void vsip_cvalldestroy_d(vsip_cvview_d *v);
void vsip_cvalldestroy_f(vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

空。

4.2.1.4.2 vsip_dvbind_p (新增 2 个)

创建矢量视图对象并将其绑定到块对象。

接口:

```
vsip_vview_u *vsip_vbind_u(const vsip_block_u *block, vsip_offset offset,
vsip_stride stride, vsip_length length); (新增接口)
vsip_vview_c *vsip_vbind_c(const vsip_block_c *block, vsip_offset offset,
vsip_stride stride, vsip_length length); (新增接口)
vsip_vview_vi *vsip_vbind_vi(const vsip_block_vi *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_mi *vsip_vbind_mi(const vsip_block_mi *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_bl *vsip_vbind_bl(const vsip_block_bl *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_d *vsip_vbind_d(const vsip_block_d *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_f *vsip_vbind_f(const vsip_block_f *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_i *vsip_vbind_i(const vsip_block_i *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
vsip_vview_si *vsip_vbind_si(const vsip_block_si *block, vsip_offset offset,
vsip_stride stride, vsip_length length);
```

```
vsip_vview_uc *vsip_vbind_uc(const vsip_block_uc *block, vsip_offset offset,  
vsip_stride stride, vsip_length length);  
vsip_cvview_d *vsip_cvbind_d(const vsip_cblock_d *block, vsip_offset offset,  
vsip_stride stride, vsip_length length);  
vsip_cvview_f *vsip_cvbind_f(const vsip_cblock_f *block, vsip_offset offset,  
vsip_stride stride, vsip_length length);
```

参数:

block: 指向块对象的指针。

offset: 元素中相对于 block 对象基础的矢量视图偏移量。

stride: 标量元素之间的矢量视图步幅。

length: 矢量视图的元素长度。

返回值:

该函数返回一个指向创建的矢量视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.4.3 vsip_dvcloneview_p

创建矢量视图的克隆。

接口:

```
vsip_vview_vi *vsip_vcloneview_vi(const vsip_vview_vi *v);  
vsip_vview_mi *vsip_vcloneview_mi(const vsip_vview_mi *v);  
vsip_vview_bl *vsip_vcloneview_bl(const vsip_vview_bl *v);  
vsip_vview_d *vsip_vcloneview_d(const vsip_vview_d *v);  
vsip_vview_f *vsip_vcloneview_f(const vsip_vview_f *v);  
vsip_vview_i *vsip_vcloneview_i(const vsip_vview_i *v);  
vsip_vview_si *vsip_vcloneview_si(const vsip_vview_si *v);  
vsip_vview_uc *vsip_vcloneview_uc(const vsip_vview_uc *v);  
vsip_cvview_d *vsip_cvcloneview_d(const vsip_cvview_d *v);  
vsip_cvview_f *vsip_cvcloneview_f(const vsip_cvview_f *v);
```

参数:

v: 源矢量视图对象。

返回值:

返回指向创建的矢量视图对象克隆的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.4.4 vsip_dvcreate_p (新增 2 个)

创建块对象和块的矢量视图(对象)。

接口:

```
vsip_vview_u *vsip_vcreate_u(vsip_length N,vsip_memory_hint hint); (新增接口)
vsip_vview_c *vsip_vcreate_c(vsip_length N,vsip_memory_hint hint); (新增接口)
vsip_vview_vi *vsip_vcreate_vi(vsip_length N,vsip_memory_hint hint);
vsip_vview_mi *vsip_vcreate_mi(vsip_length N,vsip_memory_hint hint);
vsip_vview_bl *vsip_vcreate_bl(vsip_length N,vsip_memory_hint hint);
vsip_vview_d *vsip_vcreate_d(vsip_length N,vsip_memory_hint hint);
vsip_vview_f *vsip_vcreate_f(vsip_length N,vsip_memory_hint hint);
vsip_vview_i *vsip_vcreate_i(vsip_length N,vsip_memory_hint hint);
vsip_vview_si *vsip_vcreate_si(vsip_length N,vsip_memory_hint hint);
vsip_vview_uc *vsip_vcreate_uc(vsip_length N,vsip_memory_hint hint);
vsip_cvview_d *vsip_cvcreate_d(vsip_length N,vsip_memory_hint hint);
vsip_cvview_f *vsip_cvcreate_f(vsip_length N,vsip_memory_hint hint);
```

参数:

N: 要为矢量分配的元素数。

hint: 内存运用方式。

返回值:

返回指向创建的矢量视图对象的指针,如果新对象的内存分配失败,则返回 NULL。

4.2.1.4.5 vsip_dvdestroy_p (新增 2 个)

销毁(释放)矢量视图对象并返回指向关联块对象的指针。

接口:

```
vsip_block_u *vsip_vdestroy_u(vsip_vview_u *v); (新增接口)
vsip_block_c *vsip_vdestroy_c(vsip_vview_c *v); (新增接口)
vsip_block_vi *vsip_vdestroy_vi(vsip_vview_vi *v);
vsip_block_mi *vsip_vdestroy_mi(vsip_vview_mi *v);
vsip_block_bl *vsip_vdestroy_bl(vsip_vview_bl *v);
vsip_block_d *vsip_vdestroy_d(vsip_vview_d *v);
vsip_block_f *vsip_vdestroy_f(vsip_vview_f *v);
vsip_block_i *vsip_vdestroy_i(vsip_vview_i *v);
vsip_block_si *vsip_vdestroy_si(vsip_vview_si *v);
vsip_block_uc *vsip_vdestroy_uc(vsip_vview_uc *v);
vsip_cblock_d *vsip_cvdestroy_d(vsip_cvview_d *v);
vsip_cblock_f *vsip_cvdestroy_f(vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回指向向量视图绑定到的块对象的指针，如果参数为 NULL，则返回 NULL。

4.2.1.4.6 vsip_dvget_p (新增 2 个)

获取向量视图对象的指定元素的值。

接口:

```
vsip_scalar_u vsip_vget_u(const vsip_vview_u *x, vsip_index j); (新增接口)
vsip_scalar_c vsip_vget_c(const vsip_vview_c *x, vsip_index j); (新增接口)
vsip_scalar_vi vsip_vget_vi(const vsip_vview_vi *x, vsip_index j);
vsip_scalar_mi vsip_vget_mi(const vsip_vview_mi *x, vsip_index j);
vsip_scalar_bl vsip_vget_bl(const vsip_vview_bl *x, vsip_index j);
vsip_scalar_d vsip_vget_d(const vsip_vview_d *x, vsip_index j);
vsip_scalar_f vsip_vget_f(const vsip_vview_f *x, vsip_index j);
vsip_scalar_i vsip_vget_i(const vsip_vview_i *x, vsip_index j);
vsip_scalar_si vsip_vget_si(const vsip_vview_si *x, vsip_index j);
vsip_scalar_uc vsip_vget_uc(const vsip_vview_uc *x, vsip_index j);
vsip_cscalar_d vsip_cvget_d(const vsip_cvview_d *x, vsip_index j);
vsip_cscalar_f vsip_cvget_f(const vsip_cvview_f *x, vsip_index j);
```

参数:

x: 矢量视图对象。

j: 向量元素的索引。

返回值:

返回向量视图对象的指定元素的值。

4.2.1.4.7 vsip_dvgetattrib_p

获取矢量视图对象的属性。

接口:

```
void vsip_vgetattrib_vi(const vsip_vview_vi *v, vsip_vattr_vi *attrib);
void vsip_vgetattrib_mi(const vsip_vview_mi *v, vsip_vattr_mi *attrib);
void vsip_vgetattrib_bl(const vsip_vview_bl *v, vsip_vattr_bl *attrib);
void vsip_vgetattrib_d(const vsip_vview_d *v, vsip_vattr_d *attrib);
void vsip_vgetattrib_f(const vsip_vview_f *v, vsip_vattr_f *attrib);
void vsip_vgetattrib_i(const vsip_vview_i *v, vsip_vattr_i *attrib);
void vsip_vgetattrib_si(const vsip_vview_si *v, vsip_vattr_si *attrib);
```



```
void vsip_vgetattrib_uc(const vsip_vview_uc *v, vsip_vattr_uc *attrib);  
void vsip_cvgetattrib_d(const vsip_cvview_d *v, vsip_cvattr_d *attrib);  
void vsip_cvgetattrib_f(const vsip_cvview_f *v, vsip_cvattr_f *attrib);
```

参数:

v: 矢量视图对象。

attrib: 指向输出向量属性结构的指针。

返回值:

空。

4.2.1.4.8 vsip_dvgetblock_p

获取矢量视图对象的块属性。

接口:

```
vsip_block_vi *vsip_vgetblock_vi(const vsip_vview_vi *v);  
vsip_block_mi *vsip_vgetblock_mi(const vsip_vview_mi *v);  
vsip_block_bl *vsip_vgetblock_bl(const vsip_vview_bl *v);  
vsip_block_d *vsip_vgetblock_d(const vsip_vview_d *v);  
vsip_block_f *vsip_vgetblock_f(const vsip_vview_f *v);  
vsip_block_i *vsip_vgetblock_i(const vsip_vview_i *v);  
vsip_block_si *vsip_vgetblock_si(const vsip_vview_si *v);  
vsip_block_uc *vsip_vgetblock_uc(const vsip_vview_uc *v);  
vsip_cblock_d *vsip_cvgetblock_d(const vsip_cvview_d *v);  
vsip_cblock_f *vsip_cvgetblock_f(const vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回指向向量视图对象绑定到的块对象的指针。

4.2.1.4.9 vsip_dvgetlength_p (新增 2 个)

获取矢量视图对象的长度属性。

接口:

```
vsip_length vsip_vgetlength_u(const vsip_vview_u *v); (新增接口)  
vsip_length vsip_vgetlength_c(const vsip_vview_c *v); (新增接口)  
vsip_length vsip_vgetlength_vi(const vsip_vview_vi *v);  
vsip_length vsip_vgetlength_mi(const vsip_vview_mi *v);  
vsip_length vsip_vgetlength_bl(const vsip_vview_bl *v);  
vsip_length vsip_vgetlength_d(const vsip_vview_d *v);
```



```
vsip_length vsip_vgetlength_f(const vsip_vview_f *v);  
vsip_length vsip_vgetlength_i(const vsip_vview_i *v);  
vsip_length vsip_vgetlength_si(const vsip_vview_si *v);  
vsip_length vsip_vgetlength_uc(const vsip_vview_uc *v);  
vsip_length vsip_cvgetlength_d(const vsip_cvview_d *v);  
vsip_length vsip_cvgetlength_f(const vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回向量视图对象的属性的 length 值。

4.2.1.4.10 vsip_dvgetoffset_p

获取矢量视图对象的 offset 属性。

接口:

```
vsip_offset vsip_vgetoffset_vi(const vsip_vview_vi *v);  
vsip_offset vsip_vgetoffset_mi(const vsip_vview_mi *v);  
vsip_offset vsip_vgetoffset_bl(const vsip_vview_bl *v);  
vsip_offset vsip_vgetoffset_d(const vsip_vview_d *v);  
vsip_offset vsip_vgetoffset_f(const vsip_vview_f *v);  
vsip_offset vsip_vgetoffset_i(const vsip_vview_i *v);  
vsip_offset vsip_vgetoffset_si(const vsip_vview_si *v);  
vsip_offset vsip_vgetoffset_uc(const vsip_vview_uc *v);  
vsip_offset vsip_cvgetoffset_d(const vsip_cvview_d *v);  
vsip_offset vsip_cvgetoffset_f(const vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回矢量视图对象的 offset 属性的值。

4.2.1.4.11 vsip_dvgetstride_p

获取矢量视图对象的步幅属性。

接口:

```
vsip_stride vsip_vgetstride_vi(const vsip_vview_vi *v);  
vsip_stride vsip_vgetstride_mi(const vsip_vview_mi *v);  
vsip_stride vsip_vgetstride_bl(const vsip_vview_bl *v);  
vsip_stride vsip_vgetstride_d(const vsip_vview_d *v);
```

```
vsip_stride vsip_vgetstride_f(const vsip_vview_f *v);  
vsip_stride vsip_vgetstride_i(const vsip_vview_i *v);  
vsip_stride vsip_vgetstride_si(const vsip_vview_si *v);  
vsip_stride vsip_vgetstride_uc(const vsip_vview_uc *v);  
vsip_stride vsip_cvgetstride_d(const vsip_cvview_d *v);  
vsip_stride vsip_cvgetstride_f(const vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回矢量视图对象的步幅 stride 属性值。

4.2.1.4.12 vsip_vimagview_p

从复数向量视图对象创建复数向量虚部的向量视图对象。

接口:

```
vsip_vview_d *vsip_vimagview_d(const vsip_cvview_d *v);  
vsip_vview_f *vsip_vimagview_f(const vsip_cvview_f *v);
```

参数:

v: 源矢量视图对象。

返回值:

返回一个指向创建的“假想”向量视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.4.13 vsip_dvput_p (新增 2 个)

放置(设置)向量视图对象的指定元素的值

接口:

```
void vsip_vput_u(const vsip_vview_u *y, vsip_index j, vsip_scalar_u x); (新增接口)  
void vsip_vput_c(const vsip_vview_c *y, vsip_index j, vsip_scalar_c x); (新增接口)  
void vsip_vput_vi(const vsip_vview_vi *y, vsip_index j, vsip_scalar_vi x);  
void vsip_vput_mi(const vsip_vview_mi *y, vsip_index j, vsip_scalar_mi x);  
void vsip_vput_bl(const vsip_vview_bl *y, vsip_index j, vsip_scalar_bl x);  
void vsip_vput_d(const vsip_vview_d *y, vsip_index j, vsip_scalar_d x);  
void vsip_vput_f(const vsip_vview_f *y, vsip_index j, vsip_scalar_f x);  
void vsip_vput_i(const vsip_vview_i *y, vsip_index j, vsip_scalar_i x);  
void vsip_vput_si(const vsip_vview_si *y, vsip_index j, vsip_scalar_si x);  
void vsip_vput_uc(const vsip_vview_uc *y, vsip_index j, vsip_scalar_uc x);  
void vsip_cvput_d(const vsip_cvview_d *y, vsip_index j, vsip_cscalar_d x);
```

```
void vsip_cvput_f(const vsip_cvview_f *y, vsip_index j, vsip_cscalar_f x);
```

参数:

- y: 目标矢量视图对象。
- j: 向量元素的向量索引。
- x: 要放置的标量值。

返回值:

空。

4.2.1.4.14 vsip_dvputattrib_p

放置(设置)矢量视图对象的属性。

接口:

```
vsip_vview_vi *vsip_vputattrib_vi( vsip_vview_vi *v, const vsip_vattr_vi *attrib);  
vsip_vview_mi *vsip_vputattrib_mi(vsip_vview_mi *v, const vsip_vattr_mi *attrib);  
vsip_vview_bl *vsip_vputattrib_bl(vsip_vview_bl *v, const vsip_vattr_bl *attrib);  
vsip_vview_d *vsip_vputattrib_d(vsip_vview_d *v, const vsip_vattr_d *attrib);  
vsip_vview_f *vsip_vputattrib_f(vsip_vview_f *v, const vsip_vattr_f *attrib);  
vsip_vview_i *vsip_vputattrib_i(vsip_vview_i *v, const vsip_vattr_i *attrib);  
vsip_vview_si *vsip_vputattrib_si(vsip_vview_si *v, const vsip_vattr_si *attrib);  
vsip_vview_uc *vsip_vputattrib_uc(vsip_vview_uc *v, const vsip_vattr_uc *attrib);  
vsip_cvview_d *vsip_cvputattrib_d(vsip_cvview_d *v, const vsip_cvattr_d *attrib);  
vsip_cvview_f *vsip_cvputattrib_f(vsip_cvview_f *v, const vsip_cvattr_f *attrib);
```

参数:

- v: 矢量视图对象。
- attrib: 指向输出向量属性结构的指针。

返回值:

为方便编程, 返回指向源矢量视图对象的指针。

4.2.1.4.15 vsip_dvputlength_p

放置(设置)向量视图对象的长度属性。

接口:

```
vsip_vview_vi *vsip_vputlength_vi(vsip_vview_vi *v, vsip_length length);  
vsip_vview_mi *vsip_vputlength_mi(vsip_vview_mi *v, vsip_length length);  
vsip_vview_bl *vsip_vputlength_bl(vsip_vview_bl *v, vsip_length length);  
vsip_vview_d *vsip_vputlength_d(vsip_vview_d *v, vsip_length length);  
vsip_vview_f *vsip_vputlength_f(vsip_vview_f *v, vsip_length length);
```

```
vsip_vview_i *vsip_vputlength_i(vsip_vview_i *v,vsip_length length);  
vsip_vview_si *vsip_vputlength_si(vsip_vview_si *v,vsip_length length);  
vsip_vview_uc *vsip_vputlength_uc(vsip_vview_uc *v,vsip_length length);  
vsip_cvview_d *vsip_cvputlength_d(vsip_cvview_d *v,vsip_length length);  
vsip_cvview_f *vsip_cvputlength_f(vsip_cvview_f *v,vsip_length length);
```

参数:

v: 矢量视图对象。

length: 元素长度。

返回值:

为方便编程, 返回指向源矢量视图对象的指针。

4.2.1.4.16 vsip_dvputoffset_p

放置(设置)矢量视图对象的偏移属性。

接口:

```
vsip_vview_vi *vsip_vputoffset_vi(vsip_vview_vi *v,vsip_offset offset);  
vsip_vview_mi *vsip_vputoffset_mi(vsip_vview_mi *v,vsip_offset offset);  
vsip_vview_bl *vsip_vputoffset_bl(vsip_vview_bl *v,vsip_offset offset);  
vsip_vview_d *vsip_vputoffset_d(vsip_vview_d *v,vsip_offset offset);  
vsip_vview_f *vsip_vputoffset_f(vsip_vview_f *v,vsip_offset offset);  
vsip_vview_i *vsip_vputoffset_i(vsip_vview_i *v,vsip_offset offset);  
vsip_vview_si *vsip_vputoffset_si(vsip_vview_si *v,vsip_offset offset);  
vsip_vview_uc *vsip_vputoffset_uc(vsip_vview_uc *v,vsip_offset offset);  
vsip_cvview_d *vsip_cvputoffset_d(vsip_cvview_d *v,vsip_offset offset);  
vsip_cvview_f *vsip_cvputoffset_f(vsip_cvview_f *v,vsip_offset offset);
```

参数:

v: 矢量视图对象。

offset: 元素相对于 block 对象起点的偏移量。

返回值:

为方便编程, 返回指向源矢量视图对象的指针。

4.2.1.4.17 vsip_dvputstride_p

放置(设置)矢量视图对象的 stride 属性。

接口:

```
vsip_vview_vi *vsip_vputstride_vi(vsip_vview_vi *v,vsip_stride stride);  
vsip_vview_mi *vsip_vputstride_mi(vsip_vview_mi *v,vsip_stride stride);  
vsip_vview_bl *vsip_vputstride_bl(vsip_vview_bl *v,vsip_stride stride);
```

```
vsip_vview_d *vsip_vputstride_d(vsip_vview_d *v, vsip_stride stride);  
vsip_vview_f *vsip_vputstride_f(vsip_vview_f *v, vsip_stride stride);  
vsip_vview_i *vsip_vputstride_i(vsip_vview_i *v, vsip_stride stride);  
vsip_vview_si *vsip_vputstride_si(vsip_vview_si *v, vsip_stride stride);  
vsip_vview_uc *vsip_vputstride_uc(vsip_vview_uc *v, vsip_stride stride);  
vsip_cvview_d *vsip_cvputstride_d(vsip_cvview_d *v, vsip_stride stride);  
vsip_cvview_f *vsip_cvputstride_f(vsip_cvview_f *v, vsip_stride stride);
```

参数:

v: 矢量视图对象。

stride: 元素间的步幅。

返回值:

为方便编程, 返回指向源矢量视图对象的指针。

4.2.1.4.18 vsip_vrealview_p

从复数向量视图对象创建复数向量实部的向量视图对象。

接口:

```
vsip_vview_d *vsip_vrealview_d(const vsip_cvview_d *v);  
vsip_vview_f *vsip_vrealview_f(const vsip_cvview_f *v);
```

参数:

v: 矢量视图对象。

返回值:

返回指向创建的“real”矢量视图对象的指针, 如果新建对象的内存分配失败, 则返回 NULL。

4.2.1.4.19 vsip_dvsubview_p

创建矢量视图对象, 它是矢量视图对象的子视图(偏移量和长度是相对于源视图对象, 而不是绑定块对象)。

接口:

```
vsip_vview_vi *vsip_vsubview_vi(const vsip_vview_vi *v, vsip_index index,  
vsip_length length);  
vsip_vview_mi *vsip_vsubview_mi(const vsip_vview_mi *v, vsip_index index,  
vsip_length length);  
vsip_vview_bl *vsip_vsubview_bl(const vsip_vview_bl *v, vsip_index index,  
vsip_length length);  
vsip_vview_d *vsip_vsubview_d(const vsip_vview_d *v, vsip_index index,
```

```

vsip_length length);
    vsip_vview_f *vsip_vsubview_f(const vsip_vview_f *v, vsip_index index,
vsip_length length);
    vsip_vview_i *vsip_vsubview_i(const vsip_vview_i *v, vsip_index index,
vsip_length length);
    vsip_vview_si *vsip_vsubview_si(const vsip_vview_si *v, vsip_index index,
vsip_length length);
    vsip_vview_uc *vsip_vsubview_uc(const vsip_vview_uc *v, vsip_index index,
vsip_length length);
    vsip_cvview_d *vsip_cvsubview_d(const vsip_cvview_d *v, vsip_index index,
vsip_length length);
    vsip_cvview_f *vsip_cvsubview_f(const vsip_cvview_f *v, vsip_index index,
vsip_length length);

```

参数:

v: 矢量视图对象。

index: 子视图向量第一元素(索引 0)在源向量的向量索引 “index” 处。

length: 元素长度。

返回值:

返回一个指向创建的子视图向量视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.5 矩阵视图对象函数

4.2.1.5.1 vsip_dmalldestroy_p

销毁(释放)矩阵、其关联块以及绑定到该块的任何 VSIPL-FT 数据数组。

接口:

```

void vsip_malldestroy_d(vsip_mview_d *X);
void vsip_malldestroy_f(vsip_mview_f *X);
void vsip_malldestroy_bl(vsip_mview_bl *X);
void vsip_malldestroy_i(vsip_mview_i *X);
void vsip_malldestroy_si(vsip_mview_si *X);
void vsip_malldestroy_uc(vsip_mview_uc *X);
void vsip_cmalldestroy_d(vsip_cmview_d *X);
void vsip_cmalldestroy_f(vsip_cmview_f *X);

```

参数:

X: 矩阵视图对象。

返回值:

空。

4.2.1.5.2 vsip_dmbind_p

创建一个矩阵视图对象并将其绑定到一个块对象。

接口:

```
vsip_mview_d *vsip_mbind_d(const vsip_block_d *block, vsip_offset offset ,  
vsip_stride col_stride, vsip_length col_length , vsip_stride row_stride , vsip_length  
row_length);
```

```
vsip_mview_f *vsip_mbind_f(const vsip_block_f *block, vsip_offset offset,  
vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride, vsip_length  
row_length);
```

```
vsip_mview_bl *vsip_mbind_bl(const vsip_block_bl *block, vsip_offset offset  
offset, vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride,  
vsip_length row_length);
```

```
vsip_mview_i *vsip_mbind_i(const vsip_block_i *block, vsip_offset offset offset,  
vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride, vsip_length  
row_length);
```

```
vsip_mview_si *vsip_mbind_si(const vsip_block_si *block, vsip_offset offset  
offset, vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride,  
vsip_length row_length);
```

```
vsip_mview_uc *vsip_mbind_uc(const vsip_block_uc *block, vsip_offset offset  
offset, vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride,  
vsip_length row_length);
```

```
vsip_cmview_d *vsip_cmbind_d(const vsip_cblock_d *block, vsip_offset offset  
offset, vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride,  
vsip_length row_length);
```

```
vsip_cmview_f *vsip_cmbind_f(const vsip_cblock_f *block, vsip_offset offset  
offset, vsip_stride col_stride, vsip_length col_length, vsip_stride row_stride,  
vsip_length row_length);
```

参数:

block: 指向块对象的指针。

offset: 元素中相对于 block 对象基础的矩阵视图偏移量。

col_stride: 列中元素之间的矩阵视图步幅。

col_length: 矩阵视图长度(以列的元素为单位)。

row_stride: 行中元素之间的矩阵视图步幅。

row_length: 矩阵视图长度(以行的元素为单位)。

返回值:

返回指向创建的矩阵视图对象的指针,如果新对象的内存分配失败,则返回 NULL。

4.2.1.5.3 vsip_dmcloneview_p

创建矩阵视图的克隆。

接口:

```
vsip_mview_d *vsip_mcloneview_d(const vsip_mview_d *X);
vsip_mview_f *vsip_mcloneview_f(const vsip_mview_f *X);
vsip_mview_bl *vsip_mcloneview_bl(const vsip_mview_bl *X);
vsip_mview_i *vsip_mcloneview_i(const vsip_mview_i *X);
vsip_mview_si *vsip_mcloneview_si(const vsip_mview_si *X);
vsip_mview_uc *vsip_mcloneview_uc(const vsip_mview_uc *X);
vsip_cmview_d *vsip_cmcloneview_d(const vsip_cmview_d *X);
vsip_cmview_f *vsip_cmcloneview_f(const vsip_cmview_f *X);
```

参数:

X: 源矩阵视图对象。

返回值:

返回指向创建的矩阵视图对象的指针,如果新对象的内存分配失败,则返回 NULL。

4.2.1.5.4 vsip_dmcolview_p

创建源矩阵视图对象的指定列的矢量视图对象。

接口:

```
vsip_cvview_d *vsip_cmcolview_d(const vsip_cmview_d *X, vsip_index
col_index);
vsip_cvview_f *vsip_cmcolview_f(const vsip_cmview_f *X, vsip_index col_index);
vsip_vview_bl *vsip_mcolview_bl(const vsip_mview_bl *X, vsip_index col_index);
vsip_vview_d *vsip_mcolview_d(const vsip_mview_d *X, vsip_index col_index);
vsip_vview_f *vsip_mcolview_f(const vsip_mview_f *X, vsip_index col_index);
vsip_vview_i *vsip_mcolview_i(const vsip_mview_i *X, vsip_index col_index);
vsip_vview_si *vsip_mcolview_si(const vsip_mview_si *X, vsip_index col_index);
vsip_vview_uc *vsip_mcolview_uc(const vsip_mview_uc *X, vsip_index
col_index);
```

参数:

X: 源矩阵视图对象。

col_index: 源矩阵视图对象的列索引。

返回值:

返回指向已创建的列向量视图对象的指针;如果新建对象的内存分配失败, 则返回 NULL。

4.2.1.5.5 vsip_dmcreate_p

创建块对象和块的矩阵视图(对象)。

接口:

```
vsip_mview_d *vsip_mcreate_d(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

```
vsip_mview_f *vsip_mcreate_f(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

```
vsip_mview_bl *vsip_mcreate_bl(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

```
vsip_mview_i *vsip_mcreate_i(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

```
vsip_mview_si *vsip_mcreate_si(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

```
vsip_mview_uc *vsip_mcreate_uc(vsip_length M, vsip_length N, vsip_major  
major, vsip_memory_hint hint);
```

```
vsip_cmview_d *vsip_cmcreate_d(vsip_length M, vsip_length N, vsip_major  
major, vsip_memory_hint hint);
```

```
vsip_cmview_f *vsip_cmcreate_f(vsip_length M, vsip_length N, vsip_major major,  
vsip_memory_hint hint);
```

参数:

M: 矩阵视图的行数(列长度)。

N: 矩阵视图的列数(行长度)。

major: 行或列主序。

hint: 内存运用方式。

返回值:

返回指向创建的矩阵视图对象的指针, 如果失败, 则返回 NULL。

4.2.1.5.6 vsip_dmdestroy_p

销毁(释放)矩阵视图对象并返回指向关联块对象的指针。

接口:

```
vsip_block_d *vsip_mdestroy_d(vsip_mview_d *X);
```

```
vsip_block_f *vsip_mdestroy_f(vsip_mview_f *X);
```

```
vsip_block_bl *vsip_mdestroy_bl(vsip_mview_bl *X);  
vsip_block_i *vsip_mdestroy_i(vsip_mview_i *X);  
vsip_block_si *vsip_mdestroy_si(vsip_mview_si *X);  
vsip_block_uc *vsip_mdestroy_uc(vsip_mview_uc *X);  
vsip_cblock_d *vsip_cmdestroy_d(vsip_cmview_d *X);  
vsip_cblock_f *vsip_cmdestroy_f(vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回指向矩阵视图所绑定到的块对象的指针，如果参数对象为 NULL，则返回 NULL。

4.2.1.5.7 vsip_dmdiagview_p

创建矩阵视图对象的矩阵对角线的矢量视图对象

接口:

```
vsip_vview_d *vsip_mdiagview_d(const vsip_mview_d *X, vsip_stride index);  
vsip_vview_f *vsip_mdiagview_f(const vsip_mview_f *X, vsip_stride index);  
vsip_vview_bl *vsip_mdiagview_bl(const vsip_mview_bl *X, vsip_stride index);  
vsip_vview_i *vsip_mdiagview_i(const vsip_mview_i *X, vsip_stride index);  
vsip_vview_si *vsip_mdiagview_si(const vsip_mview_si *X, vsip_stride index);  
vsip_vview_uc *vsip_mdiagview_uc(const vsip_mview_uc *X, vsip_stride  
index);  
vsip_cvview_d *vsip_cmdiagview_d(const vsip_cmview_d *X, vsip_stride  
index);  
vsip_cvview_f *vsip_cmdiagview_f(const vsip_cmview_f *X, vsip_stride index);
```

参数:

X: 矩阵视图对象。

index: 对角线索引，主对角线的索引是 0，主对角线上方的元素的索引是正数，主对角线下方的元素的索引是负数。

返回值:

返回指向创建的对角向量视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.5.8 vsip_dmget_p

获取矩阵视图对象的指定元素的值。

接口:

```
vsip_scalar_d vsip_mget_d(const vsip_mview_d *X, vsip_index i, vsip_index j);  
vsip_scalar_f vsip_mget_f(const vsip_mview_f *X, vsip_index i, vsip_index j);  
vsip_scalar_bl vsip_mget_bl(const vsip_mview_bl *X, vsip_index i, vsip_index j);  
vsip_scalar_i vsip_mget_i(const vsip_mview_i *X, vsip_index i, vsip_index j);  
vsip_scalar_si vsip_mget_si(const vsip_mview_si *X, vsip_index i, vsip_index j);  
vsip_scalar_uc vsip_mget_uc(const vsip_mview_uc *X, vsip_index i, vsip_index j);  
vsip_cscalar_d vsip_cmget_d(const vsip_cmview_d *X, vsip_index i, vsip_index j);  
vsip_cscalar_f vsip_cmget_f(const vsip_cmview_f *X, vsip_index i, vsip_index j);
```

参数:

X: 矩阵视图对象。

i: 矩阵索引 (i,j) 行索引。

j: 矩阵索引 (i,j) 列索引。

返回值:

返回矩阵视图对象的指定元素的值。

4.2.1.5.9 vsip_dmgetattrib_p

获取矩阵视图对象的属性。

接口:

```
void vsip_mgetattrib_d(const vsip_mview_d *X, vsip_mattr_d *attrib);  
void vsip_mgetattrib_f(const vsip_mview_f *X, vsip_mattr_f *attrib);  
void vsip_mgetattrib_bl(const vsip_mview_bl *X, vsip_mattr_bl *attrib);  
void vsip_mgetattrib_i(const vsip_mview_i *X, vsip_mattr_i *attrib);  
void vsip_mgetattrib_si(const vsip_mview_si *X, vsip_mattr_si *attrib);  
void vsip_mgetattrib_uc(const vsip_mview_uc *X, vsip_mattr_uc *attrib);  
void vsip_cmgetattrib_d(const vsip_cmview_d *X, vsip_cmattr_d *attrib);  
void vsip_cmgetattrib_f(const vsip_cmview_f *X, vsip_cmattr_f *attrib);
```

参数:

X: 矩阵视图对象。

attrib: 指向输出矩阵属性结构的指针。

返回值:

空。

4.2.1.5.10 vsip_dmgetblock_p

获取矩阵视图对象的块属性

接口:

```
vsip_block_d *vsip_mgetblock_d(const vsip_mview_d *X);  
vsip_block_f *vsip_mgetblock_f(const vsip_mview_f *X);  
vsip_block_bl *vsip_mgetblock_bl(const vsip_mview_bl *X);  
vsip_block_i *vsip_mgetblock_i(const vsip_mview_i *X);  
vsip_block_si *vsip_mgetblock_si(const vsip_mview_si *X);  
vsip_block_uc *vsip_mgetblock_uc(const vsip_mview_uc *X);  
vsip_cblock_d *vsip_cmgetblock_d(const vsip_cmview_d *X);  
vsip_cblock_f *vsip_cmgetblock_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回指向矩阵视图对象绑定到的块对象的指针。

4.2.1.5.11 vsip_dmgetcollength_p

获取矩阵视图对象的列长度属性。

接口:

```
vsip_length vsip_mgetcollength_d(const vsip_mview_d *X);  
vsip_length vsip_mgetcollength_f(const vsip_mview_f *X);  
vsip_length vsip_mgetcollength_bl(const vsip_mview_bl *X);  
vsip_length vsip_mgetcollength_i(const vsip_mview_i *X);  
vsip_length vsip_mgetcollength_si(const vsip_mview_si *X);  
vsip_length vsip_mgetcollength_uc(const vsip_mview_uc *X);  
vsip_length vsip_cmgetcollength_d(const vsip_cmview_d *X);  
vsip_length vsip_cmgetcollength_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回矩阵视图对象的 col_length 属性的值。

4.2.1.5.12 vsip_dmgetcolstride_p

获取矩阵视图对象的列步幅属性

接口:

```
vsip_stride vsip_mgetcolstride_d(const vsip_mview_d *X);  
vsip_stride vsip_mgetcolstride_f(const vsip_mview_f *X);  
vsip_stride vsip_mgetcolstride_bl(const vsip_mview_bl *X);
```

```
vsip_stride vsip_mgetcolstride_i(const vsip_mview_i *X);  
vsip_stride vsip_mgetcolstride_si(const vsip_mview_si *X);  
vsip_stride vsip_mgetcolstride_uc(const vsip_mview_uc *X);  
vsip_stride vsip_cmgetcolstride_d(const vsip_cmview_d *X);  
vsip_stride vsip_cmgetcolstride_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回矩阵视图对象的 col_stride 属性的值。

4.2.1.5.13 vsip_dmgetoffset_p

获取矩阵视图对象的 offset 属性。

接口:

```
vsip_offset vsip_mgetoffset_d(const vsip_mview_d *X);  
vsip_offset vsip_mgetoffset_f(const vsip_mview_f *X);  
vsip_offset vsip_mgetoffset_bl(const vsip_mview_bl *X);  
vsip_offset vsip_mgetoffset_i(const vsip_mview_i *X);  
vsip_offset vsip_mgetoffset_si(const vsip_mview_si *X);  
vsip_offset vsip_mgetoffset_uc(const vsip_mview_uc *X);  
vsip_offset vsip_cmgetoffset_d(const vsip_cmview_d *X);  
vsip_offset vsip_cmgetoffset_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回矩阵视图对象的 offset 属性的值。

4.2.1.5.14 vsip_dmgetrowlength_p

获取矩阵视图对象的行长度属性

接口:

```
vsip_length vsip_mgetrowlength_d(const vsip_mview_d *X);  
vsip_length vsip_mgetrowlength_f(const vsip_mview_f *X);  
vsip_length vsip_mgetrowlength_bl(const vsip_mview_bl *X);  
vsip_length vsip_mgetrowlength_i(const vsip_mview_i *X);  
vsip_length vsip_mgetrowlength_si(const vsip_mview_si *X);  
vsip_length vsip_mgetrowlength_uc(const vsip_mview_uc *X);  
vsip_length vsip_cmgetrowlength_d(const vsip_cmview_d *X);
```

```
vsip_length vsip_cmgetrowlength_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回矩阵视图对象的 row_length 属性的值。

4.2.1.5.15 vsip_dmgetrowstride_p

获取矩阵视图对象的行步幅属性。

接口:

```
vsip_stride vsip_mgetrowstride_d(const vsip_mview_d *X);  
vsip_stride vsip_mgetrowstride_f(const vsip_mview_f *X);  
vsip_stride vsip_mgetrowstride_bl(const vsip_mview_bl *X);  
vsip_stride vsip_mgetrowstride_i(const vsip_mview_i *X);  
vsip_stride vsip_mgetrowstride_si(const vsip_mview_si *X);  
vsip_stride vsip_mgetrowstride_uc(const vsip_mview_uc *X);  
vsip_stride vsip_cmgetrowstride_d(const vsip_cmview_d *X);  
vsip_stride vsip_cmgetrowstride_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回矩阵视图对象的 row_stride 属性的值。

4.2.1.5.16 vsip_mimagview_p

从复数矩阵视图对象创建复数矩阵虚部的矩阵视图对象。

接口:

```
vsip_mview_d *vsip_mimagview_d(const vsip_cmview_d *X);  
vsip_mview_f *vsip_mimagview_f(const vsip_cmview_f *X);
```

参数:

X: 源矩阵视图对象。

返回值:

返回一个指向创建的“假想”矩阵视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.5.17 vsip_dmput_p

放置(设置)矩阵视图对象的指定元素的值。

接口:

```
void vsip_mput_d(const vsip_mview_d *y, vsip_index i, vsip_index j,
vsip_scalar_d x);
void vsip_mput_f(const vsip_mview_f *y, vsip_index i, vsip_index j, vsip_scalar_f
x);
void vsip_mput_bl(const vsip_mview_bl *y, vsip_index i, vsip_index j,
vsip_scalar_bl x);
void vsip_mput_i(const vsip_mview_i *y, vsip_index i, vsip_index j, vsip_scalar_i
x);
void vsip_mput_si(const vsip_mview_si *y, vsip_index i, vsip_index j,
vsip_scalar_si x);
void vsip_mput_uc(const vsip_mview_uc *y, vsip_index i, vsip_index j,
vsip_scalar_uc x);
void vsip_cmput_d(const vsip_cmview_d *y, vsip_index i, vsip_index j,
vsip_cscalar_d x);
void vsip_cmput_f(const vsip_cmview_f *y, vsip_index i, vsip_index j,
vsip_cscalar_f x);
```

参数:

y: 目的矩阵视图对象。
i: 矩阵索引 (i,j) 行索引。
j: 矩阵索引 (i,j) 列索引。
x: 要放入矩阵中的标量值。

返回值:

空。

4.2.1.5.18 vsip_dmputattrib_p

放置(设置)矩阵视图对象的属性。

接口:

```
vsip_mview_d *vsip_mputattrib_d(vsip_mview_d *X,const vsip_mattr_d *attrib);
vsip_mview_f *vsip_mputattrib_f(vsip_mview_f *X,const vsip_mattr_f *attrib);
vsip_mview_bl *vsip_mputattrib_bl(vsip_mview_bl *X,const vsip_mattr_bl *attrib);
vsip_mview_i *vsip_mputattrib_i(vsip_mview_i *X,const vsip_mattr_i *attrib);
vsip_mview_si *vsip_mputattrib_si(vsip_mview_si *X,const vsip_mattr_si *attrib);
```

```
vsip_mview_uc *vsip_mputattrib_uc(vsip_mview_uc *X,const vsip_mattr_uc
*attrib);
vsip_cmview_d *vsip_cputattrib_d(vsip_cmview_d *X,const vsip_cmattr_d
*attrib);
vsip_cmview_f *vsip_cputattrib_f(vsip_cmview_f *X,const vsip_cmattr_f
*attrib);
```

参数:

X: 矩阵视图对象。

attrib: 指向矩阵属性结构的指针。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.19 vsip_dmputcollength_p

放置(设置)矩阵视图对象的列长度属性。

接口:

```
vsip_mview_d *vsip_mputcollength_d(vsip_mview_d *X,vsip_length length);
vsip_mview_f *vsip_mputcollength_f(vsip_mview_f *X,vsip_length length);
vsip_mview_bl *vsip_mputcollength_bl(vsip_mview_bl *X,vsip_length length);
vsip_mview_i *vsip_mputcollength_i(vsip_mview_i *X,vsip_length length);
vsip_mview_si *vsip_mputcollength_si(vsip_mview_si *X,vsip_length length);
vsip_mview_uc *vsip_mputcollength_uc(vsip_mview_uc *X,vsip_length length);
vsip_cmview_d *vsip_cputcollength_d(vsip_cmview_d *X,vsip_length length);
vsip_cmview_f *vsip_cputcollength_f(vsip_cmview_f *X,vsip_length length);
```

参数:

X: 矩阵视图对象。

length: 元素中的列长度。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.20 vsip_dmputcolstride_p

放置(设置)矩阵视图对象的列步幅属性。

接口:

```
vsip_mview_d *vsip_mputcolstride_d(vsip_mview_d *X,vsip_stride stride);
vsip_mview_f *vsip_mputcolstride_f(vsip_mview_f *X,vsip_stride stride);
vsip_mview_bl *vsip_mputcolstride_bl(vsip_mview_bl *X,vsip_stride stride);
vsip_mview_i *vsip_mputcolstride_i(vsip_mview_i *X,vsip_stride stride);
```



```
vsip_mview_si *vsip_mputcolstride_si(vsip_mview_si *X,vsip_stride stride);  
vsip_mview_uc *vsip_mputcolstride_uc(vsip_mview_uc *X,vsip_stride stride);  
vsip_cmview_d *vsip_cmputcolstride_d(vsip_cmview_d *X,vsip_stride stride);  
vsip_cmview_f *vsip_cmputcolstride_f(vsip_cmview_f *X,vsip_stride stride);
```

参数:

X: 矩阵视图对象。

stride: 元素中的列步幅。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.21 vsip_dmputoffset_p

放置(设置)矩阵视图对象的 offset 属性。

接口:

```
vsip_mview_d *vsip_mputoffset_d(vsip_mview_d *X,vsip_offset offset);  
vsip_mview_f *vsip_mputoffset_f(vsip_mview_f *X,vsip_offset offset);  
vsip_mview_bl *vsip_mputoffset_bl(vsip_mview_bl *X,vsip_offset offset);  
vsip_mview_i *vsip_mputoffset_i(vsip_mview_i *X,vsip_offset offset);  
vsip_mview_si *vsip_mputoffset_si(vsip_mview_si *X,vsip_offset offset);  
vsip_mview_uc *vsip_mputoffset_uc(vsip_mview_uc *X,vsip_offset offset);  
vsip_cmview_d *vsip_cmputoffset_d(vsip_cmview_d *X,vsip_offset offset);  
vsip_cmview_f *vsip_cmputoffset_f(vsip_cmview_f *X,vsip_offset offset);
```

参数:

X: 矩阵视图对象。

offset: 元素相对于块对象起点的偏移量。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.22 vsip_dmputrowlength_p

放置(设置)矩阵视图对象的行长度属性。

接口:

```
vsip_mview_d *vsip_mputrowlength_d(vsip_mview_d *X,vsip_length length);  
vsip_mview_f *vsip_mputrowlength_f(vsip_mview_f *X,vsip_length length);  
vsip_mview_bl *vsip_mputrowlength_bl(vsip_mview_bl *X,vsip_length length);  
vsip_mview_i *vsip_mputrowlength_i(vsip_mview_i *X,vsip_length length);  
vsip_mview_si *vsip_mputrowlength_si(vsip_mview_si *X,vsip_length length);  
vsip_mview_uc *vsip_mputrowlength_uc(vsip_mview_uc *X,vsip_length length);
```

```
vsip_cmview_d *vsip_cmputrowlength_d(vsip_cmview_d *X, vsip_length length);  
vsip_cmview_f *vsip_cmputrowlength_f(vsip_cmview_f *X, vsip_length length);
```

参数:

X: 矩阵视图对象。

length: 元素中的行长度。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.23 vsip_dmputrowstride_p

放置(设置)矩阵视图对象的行跨步属性。

接口:

```
vsip_mview_d *vsip_mputrowstride_d(vsip_mview_d *X, vsip_stride stride);  
vsip_mview_f *vsip_mputrowstride_f(vsip_mview_f *X, vsip_stride stride);  
vsip_mview_bl *vsip_mputrowstride_bl(vsip_mview_bl *X, vsip_stride stride);  
vsip_mview_i *vsip_mputrowstride_i(vsip_mview_i *X, vsip_stride stride);  
vsip_mview_si *vsip_mputrowstride_si(vsip_mview_si *X, vsip_stride stride);  
vsip_mview_uc *vsip_mputrowstride_uc(vsip_mview_uc *X, vsip_stride stride);  
vsip_cmview_d *vsip_cmputrowstride_d(vsip_cmview_d *X, vsip_stride stride);  
vsip_cmview_f *vsip_cmputrowstride_f(vsip_cmview_f *X, vsip_stride stride);
```

参数:

X: 矩阵视图对象。

stride: 元素中的行步幅。

返回值:

为方便编程, 返回指向源矩阵视图对象的指针。

4.2.1.5.24 vsip_mrealview_p

从复数矩阵视图对象创建复数矩阵实部的矩阵视图对象。

接口:

```
vsip_mview_d *vsip_mrealview_d(const vsip_cmview_d *X);  
vsip_mview_f *vsip_mrealview_f(const vsip_cmview_f *X);
```

参数:

X: 矩阵视图对象。

返回值:

返回一个指向创建的“真实”部分矩阵视图对象的指针, 如果新建对象的内存分配失败, 则返回 NULL。

4.2.1.5.25 vsip_dmrowview_p

创建源矩阵视图对象的指定行的矢量视图对象。

接口：

```
vsip_vview_d *vsip_mrowview_d(const vsip_mview_d *X, vsip_index row_index);
vsip_vview_f *vsip_mrowview_f(const vsip_mview_f *X, vsip_index row_index);
vsip_vview_bl *vsip_mrowview_bl(const vsip_mview_bl *X, vsip_index
row_index);
vsip_vview_i *vsip_mrowview_i(const vsip_mview_i *X, vsip_index row_index);
vsip_vview_si *vsip_mrowview_si(const vsip_mview_si *X, vsip_index
row_index);
vsip_vview_uc *vsip_mrowview_uc(const vsip_mview_uc *X, vsip_index
row_index);
vsip_cvview_d *vsip_cmrowview_d(const vsip_cmview_d *X, vsip_index
row_index);
vsip_cvview_f *vsip_cmrowview_f(const vsip_cmview_f *X, vsip_index
row_index);
```

参数：

X：源矩阵视图对象。

row_index：源矩阵视图对象的行索引。

返回值：

返回指向创建的行向量视图对象的指针，如果创建源矩阵视图对象的内存分配失败，则返回 NULL。

4.2.1.5.26 vsip_dmsubview_p

创建一个矩阵视图对象，它是矩阵视图对象的子视图。

接口：

```
vsip_mview_d *vsip_msubview_d(const vsip_mview_d *X, vsip_index row_index,
vsip_index col_index, vsip_length M, vsip_length N);
vsip_mview_f *vsip_msubview_f(const vsip_mview_f *X, vsip_index row_index,
vsip_index col_index, vsip_length M, vsip_length N);
vsip_mview_bl *vsip_msubview_bl(const vsip_mview_bl *X, vsip_index
row_index, vsip_index col_index, vsip_length M, vsip_length N);
vsip_mview_i *vsip_msubview_i(const vsip_mview_i *X, vsip_index row_index,
vsip_index col_index, vsip_length M, vsip_length N);
vsip_mview_si *vsip_msubview_si(const vsip_mview_si *X, vsip_index
```

```

row_index, vsip_index col_index, vsip_length M, vsip_length N);
    vsip_mview_uc *vsip_msubview_uc(const vsip_mview_uc *X, vsip_index
row_index, vsip_index col_index, vsip_length M, vsip_length N);
    vsip_cmview_d *vsip_cmsubview_d(const vsip_cmview_d *X, vsip_index
row_index, vsip_index col_index, vsip_length M, vsip_length N);
    vsip_cmview_f *vsip_cmsubview_f(const vsip_cmview_f *X, vsip_index
row_index, vsip_index col_index, vsip_length M, vsip_length N);

```

参数：

X：源矩阵视图对象。

row_index：源矩阵视图对象的索引(行索引、列索引)被映射到子矩阵视图对象的索引(0,0)。

col_index：源矩阵视图对象的索引(行索引、列索引)被映射到子矩阵视图对象的索引(0,0)。

M：矩阵视图的行数(列长度)。

N：矩阵视图的列数(行长度)。

返回值：

返回一个指向创建的子视图矩阵视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.5.27 vsip_dmtransview_p

创建一个矩阵视图对象，它是矩阵视图对象的转置。

接口：

```

vsip_mview_d *vsip_mtransview_d(const vsip_mview_d *X);
vsip_mview_f *vsip_mtransview_f(const vsip_mview_f *X);
vsip_mview_bl *vsip_mtransview_bl(const vsip_mview_bl *X);
vsip_mview_i *vsip_mtransview_i(const vsip_mview_i *X);
vsip_mview_si *vsip_mtransview_si(const vsip_mview_si *X);
vsip_mview_uc *vsip_mtransview_uc(const vsip_mview_uc *X);
vsip_cmview_d *vsip_cmtransview_d(const vsip_cmview_d *X);
vsip_cmview_f *vsip_cmtransview_f(const vsip_cmview_f *X);

```

参数：

X：源矩阵视图对象。

返回值：

返回一个指向创建的子视图矩阵视图对象的指针，如果新建对象的内存分配失败，则返回 NULL。

4.2.1.6 张量视图对象函数

4.2.1.6.1 vsip_dtalldestroy_p

销毁(释放)张量、其关联块以及绑定到该块的任何 VSIPL-FT 数据数组。

接口:

```
void vsip_ctalldestroy_d(vsip_ctview_d *T);
void vsip_ctalldestroy_f(vsip_ctview_f *T);
void vsip_talldestroy_d(vsip_tview_d *T);
void vsip_talldestroy_f(vsip_tview_f *T);
void vsip_talldestroy_i(vsip_tview_i *T);
void vsip_talldestroy_si(vsip_tview_si *T);
void vsip_talldestroy_uc(vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

空。

4.2.1.6.2 vsip_dtbind_p

将张量视图绑定到块。

接口:

```
vsip_ctview_d *vsip_ctbind_d(const vsip_cblock_d *block, vsip_offset offset,
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_ctview_f *vsip_ctbind_f(const vsip_cblock_f *block, vsip_offset offset,
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_tview_d *vsip_tbind_d(const vsip_block_d *block, vsip_offset offset,
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_tview_f *vsip_tbind_f(const vsip_block_f *block, vsip_offset offset,
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_tview_i *vsip_tbind_i(const vsip_block_i *block, vsip_offset offset,
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_tview_si *vsip_tbind_si(const vsip_block_si *block, vsip_offset offset,  
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,  
vsip_stride x_stride, vsip_length x_length);
```

```
vsip_tview_uc *vsip_tbind_uc(const vsip_block_uc *block, vsip_offset offset,  
vsip_stride z_stride, vsip_length z_length, vsip_stride y_stride, vsip_length y_length,  
vsip_stride x_stride, vsip_length x_length);
```

参数:

block: 指向块对象的指针。

offset: 元素中相对于 block 对象基础的张量视图偏移。

z_stride: 沿 Z 轴连续元素之间的步幅。

z_length: 沿 Z 轴的元素长度。

y_stride: 沿 Y 轴连续元素之间的步幅。

y_length: 沿 Y 轴的元素长度。

x_stride: 沿 X 轴连续元素之间的步幅。

x_length: 沿 X 轴的元素长度。

返回值:

返回指向创建的张量视图对象的指针, 如果新对象的内存分配失败, 则返回 NULL。

4.2.1.6.3 vsip_dtcloneview_p

创建张量视图的克隆。

接口:

```
vsip_ctview_d *vsip_ctcloneview_d(const vsip_ctview_d *T);  
vsip_ctview_f *vsip_ctcloneview_f(const vsip_ctview_f *T);  
vsip_tview_d *vsip_tcloneview_d(const vsip_tview_d *T);  
vsip_tview_f *vsip_tcloneview_f(const vsip_tview_f *T);  
vsip_tview_i *vsip_tcloneview_i(const vsip_tview_i *T);  
vsip_tview_si *vsip_tcloneview_si(const vsip_tview_si *T);  
vsip_tview_uc *vsip_tcloneview_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回指向创建的张量视图对象克隆的指针, 如果创建对象的内存分配失败, 则返回 NULL。

4.2.1.6.4 vsip_dtcreate_p

创建块对象和块的张量视图(对象)。

接口：

```
vsip_ctview_d* vsip_ctcreate_d(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_ctview_f* vsip_ctcreate_f(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_tview_d* vsip_tcreate_d(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_tview_f* vsip_tcreate_f(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_tview_i* vsip_tcreate_i(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_tview_si* vsip_tcreate_si(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

```
vsip_tview_uc* vsip_tcreate_uc(vsip_length P, vsip_length M, vsip_length N,  
vsip_tmajor major, vsip_memory_hint hint);
```

参数：

P：张量沿 Z 轴的元素数(Z 长度)。

M：张量沿 Y 轴的元素数(Y 长度)。

N：张量沿 X 轴的元素数(XZ 长度)。

major：尾随或前导索引是单位步幅方向。

hint：内存提示。

返回值：

返回指向创建的张量视图对象的指针，如果失败，则返回 NULL。

4.2.1.6.5 vsip_dtdestroy_p

销毁(释放)张量视图对象并返回指向关联块对象的指针。

接口：

```
vsip_cblock_d* vsip_ctdestroy_d(vsip_ctview_d *T);
```

```
vsip_cblock_f* vsip_ctdestroy_f(vsip_ctview_f *T);
```

```
vsip_block_d* vsip_tdestroy_d(vsip_tview_d *T);
```

```
vsip_block_f* vsip_tdestroy_f(vsip_tview_f *T);
```

```
vsip_block_i* vsip_tdestroy_i(vsip_tview_i *T);
```

```
vsip_block_si* vsip_tdestroy_si(vsip_tview_si *T);
```

```
vsip_block_uc *vsip_tdestroy_uc(vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回指向张量视图绑定到的块对象的指针,如果调用参数为 NULL,则返回 NULL。

4.2.1.6.6 vsip_dtget_p

获取张量视图对象的指定元素的值。

接口:

```
vsip_cscalar_d vsip_ctget_d(const vsip_ctview_d *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_cscalar_f vsip_ctget_f(const vsip_ctview_f *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_scalar_d vsip_tget_d(const vsip_tview_d *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_scalar_f vsip_tget_f(const vsip_tview_f *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_scalar_i vsip_tget_i(const vsip_tview_i *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_scalar_si vsip_tget_si(const vsip_tview_si *x, vsip_index h, vsip_index i, vsip_index j);
```

```
vsip_scalar_uc vsip_tget_uc(const vsip_tview_uc *x, vsip_index h, vsip_index i, vsip_index j);
```

参数:

x: 张量视图对象。

h: 张量(h, i, j)的指数 h。

i: 张量(h, i, j)的指数 i。

j: 张量(h, i, j)的指数 j。

返回值:

返回张量视图对象的指定元素的值。

4.2.1.6.7 vsip_dtgetattrib_p

获取张量视图对象的属性。

接口:

```
void vsip_ctgetattrib_d(const vsip_ctview_d *T, vsip_ctattr_d *attrib);
```



```
void vsip_ctgetattrib_f(const vsip_ctview_f *T, vsip_ctattr_f *attrib);
void vsip_tgetattrib_d(const vsip_tview_d *T, vsip_tattr_d *attrib);
void vsip_tgetattrib_f(const vsip_tview_f *T, vsip_tattr_f *attrib);
void vsip_tgetattrib_i(const vsip_tview_i *T, vsip_tattr_i *attrib);
void vsip_tgetattrib_si(const vsip_tview_si *T, vsip_tattr_si *attrib);
void vsip_tgetattrib_uc(const vsip_tview_uc *T, vsip_tattr_uc *attrib);
```

参数:

T: 张量视图对象。

attrib: 指向输出张量属性结构的指针。

返回值:

空。

4.2.1.6.8 vsip_dtgetblock_p

获取张量视图对象的 block 属性。

接口:

```
vsip_cblock_d *vsip_ctgetblock_d(const vsip_ctview_d *T);
vsip_cblock_f *vsip_ctgetblock_f(const vsip_ctview_f *T);
vsip_block_d *vsip_tgetblock_d(const vsip_tview_d *T);
vsip_block_f *vsip_tgetblock_f(const vsip_tview_f *T);
vsip_block_i *vsip_tgetblock_i(const vsip_tview_i *T);
vsip_block_si *vsip_tgetblock_si(const vsip_tview_si *T);
vsip_block_uc *vsip_tgetblock_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回指向张量视图对象绑定到的块对象的指针。

4.2.1.6.9 vsip_dtgetoffset_p

获取张量视图对象的 offset 属性。

接口:

```
vsip_offset vsip_ctgetoffset_d(const vsip_ctview_d *T);
vsip_offset vsip_ctgetoffset_f(const vsip_ctview_f *T);
vsip_offset vsip_tgetoffset_d(const vsip_tview_d *T);
vsip_offset vsip_tgetoffset_f(const vsip_tview_f *T);
vsip_offset vsip_tgetoffset_i(const vsip_tview_i *T);
vsip_offset vsip_tgetoffset_si(const vsip_tview_si *T);
```

```
vsip_offset vsip_tgetoffset_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回张量视图对象的 offset 属性的值。

4.2.1.6.10 vsip_dtgetxlength_p

获取张量视图对象的 X 长度属性。

接口:

```
vsip_length vsip_ctgetxlength_d(const vsip_ctview_d *T);  
vsip_length vsip_ctgetxlength_f(const vsip_ctview_f *T);  
vsip_length vsip_tgetxlength_d(const vsip_tview_d *T);  
vsip_length vsip_tgetxlength_f(const vsip_tview_f *T);  
vsip_length vsip_tgetxlength_i(const vsip_tview_i *T);  
vsip_length vsip_tgetxlength_si(const vsip_tview_si *T);  
vsip_length vsip_tgetxlength_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回张量视图对象的 X 长度属性。

4.2.1.6.11 vsip_dtgetxstride_p

获取张量视图对象的 X stride 属性。

接口:

```
vsip_stride vsip_ctgetxstride_d(const vsip_ctview_d *T);  
vsip_stride vsip_ctgetxstride_f(const vsip_ctview_f *T);  
vsip_stride vsip_tgetxstride_d(const vsip_tview_d *T);  
vsip_stride vsip_tgetxstride_f(const vsip_tview_f *T);  
vsip_stride vsip_tgetxstride_i(const vsip_tview_i *T);  
vsip_stride vsip_tgetxstride_si(const vsip_tview_si *T);  
vsip_stride vsip_tgetxstride_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回张量视图对象的 X stride 属性。

4.2.1.6.12 vsip_dtgetylength_p

获取张量视图对象的 Y 长度属性。

接口：

```
vsip_length vsip_ctgetylength_d(const vsip_ctview_d *T);  
vsip_length vsip_ctgetylength_f(const vsip_ctview_f *T);  
vsip_length vsip_tgetylength_d(const vsip_tview_d *T);  
vsip_length vsip_tgetylength_f(const vsip_tview_f *T);  
vsip_length vsip_tgetylength_i(const vsip_tview_i *T);  
vsip_length vsip_tgetylength_si(const vsip_tview_si *T);  
vsip_length vsip_tgetylength_uc(const vsip_tview_uc *T);
```

参数：

T：张量视图对象。

返回值：

返回张量视图对象的 Y 长度属性。

4.2.1.6.13 vsip_dtgetystride_p

获取张量视图对象的 Y 步幅属性。

接口：

```
vsip_stride vsip_ctgetystride_d(const vsip_ctview_d *T);  
vsip_stride vsip_ctgetystride_f(const vsip_ctview_f *T);  
vsip_stride vsip_tgetystride_d(const vsip_tview_d *T);  
vsip_stride vsip_tgetystride_f(const vsip_tview_f *T);  
vsip_stride vsip_tgetystride_i(const vsip_tview_i *T);  
vsip_stride vsip_tgetystride_si(const vsip_tview_si *T);  
vsip_stride vsip_tgetystride_uc(const vsip_tview_uc *T);
```

参数：

T：张量视图对象。

返回值：

返回张量视图对象的 Y 步幅属性。

4.2.1.6.14 vsip_dtgetzlength_p

获取张量视图对象的 Z 长度属性。

接口：

```
vsip_length vsip_ctgetzlength_d(const vsip_ctview_d *T);
```

```
vsip_length vsip_ctgetzlength_f(const vsip_ctview_f *T);  
vsip_length vsip_tgetzlength_d(const vsip_tview_d *T);  
vsip_length vsip_tgetzlength_f(const vsip_tview_f *T);  
vsip_length vsip_tgetzlength_i(const vsip_tview_i *T);  
vsip_length vsip_tgetzlength_si(const vsip_tview_si *T);  
vsip_length vsip_tgetzlength_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回张量视图对象的 Z 长度属性。

4.2.1.6.15 vsip_dtgetzstride_p

获取张量视图对象的 Z 步幅属性。

接口:

```
vsip_stride vsip_ctgetzstride_d(const vsip_ctview_d *T);  
vsip_stride vsip_ctgetzstride_f(const vsip_ctview_f *T);  
vsip_stride vsip_tgetzstride_d(const vsip_tview_d *T);  
vsip_stride vsip_tgetzstride_f(const vsip_tview_f *T);  
vsip_stride vsip_tgetzstride_i(const vsip_tview_i *T);  
vsip_stride vsip_tgetzstride_si(const vsip_tview_si *T);  
vsip_stride vsip_tgetzstride_uc(const vsip_tview_uc *T);
```

参数:

T: 张量视图对象。

返回值:

返回张量视图对象的 Z 步幅属性。

4.2.1.6.16 vsip_timagview_p

从复数张量视图对象创建复数张量虚部的张量视图对象。

接口:

```
vsip_tview_d *vsip_timagview_d(const vsip_ctview_d *T);  
vsip_tview_f *vsip_timagview_f(const vsip_ctview_f *T);
```

参数:

T: 张量视图对象。

返回值:

返回指向创建的“虚构”部分张量视图对象的指针，如果新对象的内存分配失败，

则返回 NULL。

4.2.1.6.17 vsip_dtmatrixview_p

创建张量视图的二维切片的矩阵视图。

接口：

```
vsip_cmview_d *vsip_ctmatrixview_d(const vsip_ctview_d *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_cmview_f *vsip_ctmatrixview_f(const vsip_ctview_f *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_mview_d *vsip_tmatrixview_d(const vsip_tview_d *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_mview_f *vsip_tmatrixview_f(const vsip_tview_f *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_mview_i *vsip_tmatrixview_i(const vsip_tview_i *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_mview_si *vsip_tmatrixview_si(const vsip_tview_si *T, vsip_tmslice slice,
vsip_index j);
```

```
vsip_mview_uc *vsip_tmatrixview_uc(const vsip_tview_uc *T, vsip_tmslice slice,
vsip_index j);
```

参数：

T：张量视图对象。

slice：张量的切片。

j：垂直于切片的轴的索引。

返回值：

返回指向创建的矩阵视图对象的指针，如果新对象的内存分配失败，则返回 NULL。

4.2.1.6.18 vsip_dtput_p

放置(设置)张量视图对象的指定元素的值

接口：

```
void vsip_ctput_d(const vsip_ctview_d *y, vsip_index h, vsip_index i, vsip_index j,
vsip_cscalar_d *x);
```

```
void vsip_ctput_f(const vsip_ctview_f *y, vsip_index h, vsip_index i, vsip_index j,
vsip_cscalar_f *x);
```

```
void vsip_tput_d(const vsip_tview_d *y, vsip_index h, vsip_index i, vsip_index j,
vsip_scalar_d *x);
```

```
void vsip_tput_f(const vsip_tview_f *y, vsip_index h, vsip_index i, vsip_index j,
```

```
vsip_scalar_f *x);  
void vsip_tput_i(const vsip_tview_i *y, vsip_index h, vsip_index i, vsip_index j,  
vsip_scalar_i *x);  
void vsip_tput_si(const vsip_tview_si *y, vsip_index h, vsip_index i, vsip_index j,  
vsip_scalar_si *x);  
void vsip_tput_uc(const vsip_tview_uc *y, vsip_index h, vsip_index i, vsip_index j,  
vsip_scalar_uc *x);
```

参数:

- y: 目标的张量视图对象。
- h: 张量(h, i, j)的指数 h。
- i: 张量(h, i, j)的指数 i。
- j: 张量(h, i, j)的指数 j。
- x: 要放入张量的标量值。

返回值:

空。

4.2.1.6.19 vsip_dtputattrib_p

放置(设置)张量视图对象的属性。

接口:

```
void vsip_ctputattrib_d(vsip_ctview_d *T, const vsip_ctattr_d *attrib);  
void vsip_ctputattrib_f(vsip_ctview_f *T, const vsip_ctattr_f *attrib);  
void vsip_tputattrib_d(vsip_tview_d *T, const vsip_tattr_d *attrib);  
void vsip_tputattrib_f(vsip_tview_f *T, const vsip_tattr_f *attrib);  
void vsip_tputattrib_i(vsip_tview_i *T, const vsip_tattr_i *attrib);  
void vsip_tputattrib_si(vsip_tview_si *T, const vsip_tattr_si *attrib);  
void vsip_tputattrib_uc(vsip_tview_uc *T, const vsip_tattr_uc *attrib);
```

参数:

- T: 张量视图对象。
- attrib: 指向张量属性结构的指针。

返回值:

空。

4.2.1.6.20 vsip_dtputoffset_p

放置(设置)张量视图对象的 offset 属性。

接口:

```
vsip_ctview_d *vsip_ctputoffset_d(vsip_ctview_d *T, vsip_offset offset);  
vsip_ctview_f *vsip_ctputoffset_f(vsip_ctview_f *T, vsip_offset offset);  
vsip_tview_d *vsip_tputoffset_d(vsip_tview_d *T, vsip_offset offset);  
vsip_tview_f *vsip_tputoffset_f(vsip_tview_f *T, vsip_offset offset);  
vsip_tview_i *vsip_tputoffset_i(vsip_tview_i *T, vsip_offset offset);  
vsip_tview_si *vsip_tputoffset_si(vsip_tview_si *T, vsip_offset offset);  
vsip_tview_uc *vsip_tputoffset_uc(vsip_tview_uc *T, vsip_offset offset);
```

参数:

T: 张量视图对象。

offset: 元素相对于 block 对象起点的偏移量。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.21 vsip_dtputxlength_p

放置(设置)张量视图对象的 X 长度属性。

接口:

```
vsip_ctview_d *vsip_dtputxlength_d(vsip_ctview_d *T, vsip_length length);  
vsip_ctview_f *vsip_dtputxlength_f(vsip_ctview_f *T, vsip_length length);  
vsip_tview_d *vsip_dtputxlength_d(vsip_tview_d *T, vsip_length length);  
vsip_tview_f *vsip_dtputxlength_f(vsip_tview_f *T, vsip_length length);  
vsip_tview_i *vsip_dtputxlength_i(vsip_tview_i *T, vsip_length length);  
vsip_tview_si *vsip_dtputxlength_si(vsip_tview_si *T, vsip_length length);  
vsip_tview_uc *vsip_dtputxlength_uc(vsip_tview_uc *T, vsip_length length);
```

参数:

T: 张量视图对象。

length: X 轴的长度。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.22 vsip_dtputxstride_p

放置(设置)张量视图对象的 X stride 属性。

接口:

```
vsip_ctview_d *vsip_dtputxstride_d(vsip_ctview_d *T, vsip_stride stride);  
vsip_ctview_f *vsip_dtputxstride_f(vsip_ctview_f *T, vsip_stride stride);  
vsip_tview_d *vsip_dtputxstride_d(vsip_tview_d *T, vsip_stride stride);  
vsip_tview_f *vsip_dtputxstride_f(vsip_tview_f *T, vsip_stride stride);
```

```
vsip_tview_i *vsip_tputxstride_i(vsip_tview_i *T, vsip_stride stride);  
vsip_tview_si *vsip_tputxstride_si(vsip_tview_si *T, vsip_stride stride);  
vsip_tview_uc *vsip_tputxstride_uc(vsip_tview_uc *T, vsip_stride stride);
```

参数:

T: 张量视图对象。

stride: 沿 X 轴在连续元素之间跨越元素。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.23 vsip_dtputylength_p

放置(设置)张量视图对象的 Y 长度属性。

接口:

```
vsip_ctview_d *vsip_ctputylength_d(vsip_ctview_d *T, vsip_length length);  
vsip_ctview_f *vsip_ctputylength_f(vsip_ctview_f *T, vsip_length length);  
vsip_tview_d *vsip_tputylength_d(vsip_tview_d *T, vsip_length length);  
vsip_tview_f *vsip_tputylength_f(vsip_tview_f *T, vsip_length length);  
vsip_tview_i *vsip_tputylength_i(vsip_tview_i *T, vsip_length length);  
vsip_tview_si *vsip_tputylength_si(vsip_tview_si *T, vsip_length length);  
vsip_tview_uc *vsip_tputylength_uc(vsip_tview_uc *T, vsip_length length);
```

参数:

T: 张量视图对象。

length: Y 轴的长度。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.24 vsip_dtputystride_p

放置(设置)张量视图对象的 Y stride 属性。

接口:

```
vsip_ctview_d *vsip_ctputystride_d(vsip_ctview_d *T, vsip_stride stride);  
vsip_ctview_f *vsip_ctputystride_f(vsip_ctview_f *T, vsip_stride stride);  
vsip_tview_d *vsip_tputystride_d(vsip_tview_d *T, vsip_stride stride);  
vsip_tview_f *vsip_tputystride_f(vsip_tview_f *T, vsip_stride stride);  
vsip_tview_i *vsip_tputystride_i(vsip_tview_i *T, vsip_stride stride);  
vsip_tview_si *vsip_tputystride_si(vsip_tview_si *T, vsip_stride stride);  
vsip_tview_uc *vsip_tputystride_uc(vsip_tview_uc *T, vsip_stride stride);
```


参数:

T: 张量视图对象。

stride: 沿 Y 轴在连续元素之间跨越元素。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.25 vsip_dtputlength_p

放置(设置)张量视图对象的 Z 长度属性。

接口:

```
vsip_ctview_d *vsip_ctputlength_d(vsip_ctview_d *T, vsip_length length);  
vsip_ctview_f *vsip_ctputlength_f(vsip_ctview_f *T, vsip_length length);  
vsip_tview_d *vsip_tputlength_d(vsip_tview_d *T, vsip_length length);  
vsip_tview_f *vsip_tputlength_f(vsip_tview_f *T, vsip_length length);  
vsip_tview_i *vsip_tputlength_i(vsip_tview_i *T, vsip_length length);  
vsip_tview_si *vsip_tputlength_si(vsip_tview_si *T, vsip_length length);  
vsip_tview_uc *vsip_tputlength_uc(vsip_tview_uc *T, vsip_length length);
```

参数:

T: 张量视图对象。

length: Z 轴的长度。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.26 vsip_dtputzstride_p

放置(设置)张量视图对象的 Z 步幅属性。

接口:

```
vsip_ctview_d *vsip_ctputzstride_d(vsip_ctview_d *T, vsip_stride stride);  
vsip_ctview_f *vsip_ctputzstride_f(vsip_ctview_f *T, vsip_stride stride);  
vsip_tview_d *vsip_tputzstride_d(vsip_tview_d *T, vsip_stride stride);  
vsip_tview_f *vsip_tputzstride_f(vsip_tview_f *T, vsip_stride stride);  
vsip_tview_i *vsip_tputzstride_i(vsip_tview_i *T, vsip_stride stride);  
vsip_tview_si *vsip_tputzstride_si(vsip_tview_si *T, vsip_stride stride);  
vsip_tview_uc *vsip_tputzstride_uc(vsip_tview_uc *T, vsip_stride stride);
```

参数:

T: 张量视图对象。

stride: 沿 Z 轴在连续元素之间跨越元素。

返回值:

为方便编程, 返回指向源张量视图对象的指针。

4.2.1.6.27 vsip_trealview_p

从复数张量视图对象创建复数张量的实部的张量视图对象。

接口:

```
vsip_tview_d *vsip_trealview_d(const vsip_ctview_d *T);  
vsip_tview_f *vsip_trealview_f(const vsip_ctview_f *T);
```

参数:

T: 张量视图对象。

返回值:

返回指向创建的“真实”部分张量视图对象的指针, 如果新建对象的内存分配失败, 则返回 NULL。

4.2.1.6.28 vsip_dtsubview_p

创建张量视图对象, 该对象是张量视图对象的子视图。

接口:

```
vsip_ctview_d *vsip_ctsubview_d(const vsip_ctview_d *T, vsip_index z,  
vsip_index y, vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_ctview_f *vsip_ctsubview_f(const vsip_ctview_f *T, vsip_index z, vsip_index  
y, vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_tview_d *vsip_tsubview_d(const vsip_tview_d *T, vsip_index z, vsip_index y,  
vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_tview_f *vsip_tsubview_f(const vsip_tview_f *T, vsip_index z, vsip_index y,  
vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_tview_i *vsip_tsubview_i(const vsip_tview_i *T, vsip_index z, vsip_index y,  
vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_tview_si *vsip_tsubview_si(const vsip_tview_si *T, vsip_index z, vsip_index  
y, vsip_index x, vsip_length P, vsip_length M, vsip_length N);  
vsip_tview_uc *vsip_tsubview_uc(const vsip_tview_uc *T, vsip_index z,  
vsip_index y, vsip_index x, vsip_length P, vsip_length M, vsip_length N);
```

参数:

T: 张量视图对象。

z、y、x: 源矩阵视图对象的索引 (Z 索引、Y 索引、X 索引) 映射到子视图张量对象的索引 (0,0,0)。

P: 张量子视图沿 Z 轴的元素数(Z 长度)。

M: 张量子视图沿 Y 轴的元素数(Y 长度)。

N: 张量子视图沿 X 轴的元素数(X 长度)。

返回值:

返回指向创建的子视图张量视图对象的指针, 如果新建对象的内存分配失败, 则返回 NULL。

4.2.1.6.29 vsip_dttransview_p

创建一个转置张量视图。

接口:

```
vsip_ctview_d *vsip_cttransview_d(const vsip_ctview_d *T, vsip_ttrans trans);
vsip_ctview_f *vsip_cttransview_f(const vsip_ctview_f *T, vsip_ttrans trans);
vsip_tview_d *vsip_ttransview_d(const vsip_tview_d *T, vsip_ttrans trans);
vsip_tview_f *vsip_ttransview_f(const vsip_tview_f *T, vsip_ttrans trans);
vsip_tview_i *vsip_ttransview_i(const vsip_tview_i *T, vsip_ttrans trans);
vsip_tview_si *vsip_ttransview_si(const vsip_tview_si *T, vsip_ttrans trans);
vsip_tview_uc *vsip_ttransview_uc(const vsip_tview_uc *T, vsip_ttrans trans);
```

参数:

T: 张量视图对象。

trans: 指定转置类型。

返回值:

返回指向创建的张量视图对象的指针, 如果创建失败, 则返回 NULL。

4.2.1.6.30 vsip_dtvectview_p

创建张量视图的一维切片的矢量视图。

接口:

```
vsip_cvview_d *vsip_ctvectview_d(const vsip_ctview_d *T, vsip_tvslice slice,
vsip_index i, vsip_index j);
vsip_cvview_f *vsip_ctvectview_f(const vsip_ctview_f *T, vsip_tvslice slice,
vsip_index i, vsip_index j);
vsip_vview_d *vsip_tvectview_d(const vsip_tview_d *T, vsip_tvslice slice,
vsip_index i, vsip_index j);
vsip_vview_f *vsip_tvectview_f(const vsip_tview_f *T, vsip_tvslice slice,
vsip_index i, vsip_index j);
vsip_vview_i *vsip_tvectview_i(const vsip_tview_i *T, vsip_tvslice slice,
```

```
vsip_index i, vsip_index j);  
    vsip_vview_si *vsip_tvectview_si(const vsip_tview_si *T, vsip_tvslice slice,  
vsip_index i, vsip_index j);  
    vsip_vview_uc *vsip_tvectview_uc(const vsip_tview_uc *T, vsip_tvslice slice,  
vsip_index i, vsip_index j);
```

参数:

T: 张量视图对象。

slice: 一维切片沿着切片轴。

i: 固定张量的第一个指数。

j: 固定张量的第二个指数。

返回值:

返回指向创建的矢量视图对象的指针, 如果新对象的内存分配失败, 则返回 NULL。

4.2.2 VSIPL-FT 标量函数（新增 23 个）

4.2.2.1 标量函数介绍

本章节将介绍实标量、复标量以及索引标量函数。VSIPL-FT 未指定标量函数的域和范围，结果取决于实现。ANSI C9x 用复数类型扩展 C，`vsip_cscalar_d` (`vsip_cscalar_f`) 与 ANSI C9x 中的复数双精度（复浮点数）兼容。

4.2.2.2 实标量函数（新增 11 个）

4.2.2.2.1 `vsip_sin_p`（新增 2 个）

计算标量元素的正弦。

接口：

`vsip_scalar_f vsip_sin_f(vsip_scalar_f x);`（新增接口）

`vsip_scalar_d vsip_sin_d(vsip_scalar_d x);`（新增接口）

参数：

`x`：实数标量参数。

返回值：

返回正弦值。

4.2.2.2.2 `vsip_cos_p`（新增 2 个）

计算标量元素的余弦。

接口：

`vsip_scalar_f vsip_cos_f(vsip_scalar_f x);`（新增接口）

`vsip_scalar_d vsip_cos_d(vsip_scalar_d x);`（新增接口）

参数：

`x`：实数标量参数。

返回值：

返回余弦值。

4.2.2.2.3 `vsip_tan_p`（新增 2 个）

计算标量元素的正切。

接口：

`vsip_scalar_f vsip_tan_f(vsip_scalar_f x);`（新增接口）

`vsip_scalar_d vsip_tan_d(vsip_scalar_d x);` (新增接口)

参数:

`x`: 实数标量参数。

返回值:

返回正切值。

4.2.2.2.4 vsip_acos_p

计算标量元素的反余弦。

接口:

`vsip_scalar_f vsip_acos_f(vsip_scalar_f theta);`

`vsip_scalar_d vsip_acos_d(vsip_scalar_d theta);`

参数:

`theta`: 实数标量参数。

返回值:

返回反余弦函数值。

4.2.2.2.5 vsip_asin_p

计算标量元素的反正弦。

接口:

`vsip_scalar_f vsip_asin_f(vsip_scalar_f theta);`

`vsip_scalar_d vsip_asin_d(vsip_scalar_d theta);`

参数:

`theta`: 实数标量参数。

返回值:

返回反正弦值。

4.2.2.2.6 vsip_atan_p (新增 2 个)

计算标量元素的反正切。

接口:

`vsip_scalar_f vsip_atan_f(vsip_scalar_f theta);` (新增接口)

`vsip_scalar_d vsip_atan_d(vsip_scalar_d theta);` (新增接口)

参数:

`theta`: 实数标量参数。

返回值:

返回反正切值。

4.2.2.2.7 vsip_atan2_p (新增 3 个)

计算标量元素的反正切。

接口：

```
vsip_scalar_f vsip_atan2_f(vsip_scalar_f y, vsip_scalar_d x); (新增接口)
vsip_scalar_d vsip_atan2_d(vsip_scalar_d y, vsip_scalar_d x); (新增接口)
vsip_scalar_si vsip_atan2_f_si(vsip_scalar_f y, vsip_scalar_f x); (新增接口)
```

参数：

x、y：实数标量参数。

返回值：

返回 y/x 的反正切值。

4.2.2.2.8 vsip_ceil_p

计算标量元素的上限，即找到不小于该标量的最小整数。

接口：

```
vsip_scalar_f vsip_ceil_f(vsip_scalar_f x);
vsip_scalar_d vsip_ceil_d(vsip_scalar_d x);
```

参数：

x：实数标量参数。

返回值：

返回上限值。

4.2.2.2.9 vsip_cosh_p

计算标量元素的双曲余弦。

接口：

```
vsip_scalar_f vsip_cosh_f(vsip_scalar_f x);
vsip_scalar_d vsip_cosh_d(vsip_scalar_d x);
```

参数：

x：实数标量参数。

返回值：

返回双曲余弦值。

4.2.2.2.10 vsip_cot_p

计算标量元素的余切。

接口：

```
vsip_scalar_f vsip_cot_f(vsip_scalar_f x);  
vsip_scalar_d vsip_cot_d(vsip_scalar_d x);
```

参数：

x：实数标量参数。

返回值：

返回余切值。

4.2.2.2.11 vsip_csc_p

计算标量元素的余割。

接口：

```
vsip_scalar_f vsip_csc_f(vsip_scalar_f x);  
vsip_scalar_d vsip_csc_d(vsip_scalar_d x);
```

参数：

x：实数标量参数。

返回值：

返回余割值。

4.2.2.2.12 vsip_exp_p

计算标量元素的自然指数。

接口：

```
vsip_scalar_f vsip_exp_f(vsip_scalar_f x);  
vsip_scalar_d vsip_exp_d(vsip_scalar_d x);
```

参数：

x：实数标量参数。

返回值：

返回自然指数值。

4.2.2.2.13 vsip_exp10_p

计算标量元素的以 10 为底的指数。

接口：


```
vsip_scalar_f vsip_exp10_f(vsip_scalar_f x);  
vsip_scalar_d vsip_exp10_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回以 10 为底的指数值。

4.2.2.2.14 vsip_floor_p

计算标量元素的下限，向下取整。

接口:

```
vsip_scalar_f vsip_floor_f(vsip_scalar_f x);  
vsip_scalar_d vsip_floor_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回下限值。

4.2.2.2.15 vsip_fmod_p

计算两个标量的商（即除法结果）的余数，即取模运算。

接口:

```
vsip_scalar_f vsip_fmod_f(vsip_scalar_f x, vsip_scalar_f y);  
vsip_scalar_d vsip_fmod_d(vsip_scalar_d x, vsip_scalar_d y);
```

参数:

x、y: 实数标量参数。

返回值:

返回余数。

4.2.2.2.16 vsip_hypot_p

计算两个标量的平方和的平方根（即斜边长度）。

接口:

```
vsip_scalar_f vsip_hypot_f(vsip_scalar_f a, vsip_scalar_f b);  
vsip_scalar_d vsip_hypot_d(vsip_scalar_d a, vsip_scalar_d b);
```

参数:

a、b: 实数标量参数。

返回值:

返回斜边长度值。

4.2.2.2.17 vsip_log_p

计算标量元素的自然对数。

接口:

```
vsip_scalar_f vsip_log_f(vsip_scalar_f x);  
vsip_scalar_d vsip_log_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回自然对数值。

4.2.2.2.18 vsip_log10_p

计算标量元素的以 10 为底的对数。

接口:

```
vsip_scalar_f vsip_log10_f(vsip_scalar_f x);  
vsip_scalar_d vsip_log10_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回以 10 为底的对数值。

4.2.2.2.19 vsip_mag_p

计算标量元素的绝对值。

接口:

```
vsip_scalar_f vsip_mag_f(vsip_scalar_f a);  
vsip_scalar_d vsip_mag_d(vsip_scalar_d a);
```

参数:

a: 实数标量参数。

返回值:

返回绝对值。

4.2.2.2.20 vsip_max_p

计算两个标量的最大值。

接口：

```
vsip_scalar_f vsip_max_f(vsip_scalar_f a, vsip_scalar_f b);  
vsip_scalar_d vsip_max_d(vsip_scalar_d a, vsip_scalar_d b);
```

参数：

a、b：实数标量参数。

返回值：

返回最大值。

4.2.2.2.21 vsip_min_p

计算两个标量的最小值。

接口：

```
vsip_scalar_f vsip_min_f(vsip_scalar_f a, vsip_scalar_f b);  
vsip_scalar_d vsip_min_d(vsip_scalar_d a, vsip_scalar_d b);
```

参数：

a、b：实数标量参数。

返回值：

返回最小值。

4.2.2.2.22 vsip_pow_p

计算两个标量的幂函数。

接口：

```
vsip_scalar_f vsip_pow_f(vsip_scalar_f x, vsip_scalar_f y);  
vsip_scalar_d vsip_pow_d(vsip_scalar_d x, vsip_scalar_d y);
```

参数：

x、y：实数标量参数。

返回值：

返回幂值。

4.2.2.2.23 vsip_recip_p

计算标量元素的倒数。

接口：

```
vsip_scalar_f vsip_recip_f(vsip_scalar_f x);  
vsip_scalar_d vsip_recip_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回倒数值。

4.2.2.2.24 vsip_rsqrt_p

计算标量元素的倒数平方根。

接口:

```
vsip_scalar_f vsip_rsqrt_f(vsip_scalar_f a);  
vsip_scalar_d vsip_rsqrt_d(vsip_scalar_d a);
```

参数:

a: 实数标量参数。

返回值:

返回倒数平方根值。

4.2.2.2.25 vsip_sec_p

计算标量元素的正割。

接口:

```
vsip_scalar_f vsip_sec_f(vsip_scalar_f x);  
vsip_scalar_d vsip_sec_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回正割值。

4.2.2.2.26 vsip_sinh_p

计算标量元素的双曲正弦。

接口:

```
vsip_scalar_f vsip_sinh_f(vsip_scalar_f x);  
vsip_scalar_d vsip_sinh_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回双曲正弦值。

4.2.2.2.27 vsip_sqrt_p

计算标量元素的平方根。

接口:

```
vsip_scalar_f vsip_sqrt_f(vsip_scalar_f a);  
vsip_scalar_d vsip_sqrt_d(vsip_scalar_d a);
```

参数:

a: 实数标量参数。

返回值:

返回平方根值。

4.2.2.2.28 vsip_tanh_p

计算标量元素的双曲正切。

接口:

```
vsip_scalar_f vsip_tanh_f(vsip_scalar_f x);  
vsip_scalar_d vsip_tanh_d(vsip_scalar_d x);
```

参数:

x: 实数标量参数。

返回值:

返回双曲正切值。

4.2.2.3 复数标量函数（新增 12 个）

4.2.2.3.1 vsip_cmplx_p

把两个实数标量组成一个复数的标量。

接口:

```
vsip_cscalar_f vsip_cmplx_f(vsip_scalar_f x, vsip_scalar_f y);  
vsip_cscalar_d vsip_cmplx_d(vsip_scalar_d x, vsip_scalar_d y);  
void vsip_CMPLX_f(vsip_scalar_f a, vsip_scalar_f b, vsip_cscalar_f *r);  
void vsip_CMPLX_d(vsip_scalar_d a, vsip_scalar_d b, vsip_cscalar_d *r);
```

参数:

x、y: 实数标量参数。

返回值:

返回 x 、 y 组成的复数值。

4.2.2.3.2 vsip_imag_p

提取复数的虚部。

接口:

```
vsip_scalar_f vsip_imag_f(vsip_cscalar_f x);  
vsip_scalar_d vsip_imag_d(vsip_cscalar_d x);
```

参数:

x : 复数标量参数。

返回值:

返回复数的虚部。

4.2.2.3.3 vsip_real_p

提取复数的实部。

接口:

```
vsip_scalar_f vsip_real_f(vsip_cscalar_f x);  
vsip_scalar_d vsip_real_d(vsip_cscalar_d x);
```

参数:

x : 复数标量参数。

返回值:

返回复数的实部。

4.2.2.3.4 vsip_cneg_p

计算复数标量的求反。

接口:

```
vsip_cscalar_f vsip_cneg_f(vsip_cscalar_f x);  
vsip_cscalar_d vsip_cneg_d(vsip_cscalar_d x);  
void vsip_CNEG_f(vsip_cscalar_f x, vsip_cscalar_f *r);  
void vsip_CNEG_d(vsip_cscalar_d x, vsip_cscalar_d *r);
```

参数:

x : 复数标量参数。

r : 指向输出的复数标量。

返回值:

返回 x 求反后的值或空。

4.2.2.3.5 vsip_cmul_p

计算两个标量的复数乘积。

接口：

```
vsip_cscalar_f vsip_cmul_f(vsip_cscalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_cmul_d(vsip_cscalar_d x, vsip_cscalar_d y);  
void vsip_CMUL_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);  
void vsip_CMUL_d(vsip_cscalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r);
```

参数：

x 、 y ：复数标量参数。

r ：指向输出的复数标量。

返回值：

返回复数乘积之值。

4.2.2.3.6 vsip_rcmul_p

计算实数与复数的乘积。

接口：

```
vsip_cscalar_f vsip_rcmul_f(vsip_scalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_rcmul_d(vsip_scalar_d x, vsip_cscalar_d y);  
void vsip_RCMUL_f(vsip_scalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);  
void vsip_RCMUL_d(vsip_scalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r);
```

参数：

x ：实数标量参数。

y ：复数标量参数。

r ：指向输出的复数标量。

返回值：

返回实数与复数乘积之值。

4.2.2.3.7 vsip_cdiv_p

计算两个标量的复商。

接口：

```
vsip_cscalar_f vsip_cdiv_f(vsip_cscalar_f x, vsip_cscalar_f y);  
void vsip_CDIV_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *z);
```

```
vsip_cscalar_d vsip_cdiv_d(vsip_cscalar_d x, vsip_cscalar_d y);  
void vsip_CDIV_d(vsip_cscalar_d x, vsip_cscalar_d y, vsip_cscalar_d *z);
```

参数:

x、y: 复数标量参数。

z: 复数标量之商。

返回值:

返回复数之商或空。

4.2.2.3.8 vsip_rcdiv_p (新增 2 个)

计算实数除以复数。

接口:

```
vsip_cscalar_f vsip_rcdiv_f(vsip_scalar_f x, vsip_cscalar_f y); (新增接口)  
vsip_cscalar_d vsip_rcdiv_d(vsip_scalar_d x, vsip_cscalar_d y); (新增接口)
```

参数:

x: 实数标量参数。

y: 复数标量参数。

返回值:

返回实数与复数之商。

4.2.2.3.9 vsip_crdiv_p

计算复数除以实数。

接口:

```
vsip_cscalar_f vsip_crdiv_f(vsip_cscalar_f x, vsip_scalar_f y);  
vsip_cscalar_d vsip_crdiv_d(vsip_cscalar_d x, vsip_scalar_d y);  
void vsip_CRDIV_f(vsip_cscalar_f x, vsip_scalar_f y, vsip_cscalar_f *r);  
void vsip_CRDIV_d(vsip_cscalar_d x, vsip_scalar_d y, vsip_cscalar_d *r);
```

参数:

x: 复数标量参数。

y: 实数标量参数。

r: 指向输出的复数标量。

返回值:

返回复数与实数之商。

4.2.2.3.10 vsip_cadd_p

计算两个标量复数的和。

接口：

```
vsip_cscalar_f vsip_cadd_f(vsip_cscalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_cadd_d(vsip_cscalar_d x, vsip_cscalar_d y);  
void vsip_CADD_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);  
void vsip_RCADD_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);
```

参数：

x、y：复数标量参数。

r：指向输出的复数标量。

返回值：

返回复数之和。

4.2.2.3.11 vsip_rcadd_p

计算实数与复数的和。

接口：

```
vsip_cscalar_f vsip_rcadd_f(vsip_scalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_rcadd_d(vsip_scalar_d x, vsip_cscalar_d y);  
void vsip_RCADD_f(vsip_scalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);  
void vsip_RCADD_d(vsip_scalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r);
```

参数：

x：实数标量参数。

y：复数标量参数。

r：指向输出的复数标量。

返回值：

返回实数与复数之和。

4.2.2.3.12 vsip_csub_p

计算两个复数减法。

接口：

```
vsip_cscalar_f vsip_csub_f(vsip_cscalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_csub_d(vsip_cscalar_d x, vsip_cscalar_d y);  
void vsip_CSUB_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);  
void vsip_CSUB_d(vsip_cscalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r);
```

参数:

- x、y: 复数标量参数。
- r: 指向输出的复数标量。

返回值:

返回复数之差。

4.2.2.3.13 vsip_rcsb_p

计算实数减复数。

接口:

```
vsip_cscalar_f vsip_rcsb_f(vsip_scalar_f x, vsip_cscalar_f y);  
vsip_cscalar_d vsip_rcsb_d(vsip_scalar_d x, vsip_cscalar_d y);  
void vsip_RCSUB_f(vsip_scalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r,);  
void vsip_RCSUB_d(vsip_scalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r,);
```

参数:

- x: 实数标量参数。
- y: 复数标量参数。
- r: 指向输出的复数标量。

返回值:

返回实数与复数之差。

4.2.2.3.14 vsip_crsub_p

计算复数减实数。

接口:

```
vsip_cscalar_f vsip_crsub_f(vsip_cscalar_f x, vsip_scalar_f y);  
vsip_cscalar_d vsip_crsub_d(vsip_cscalar_d x, vsip_scalar_d y);  
void vsip_CRSUB_f(vsip_cscalar_f x, vsip_scalar_f y, vsip_cscalar_f *r,);  
void vsip_CRSUB_d(vsip_cscalar_d x, vsip_scalar_d y, vsip_cscalar_d *r,);
```

参数:

- x: 复数标量参数。
- y: 实数标量参数。
- r: 指向输出的复数标量。

返回值:

返回复数与实数之差。

4.2.2.3.15 vsip_cleq_p (新增 2 个)

判断两复数是否相等。

接口：

vsip_scalar_bl vsip_cleq_f(vsip_cscalar_f x, vsip_cscalar_f y); (新增接口)

vsip_scalar_bl vsip_cleq_d(vsip_cscalar_d x, vsip_cscalar_d y); (新增接口)

参数：

x、y：复数标量参数。

返回值：

相等返回 1，否则返回 0。

4.2.2.3.16 vsip_clne_p (新增 2 个)

判断两复数是否不等。

接口：

vsip_scalar_bl vsip_clne_f(vsip_cscalar_f x, vsip_cscalar_f y); (新增接口)

vsip_scalar_bl vsip_clne_d(vsip_cscalar_d x, vsip_cscalar_d y); (新增接口)

参数：

x、y：复数标量参数。

返回值：

不等返回 1，否则返回 0。

4.2.2.3.17 vsip_cmag_p

计算复数标量的模值。

接口：

vsip_scalar_f vsip_cmag_f(vsip_cscalar_f x);

vsip_scalar_d vsip_cmag_d(vsip_cscalar_d x);

参数：

x：复数标量参数。

返回值：

返回模值。

4.2.2.3.18 vsip_arg_p

以复数标量的弧度 $[-\pi, \pi]$ 返回参数：

$$r \leftarrow \tan^{-1} \frac{\text{Im}\{x\}}{\text{Re}\{x\}}$$

接口:

```
vsip_scalar_f vsip_arg_f(vsip_cscalar_f x);  
vsip_scalar_d vsip_arg_d(vsip_cscalar_d x);
```

参数:

x: 复数标量参数。

返回值:

以弧度为形式返回实数标量。

4.2.2.3.19 vsip_conj_p

计算标量的复共轭。

接口:

```
vsip_cscalar_f vsip_conj_f(vsip_cscalar_f x);  
vsip_cscalar_d vsip_conj_d(vsip_cscalar_d x);  
void vsip_CONJ_f(vsip_cscalar_f x, vsip_cscalar_f *r);  
void vsip_CONJ_d(vsip_cscalar_d x, vsip_cscalar_d *r);
```

参数:

x: 复数标量参数。

r: 指向输出的复数标量。

返回值:

返回复数的共轭。

4.2.2.3.20 vsip_cmagsq_p

计算复数标量的幅度平方: $r \leftarrow |x|^2 = xx^*$ 。

接口:

```
vsip_scalar_f vsip_cmagsq_f(vsip_cscalar_f x);  
vsip_scalar_d vsip_cmagsq_d(vsip_cscalar_d x);
```

参数:

x: 复数标量参数。

返回值:

返回幅度的平方实数。

4.2.2.3.21 vsip_cjmul_p

计算复数标量与第二个复数标量的共轭的乘积: $r \leftarrow xy^*$ 。

接口:

```
vsip_cscalar_f vsip_cjmul_f(vsip_cscalar_f x, vsip_cscalar_f y);
vsip_cscalar_d vsip_cjmul_d(vsip_cscalar_d x, vsip_cscalar_d y);
void vsip_CJMUL_f(vsip_cscalar_f x, vsip_cscalar_f y, vsip_cscalar_f *r);
void vsip_CJMUL_d(vsip_cscalar_d x, vsip_cscalar_d y, vsip_cscalar_d *r);
```

参数:

x、y: 复数标量参数。
r: 指向输出的复数标量。

返回值:

返回复数标量与第二个复数标量的共轭的乘积。

4.2.2.3.22 vsip_polar_p

将复数标量从矩形转换为极坐标形式。极坐标数据由包含半径的实数标量和包含复数标量的自变量(角度)的相应实数标量组成: $r \leftarrow |a|$, $t \leftarrow \tan^{-1} \frac{\text{Im}\{x\}}{\text{Re}\{x\}}$ 。

接口:

```
void vsip_polar_f(vsip_cscalar_f a, vsip_scalar_f *r, vsip_scalar_f *t);
void vsip_polar_d(vsip_cscalar_d a, vsip_scalar_d *r, vsip_scalar_d *t);
```

参数:

a: 复数标量参数。
r: 输出半径的实数标量参数。
t: 输出角度的实数标量参数。

返回值:

空。

4.2.2.3.23 vsip_rect_p (新增 2 个)

将一复数极坐标转换为复数矩形。

接口:

```
vsip_cscalar_f vsip_rect_f(vsip_scalar_f r, vsip_scalar_f t);
void vsip_RECT_f(vsip_scalar_f r, vsip_scalar_f t, vsip_cscalar_f *a); (新增接口)
vsip_cscalar_d vsip_rect_d(vsip_scalar_d r, vsip_scalar_d t);
void vsip_RECT_d(vsip_scalar_d r, vsip_scalar_d t, vsip_cscalar_d *a); (新增接口)
```

参数:

r: 输入半径实数标量参数。
t: 输入角度的实数标量参数。
a: 输出矩形形式的复数标量。

返回值:

返回矩形形式的复数标量或空。

4.2.2.3.24 vsip_cexp_p

计算标量的复指数： $y \leftarrow e^a = (\cos(\text{im}\{x\}) + j\sin(\text{im}\{x\}))e^{\text{Re}\{x\}}$ 。

接口：

```
vsip_cscalar_f vsip_cexp_f(vsip_cscalar_f x);  
void vsip_CEXP_f(vsip_cscalar_f x, vsip_cscalar_f *y);  
vsip_cscalar_d vsip_cexp_d(vsip_cscalar_d x);  
void vsip_CEXP_d(vsip_cscalar_d x, vsip_cscalar_d *y);
```

参数：

x：复数标量参数。

y：输出指数复数标量。

返回值：

返回指数复数标量或空。

4.2.2.3.25 vsip_crecip_p

计算复数标量的倒数。

接口：

```
vsip_cscalar_f vsip_crecip_f(vsip_cscalar_f x);  
void vsip_CRECIP_f(vsip_cscalar_f x, vsip_cscalar_f *y);  
vsip_cscalar_d vsip_crecip_d(vsip_cscalar_d x);  
void vsip_CRECIP_d(vsip_cscalar_d x, vsip_cscalar_d *y);
```

参数：

x：复数标量参数。

y：输出倒数复数标量。

返回值：

返回倒数复数标量或空。

4.2.2.3.26 vsip_clog_p (新增 2 个)

计算复数对数值。

接口：

```
vsip_cscalar_f vsip_clog_f(vsip_cscalar_f x);  
void vsip_CLOG_f(vsip_cscalar_f x, vsip_cscalar_f *y); (新增接口)  
vsip_cscalar_d vsip_clog_d(vsip_cscalar_d x);  
void vsip_CLOG_d(vsip_cscalar_d x, vsip_cscalar_d *y); (新增接口)
```

参数:

x: 复数标量参数。

y: 输出对数复数标量。

返回值:

返回对数复数标量或空。

4.2.2.3.27 vsip_csqrt_p

计算复数标量的平方根。

接口:

```
vsip_cscalar_f vsip_csqrt_f(vsip_cscalar_f x);  
void vsip_CSQRT_f(vsip_cscalar_f x, vsip_cscalar_f *y);  
vsip_cscalar_d vsip_csqrt_d(vsip_cscalar_d x);  
void vsip_CSQRT_d(vsip_cscalar_d x, vsip_cscalar_d *y);
```

参数:

x: 复数标量参数。

y: 输出平方根复数标量。

返回值:

返回平方根复数标量或空。

4.2.2.3.28 vsip_crsqrt_p (新增 2 个)

求平方根倒数。

接口:

```
vsip_cscalar_f vsip_crsqrt_f(vsip_cscalar_f x); (新增接口)  
vsip_cscalar_d vsip_crsqrt_d(vsip_cscalar_d x); (新增接口)
```

参数:

x: 复数标量参数。

返回值:

返回平方根倒数复数标量。

4.2.2.4 索引标量函数

4.2.2.4.1 vsip_matindex

将两个向量索引组成矩阵索引。

接口:

```
vsip_scalar_mi vsip_matindex(vsip_scalar_vi row,vsip_scalar_vi col);  
void vsip_MATINDEX(vsip_scalar_vi row,vsip_scalar_vi col,vsip_scalar_mi *r);
```

参数:

row: 行向量索引。

col: 列向量索引

r: 指向输出矩阵索引的指针。

返回值:

返回矩阵索引或空。

4.2.2.4.2 vsip_rowindex

从一个矩阵索引中提取出行向量索引。

接口:

```
vsip_scalar_vi vsip_rowindex(vsip_scalar_mi index);
```

参数:

index: 矩阵索引。

返回值:

返回行向量索引。

4.2.2.4.3 vsip_colindex

从一个矩阵索引中提取出列向量索引。

接口:

```
vsip_scalar_vi vsip_colindex(vsip_scalar_mi index);
```

参数:

index: 矩阵索引。

返回值:

返回列向量索引。

4.2.3 VSIPL-FT 随机数字生成函数

4.2.3.1 函数介绍

本节介绍 VSIPL-FT 随机数生成功能。定义了两个伪随机数生成器。第一个在(0,1)的开放区间上生成均匀的随机数。第二个近似于均值和单位方差为零的高斯随机数。

4.2.3.2 随机数函数

4.2.3.2.1 vsip_randcreate

创建随机数生成器状态对象。

接口：

```
vsip_randstate *vsip_randcreate(vsip_index seed, vsip_index numprocs,  
vsip_index id, vsip_rng portable);
```

参数：

seed：初始化生成器的种子。

numprocs：生成器的数量(主序列被划分成的子序列的数量)。

id：处理器 ID(从 numprocs 子序列组中选择特定子序列的索引)。

portable：在可移植和不可移植随机数序列之间进行选择。

返回值：

返回指向 vsip_randstate 类型的随机数状态对象的指针，如果创建失败，则返回 NULL。

4.2.3.2.2 vsip_randdestroy

销毁随机数生成器状态对象。

接口：

```
int vsip_randdestroy(vsip_randstate *state);
```

参数：

state：指向随机数状态对象的指针。

返回值：

成功时返回零，失败时返回非零。

4.2.3.2.3 vsip_dsrandu_p

生成均匀分布的(伪)随机数。浮点值均匀分布在开放间隔中。整数偏差在开区间 $(0, 2^{31}-1)$ 上均匀分布。

接口:

```
void vsip_vrandu_f(vsip_randstate *state, const vsip_vview_f *r);
void vsip_cvrandu_f(vsip_randstate *state, const vsip_cvview_f *r);
void vsip_vrandu_d(vsip_randstate *state, const vsip_vview_d *r);
void vsip_cvrandu_d(vsip_randstate *state, const vsip_cvview_d *r);
vsip_scalar_d vsip_randu_d(vsip_randstate*);
vsip_scalar_f vsip_randu_f(vsip_randstate*);
vsip_cscalar_d vsip_crandu_d(vsip_randstate*);
vsip_cscalar_f vsip_crandu_f(vsip_randstate*);
```

参数:

state: 指向随机数状态对象的指针。

r: 输出矢量或矩阵视图对象。

返回值:

空或返回一个随机实数或复数。

4.2.3.2.4 vsip_dsrandn_p

生成具有均值零和单方差的近似正态分布(伪)随机偏差; $N(0,1)$ 。

接口:

```
void vsip_vrandn_f(vsip_randstate *state, const vsip_vview_f *r);
void vsip_cvrandn_f(vsip_randstate *state, const vsip_cvview_f *r);
void vsip_vrandn_d(vsip_randstate *state, const vsip_vview_d *r);
void vsip_cvrandn_d(vsip_randstate *state, const vsip_cvview_d *r);
vsip_scalar_d vsip_randn_d(vsip_randstate*);
vsip_scalar_f vsip_randn_f(vsip_randstate*);
vsip_cscalar_d vsip_crandn_d(vsip_randstate*);
vsip_cscalar_f vsip_crandn_f(vsip_randstate*);
```

参数:

state: 指向随机数状态对象的指针。

r: 输出矢量或矩阵视图对象。

返回值:

空或返回高斯随机数。

4.2.4 VSIPL-FT 向量&元素运算函数（新增 114 个）

4.2.4.1 介绍

本章节主要介绍对 VSIPL-FT 对象的单个元素或相应的元素对执行的简单操作。也可以是逐元素添加两个向量和逐元素添加两个矩阵，或者为向量或矩阵的每个元素取正弦或余弦。因此，该章节被扩展为涵盖尽可能多的元素操作，而不考虑底层对象的形状。

4.2.4.2 初等数学函数（新增 16 个）

4.2.4.2.1 vsip_sacos_p

计算向量/矩阵的每个元素的反余弦的主弧度值 $[0, +]$ 。

接口：

```
void vsip_vacos_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vacos_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_macos_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_macos_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.2.2 vsip_sasin_p

为向量/矩阵的每个元素计算反正弦的主弧度值。

接口：

```
void vsip_vasin_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vasin_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_masin_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_masin_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值:

空。

4.2.4.2.3 vsip_satan_p

计算向量/矩阵的每个元素的反正切的主弧度值。

接口:

```
void vsip_vatan_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vatan_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_matan_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_matan_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.4 vsip_satan2_p

计算两个输入向量/矩阵的元素之比的反正切的四象限弧度值。

接口:

```
void vsip_vatan2_d(const vsip_vview_d *a, const vsip_vview_d *b, const  
vsip_vview_d *r);  
void vsip_vatan2_f(const vsip_vview_f *a, const vsip_vview_f *b, const  
vsip_vview_f *r);  
void vsip_matan2_f(const vsip_mview_f *a, const vsip_mview_f *b, const  
vsip_mview_f *r);  
void vsip_matan2_d(const vsip_mview_d *a, const vsip_mview_d *b, const  
vsip_mview_d *r);
```

参数:

a: 与分子对应的输入向量/矩阵视图。

b: 与分母对应的输入向量/矩阵的视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.5 vsip_scos_p

计算向量/矩阵的每个元素的余弦。元素角度值以弧度表示。

接口：

```
void vsip_vcos_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vcos_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_mcos_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_mcos_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.2.6 vsip_scosh_p

计算向量/矩阵的每个元素的双曲余弦。

接口：

```
void vsip_vcosh_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vcosh_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_mcosh_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_mcosh_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.2.7 vsip_ssin_p

计算向量/矩阵的每个元素的正弦。元素角度值以弧度表示。

接口：

```
void vsip_vsin_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vsin_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_msin_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_msin_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.8 vsip_ssinh_p

计算向量/矩阵的每个元素的双曲正弦。

接口:

```
void vsip_vsinh_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vsinh_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_msinh_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_msinh_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.9 vsip_dsexp_p

计算向量/矩阵的每个元素的以 e 为底且以各元素为指数的指数幂值。

接口:

```
void vsip_vexp_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vexp_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_cvexp_d(const vsip_cvview_d *a, const vsip_cvview_d *r);  
void vsip_cvexp_f(const vsip_cvview_f *a, const vsip_cvview_f *r);  
void vsip_cmexp_f(const vsip_cmview_f *a, const vsip_cmview_f *r);  
void vsip_mexp_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_cmexp_d(const vsip_cmview_d *a, const vsip_cmview_d *r);  
void vsip_mexp_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.10 vsip_sexp10_p (新增 4 个)

计算向量/矩阵的每个元素的以 10 为底且以各元素为指数的指数幂值。

接口:

```
void vsip_vexp10_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vexp10_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvexp10_d(const vsip_cvview_d *a, const vsip_cvview_d *r); (新增接口)
void vsip_cvexp10_f(const vsip_cvview_f *a, const vsip_cvview_f *r); (新增接口)
void vsip_cmexp10_f(const vsip_cmview_f *a, const vsip_cmview_f *r); (新增接口)
void vsip_mexp10_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_cmexp10_d(const vsip_cmview_d *a, const vsip_cmview_d *r); (新增接口)
void vsip_mexp10_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.11 vsip_dslog_p

计算向量/矩阵的每个元素以 e 为底的对数值。

接口:

```
void vsip_vlog_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vlog_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvlog_d(const vsip_cvview_d *a, const vsip_cvview_d *r);
void vsip_cvlog_f(const vsip_cvview_f *a, const vsip_cvview_f *r);
void vsip_mlog_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_cmlog_f(const vsip_cmview_f *a, const vsip_cmview_f *r);
void vsip_mlog_d(const vsip_mview_d *a, const vsip_mview_d *r);
void vsip_cmlog_d(const vsip_cmview_d *a, const vsip_cmview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.12 vsip_slog10_p

为向量/矩阵的每个元素计算以 10 为底的对数值。

接口：

```
void vsip_vlog10_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vlog10_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_mlog10_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_mlog10_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.2.13 vsip_dssqrt_p

计算向量/矩阵的每个元素的平方根。

接口：

```
void vsip_vsqrt_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vsqrt_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvsqrt_d(const vsip_cvview_d *a, const vsip_cvview_d *r);
void vsip_cvsqrt_f(const vsip_cvview_f *a, const vsip_cvview_f *r);
void vsip_msqrt_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_cmsqrt_f(const vsip_cmview_f *a, const vsip_cmview_f *r);
void vsip_msqrt_d(const vsip_mview_d *a, const vsip_mview_d *r);
void vsip_cmsqrt_d(const vsip_cmview_d *a, const vsip_cmview_d *r);
```

参数：

a：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.2.14 vsip_stan_p

计算向量/矩阵的每个元素的正切值。元素角度值以弧度表示。

接口：

```
void vsip_vtan_d(const vsip_vview_d *a, const vsip_vview_d *r);
```



```
void vsip_vtan_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_mtan_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_mtan_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.15 vsip_stanh_p

计算向量/矩阵的每个元素的双曲正切。

接口:

```
void vsip_vtanh_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vtanh_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_mtanh_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_mtanh_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.16 vsip_vsincos_p (新增 2 个)

计算向量中元素对应的正弦值和余弦值。

接口:

```
void vsip_vsincos_d(const vsip_vview_d *a, const vsip_vview_d *rs, const  
vsip_vview_d *rc); (新增接口)  
void vsip_vsincos_f(const vsip_vview_f *a, const vsip_vview_f *rs, const  
vsip_vview_f *rc); (新增接口)
```

参数:

a: 输入向量视图。

rs: 输出 sin 值向量/矩阵视图。

rc: 输出 cos 值向量/矩阵视图。

返回值:

空。

4.2.4.2.17 vsip_vcot_p (新增 2 个)

计算向量/矩阵元素的余切。

接口:

```
void vsip_vcot_f(const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)
```

```
void vsip_vcot_d(const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.18 vsip_ssec_p (新增 4 个)

计算向量/矩阵元素的正割。

接口:

```
void vsip_vsec_f(const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)
```

```
void vsip_vsec_d(const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)
```

```
void vsip_msec_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)
```

```
void vsip_msec_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.2.19 vsip_scsc_p (新增 4 个)

计算向量/矩阵元素的余割。

接口:

```
void vsip_vcsc_f(const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)
```

```
void vsip_vcsc_d(const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)
```

```
void vsip_mcsc_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)
```

```
void vsip_mcsc_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3 一元运算函数（新增 28 个）

4.2.4.3.1 vsip_sarg_p

为复数向量/矩阵的每个元素计算区间内的弧度值参数。

接口:

```
void vsip_varg_d(const vsip_cvview_d *a, const vsip_vview_d *r);  
void vsip_varg_f(const vsip_cvview_f *a, const vsip_vview_f *r);  
void vsip_marg_f(const vsip_cmview_f *a, const vsip_mview_f *r);  
void vsip_marg_d(const vsip_cmview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.2 vsip_sceil_p_p（新增 8 个）

对于输入视图中的每个元素，舍入到不小于输入的最小整数值，并将结果按元素放置在输出视图中。

接口:

```
void vsip_vceil_d_d(const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)  
void vsip_vceil_d_i(const vsip_vview_d *a, const vsip_vview_i *r); (新增接口)  
void vsip_vceil_f_f(const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)  
void vsip_vceil_f_i(const vsip_vview_f *a, const vsip_vview_i *r); (新增接口)  
void vsip_mceil_f_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)  
void vsip_mceil_f_i(const vsip_mview_f *a, const vsip_mview_i *r); (新增接口)  
void vsip_mceil_d_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)  
void vsip_mceil_d_i(const vsip_mview_d *a, const vsip_mview_i *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.3 vsip_csconj_p

计算复向量/矩阵的每个元素的共轭。

接口：

```
void vsip_cvconj_d(const vsip_cvview_d *a, const vsip_cvview_d *r);
void vsip_cvconj_f(const vsip_cvview_f *a, const vsip_cvview_f *r);
void vsip_cmconj_f(const vsip_cmview_f *a, const vsip_cmview_f *r);
void vsip_cmconj_d(const vsip_cmview_d *a, const vsip_cmview_d *r);
```

参数：

a: 输入向量/矩阵视图。
r: 输出向量/矩阵视图。

返回值：

空。

4.2.4.3.4 vsip_dscumsum_p

计算向量/矩阵逐个元素的累加和。

接口：

```
void vsip_vcumsum_i(const vsip_vview_i *a, const vsip_vview_i *r);
void vsip_vcumsum_si(const vsip_vview_si *a, const vsip_vview_si *r);
void vsip_vcumsum_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vcumsum_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_mcumsum_f(const vsip_mview_f *a, vsip_major dir, const vsip_mview_f
*r);
void vsip_mcumsum_d(const vsip_mview_d *a, vsip_major dir, const
vsip_mview_d *r);
void vsip_mcumsum_i(const vsip_mview_i *a, vsip_major dir, const vsip_mview_i
*r);
void vsip_mcumsum_si(const vsip_mview_si *a, vsip_major dir, const
vsip_mview_si *r);
```

参数：

a: 输入向量/矩阵视图。
dir: 对于矩阵，累积总和指定总和完成的方向。
r: 输出向量/矩阵视图。

返回值：

空。

4.2.4.3.5 vsip_seuler_p

为向量/矩阵的每个元素计算复平面中单位向量的角度对应的复数。

接口：

```
void vsip_veuler_d(const vsip_vview_d *a, const vsip_cvview_d *r);  
void vsip_veuler_f(const vsip_vview_f *a, const vsip_cvview_f *r);  
void vsip_meuler_f(const vsip_mview_f *a, const vsip_cmview_f *r);  
void vsip_meuler_d(const vsip_mview_d *a, const vsip_cmview_d *r);
```

参数：

a：输入向量/矩阵视图。
r：输出向量/矩阵视图。

返回值：

空。

4.2.4.3.6 vsip_sfloor_p_p (新增 4 个)

对于输入视图中的每个元素，舍入到不大于输入的最大整数值，并将结果按元素放置在输出视图中。

接口：

```
void vsip_vfloor_d_d(const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)  
void vsip_vfloor_f_f(const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)  
void vsip_mfloor_f_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)  
void vsip_mfloor_d_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)
```

参数：

a：输入向量/矩阵视图。
r：输出向量/矩阵视图。

返回值：

空。

4.2.4.3.7 vsip_sfix_p_p (新增 4 个)

以 0 为中心取整。

接口：

```
void vsip_mfix_f_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)  
void vsip_mfix_d_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)  
void vsip_vfix_f_f(const vsip_vview_f *x, const vsip_vview_f *r); (新增接口)  
void vsip_vfix_d_d(const vsip_vview_d *x, const vsip_vview_d *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.8 vsip_dsmag_p

计算向量/矩阵的每个元素的大小。

接口:

```
void vsip_vmag_i(const vsip_vview_i *a, const vsip_vview_i *r);
void vsip_vmag_si(const vsip_vview_si *a, const vsip_vview_si *r);
void vsip_vmag_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vmag_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvmag_d(const vsip_cvview_d *a, const vsip_vview_d *r);
void vsip_cvmag_f(const vsip_cvview_f *a, const vsip_vview_f *r);
void vsip_mmag_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_cmmag_f(const vsip_cmview_f *a, const vsip_mview_f *r);
void vsip_mmag_d(const vsip_mview_d *a, const vsip_mview_d *r);
void vsip_cmmag_d(const vsip_cmview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.9 vsip_scmagsq_p

计算向量/矩阵的每个元素的大小的平方。

接口:

```
void vsip_vcmagsq_d(const vsip_cvview_d *a, const vsip_vview_d *r);
void vsip_vcmagsq_f(const vsip_cvview_f *a, const vsip_vview_f *r);
void vsip_mcmagsq_f(const vsip_cmview_f *a, const vsip_mview_f *r);
void vsip_mcmagsq_d(const vsip_cmview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.10 vsip_dsmeanval_p

返回向量/矩阵的元素平均值。

接口:

```
vsip_scalar_d vsip_vmeanval_d(const vsip_vview_d *a);  
vsip_scalar_f vsip_vmeanval_f(const vsip_vview_f *a);  
vsip_cscalar_d vsip_cvmeanval_d(const vsip_cvview_d *a);  
vsip_cscalar_f vsip_cvmeanval_f(const vsip_cvview_f *a);  
vsip_scalar_f vsip_mmeanval_f(const vsip_mview_f *a);  
vsip_cscalar_f vsip_cmmeanval_f(const vsip_cmview_f *a);  
vsip_scalar_d vsip_mmeanval_d(const vsip_mview_d *a);  
vsip_cscalar_d vsip_cmmeanval_d(const vsip_cmview_d *a);
```

参数:

a: 输入向量/矩阵视图。

返回值:

返回向量/矩阵元素的平均值。

4.2.4.3.11 vsip_dsmeansqval_p

返回向量/矩阵元素的平均幅度平方值。

接口:

```
vsip_scalar_d vsip_vmeansqval_d(const vsip_vview_d *a);  
vsip_scalar_f vsip_vmeansqval_f(const vsip_vview_f *a);  
vsip_scalar_d vsip_cvmeansqval_d(const vsip_cvview_d *a);  
vsip_scalar_f vsip_cvmeansqval_f(const vsip_cvview_f *a);  
vsip_scalar_f vsip_mmeansqval_f(const vsip_mview_f *a);  
vsip_scalar_f vsip_cmmeansqval_f(const vsip_cmview_f *a);  
vsip_scalar_d vsip_mmeansqval_d(const vsip_mview_d *a);  
vsip_scalar_d vsip_cmmeansqval_d(const vsip_cmview_d *a);
```

参数:

a: 输入向量/矩阵视图。

返回值:

返回向量/矩阵中所有元素的平方均值。

4.2.4.3.12 vsip_dvmodulate_p

通过指定的复频率计算实向量的调制：

$$r_k \leftarrow a_k(\cos(k * nu + phi) + j\sin(k * nu + phi)) = a_k e^{j(k * nu + phi)}, \quad k=0,1,\dots,N-1。$$

接口：

```
vsip_scalar_d vsip_vmodulate_d(const vsip_vview_d *a, vsip_scalar_d nu,
vsip_scalar_d phi, const vsip_cvview_d *r);
```

```
vsip_scalar_d vsip_cvmodulate_d(const vsip_cvview_d *a, vsip_scalar_d nu,
vsip_scalar_d phi, const vsip_cvview_d *r);
```

```
vsip_scalar_f vsip_vmodulate_f(const vsip_vview_f *a, vsip_scalar_f nu,
vsip_scalar_f phi, const vsip_cvview_f *r);
```

```
vsip_scalar_f vsip_cvmodulate_f(const vsip_cvview_f *a, vsip_scalar_f nu,
vsip_scalar_f phi, const vsip_cvview_f *r);
```

参数：

a：输入向量/矩阵视图。

nu：每个指数的标量频率（以弧度为单位）。

phi：以弧度为单位的标量初始阶段。

r：输出向量/矩阵视图。

返回值：

返回一个值，该值将用作下一次调用的初始阶段参数。对于长度为 N 的向量，返回值为 $\phi + \sum_0^{N-1} nu$ 。

4.2.4.3.13 vsip_dsneg_p（新增 2 个）

计算向量/矩阵的每个元素的求反。

接口：

```
void vsip_vneg_i(const vsip_vview_i *a, const vsip_vview_i *r);
```

```
void vsip_vneg_si(const vsip_vview_si *a, const vsip_vview_si *r);
```

```
void vsip_vneg_d(const vsip_vview_d *a, const vsip_vview_d *r);
```

```
void vsip_vneg_f(const vsip_vview_f *a, const vsip_vview_f *r);
```

```
void vsip_cvneg_d(const vsip_cvview_d *a, const vsip_cvview_d *r);
```

```
void vsip_cvneg_f(const vsip_cvview_f *a, const vsip_cvview_f *r);
```

```
void vsip_mneg_f(const vsip_mview_f *a, const vsip_mview_f *r);
```

```
void vsip_mneg_f_con(const vsip_mview_f *a, const vsip_mview_f *r);（新增接口）
```

```
void vsip_cmneg_f(const vsip_cmview_f *a, const vsip_cmview_f *r);
```

```
void vsip_cmneg_d(const vsip_cmview_d *a, const vsip_cmview_d *r);
```

```
void vsip_mneg_d(const vsip_mview_d *a, const vsip_mview_d *r);
```



```
void vsip_mneg_d_con(const vsip_mview_d *a, const vsip_mview_d *r); ( 新增接口 )
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.14 vsip_dsrecip_p

计算向量/矩阵的每个元素的倒数。

接口:

```
void vsip_vrecip_d(const vsip_vview_d *a, const vsip_vview_d *r);  
void vsip_vrecip_f(const vsip_vview_f *a, const vsip_vview_f *r);  
void vsip_cvrecip_d(const vsip_cvview_d *a, const vsip_cvview_d *r);  
void vsip_cvrecip_f(const vsip_cvview_f *a, const vsip_cvview_f *r);  
void vsip_mrecip_f(const vsip_mview_f *a, const vsip_mview_f *r);  
void vsip_cmrecip_f(const vsip_cmview_f *a, const vsip_cmview_f *r);  
void vsip_mrecip_d(const vsip_mview_d *a, const vsip_mview_d *r);  
void vsip_cmrecip_d(const vsip_cmview_d *a, const vsip_cmview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.15 vsip_sround_p_p (新增 4 个)

对于输入视图中的每个元素，舍入到最接近的整数值。

接口:

```
void vsip_vround_d_d(const vsip_vview_d *a, const vsip_vview_d *r); ( 新增接口 )  
void vsip_vround_f_f(const vsip_vview_f *a, const vsip_vview_f *r); ( 新增接口 )  
void vsip_mround_f_f(const vsip_mview_f *a, const vsip_mview_f *r); ( 新增接口 )  
void vsip_mround_d_d(const vsip_mview_d *a, const vsip_mview_d *r); ( 新增接口 )
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.16 vsip_dsrsqrt_p (新增 2 个)

计算向量/矩阵的每个元素的平方根的倒数。

接口:

```
void vsip_vrsqrt_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vrsqrt_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvrsqrt_d(const vsip_cvview_d *a, const vsip_cvview_d *r); (新增接口)
void vsip_cvrsqrt_f(const vsip_cvview_f *a, const vsip_cvview_f *r); (新增接口)
void vsip_mrsqrt_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_mrsqrt_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.17 vsip_dssq_p (新增 2 个)

计算向量/矩阵的每个元素的平方。

接口:

```
void vsip_vsq_d(const vsip_vview_d *a, const vsip_vview_d *r);
void vsip_vsq_f(const vsip_vview_f *a, const vsip_vview_f *r);
void vsip_cvsq_d(const vsip_cvview_d *a, const vsip_cvview_d *r); (新增接口)
void vsip_cvsq_f(const vsip_cvview_f *a, const vsip_cvview_f *r); (新增接口)
void vsip_msq_f(const vsip_mview_f *a, const vsip_mview_f *r);
void vsip_msq_d(const vsip_mview_d *a, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.3.18 vsip_dssumval_p

返回向量/矩阵的元素之和。

接口:

```
vsip_scalar_vi vsip_vsumval_bl(const vsip_vview_bl *a);
vsip_scalar_i vsip_vsumval_i(const vsip_vview_i *a);
vsip_scalar_si vsip_vsumval_si(const vsip_vview_si *a);
vsip_scalar_uc vsip_vsumval_uc(const vsip_vview_uc *a);
vsip_scalar_d vsip_vsumval_d(const vsip_vview_d *a);
vsip_scalar_f vsip_vsumval_f(const vsip_vview_f *a);
vsip_cscalar_d vsip_cvsumval_d(const vsip_cvview_d *a);
vsip_cscalar_f vsip_cvsumval_f(const vsip_cvview_f *a);
vsip_scalar_f vsip_msumval_f(const vsip_mview_f *a);
vsip_cscalar_f vsip_cmsumval_f(const vsip_cmview_f *a);
vsip_scalar_d vsip_msumval_d(const vsip_mview_d *a);
vsip_cscalar_d vsip_cmsumval_d(const vsip_cmview_d *a);
vsip_scalar_vi vsip_msumval_bl(const vsip_mview_bl *a);
```

参数：

a：输入向量/矩阵视图。

返回值：

如果输入是布尔类型，则该函数返回 vsip_scalar_vi 类型的整数值由真值的数量组成。否则，它将返回与输入向量/矩阵的类型和精度相同。

4.2.4.3.19 vsip_dssumsqval_p (新增 2 个)

返回向量/矩阵的元素的平方和。

接口：

```
vsip_scalar_d vsip_vssumsqval_d(const vsip_vview_d *a);
vsip_scalar_f vsip_vssumsqval_f(const vsip_vview_f *a);
vsip_cscalar_d vsip_cvssumsqval_d(const vsip_cvview_d *a); (新增接口)
vsip_cscalar_f vsip_cvssumsqval_f(const vsip_cvview_f *a); (新增接口)
vsip_scalar_f vsip_mssumsqval_f(const vsip_mview_f *a);
vsip_scalar_d vsip_mssumsqval_d(const vsip_mview_d *a);
```

参数：

a：输入向量/矩阵视图。

返回值：

返回向量元素的平方和。

4.2.4.4 二元运算函数（新增 23 个）

4.2.4.4.1 vsip_dsadd_p（新增 2 个）

按元素计算两个向量/矩阵对应元素的和。

接口：

```
void vsip_vadd_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i *r);  
void vsip_vadd_vi(const vsip_vview_vi *a, const vsip_vview_vi *b, const vsip_vview_vi *r);  
void vsip_vadd_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_si *r);  
void vsip_vadd_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_uc *r);  
void vsip_vadd_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);  
void vsip_vadd_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);  
void vsip_cvadd_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_cvadd_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_rcvadd_d(const vsip_vview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_rcvadd_f(const vsip_vview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_madd_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);  
void vsip_madd_d_con(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r); (新增接口)  
void vsip_madd_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);  
void vsip_madd_f_con(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r); (新增接口)  
void vsip_madd_i(const vsip_mview_i *a, const vsip_mview_i *b, const vsip_mview_i *r);  
void vsip_madd_si(const vsip_mview_si *a, const vsip_mview_si *b, const vsip_mview_si *r);
```

```
void vsip_cmadd_d(const vsip_cmview_d *a,const vsip_cmview_d *b, const
vsip_cmview_d *r);
void vsip_cmadd_f(const vsip_cmview_f *a,const vsip_cmview_f *b, const
vsip_cmview_f *r);
void vsip_rcmadd_d(const vsip_mview_d *a,const vsip_cmview_d *b, const
vsip_cmview_d *r);
void vsip_rcmadd_f(const vsip_mview_f *a,const vsip_cmview_f *b, const
vsip_cmview_f *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.2 vsip_dssadd_p

按元素计算标量和每一向量/矩阵元素的和。

接口:

```
void vsip_svadd_i(vsip_scalar_i alpha, const vsip_vview_i *b, const vsip_vview_i
*r);
void vsip_svadd_vi(vsip_scalar_vi alpha, const vsip_vview_vi *b, const
vsip_vview_vi *r);
void vsip_svadd_si(vsip_scalar_si alpha, const vsip_vview_si *b, const
vsip_vview_si *r);
void vsip_svadd_uc(vsip_scalar_uc alpha, const vsip_vview_uc *b, const
vsip_vview_uc *r);
void vsip_svadd_f(vsip_scalar_f alpha, const vsip_vview_f *b, const vsip_vview_f
*r);
void vsip_svadd_d(vsip_scalar_d alpha, const vsip_vview_d *b, const
vsip_vview_d *r);
void vsip_csvadd_d(vsip_cscalar_d alpha, const vsip_cvview_d *b, const
vsip_cvview_d *r);
void vsip_rscvadd_d(vsip_scalar_d alpha, const vsip_cvview_d *b, const
vsip_cvview_d *r);
void vsip_csvadd_f(vsip_cscalar_f alpha, const vsip_cvview_f *b, const
vsip_cvview_f *r);
void vsip_rscvadd_f(vsip_scalar_f alpha, const vsip_cvview_f *b, const
```

```
vsip_cvview_f *r);  
void vsip_csmadd_d(vsip_cscalar_d alpha, const vsip_cmview_d *b, const  
vsip_cmview_d *r);  
void vsip_csmadd_f(vsip_cscalar_f alpha, const vsip_cmview_f *b, const  
vsip_cmview_f *r);  
void vsip_rsmadd_d(vsip_scalar_d alpha, const vsip_cmview_d *b, const  
vsip_cmview_d *r);  
void vsip_rsmadd_f(vsip_scalar_f alpha, const vsip_cmview_f *b, const  
vsip_cmview_f *r);  
void vsip_smadd_d(vsip_scalar_d alpha, const vsip_mview_d *b, const  
vsip_mview_d *r);  
void vsip_smadd_f(vsip_scalar_f alpha, const vsip_mview_f *b, const  
vsip_mview_f *r);
```

参数:

alpha: 输入标量。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.3 vsip_cvaddspl_p (新增 2 个)

单精度复数累加采样接口。

接口:

```
void vsip_cvaddspl_f(const vsip_cvview_f *a, const vsip_cvview_f *r, vsip_length  
spl); (新增接口)
```

```
void vsip_cvaddspl_d(const vsip_cvview_d *a, const vsip_cvview_d *r, vsip_length  
spl); (新增接口)
```

参数:

alpha: 输入标量。

spl: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4 vsip_dsdiv_p (新增 2 个)

按元素计算两个向量/矩阵每对元素的商。

接口:

```
void vsip_vdiv_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
void vsip_vdiv_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);
void vsip_cvdiv_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);
void vsip_cvdiv_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);
void vsip_rcvdiv_d(const vsip_vview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);
void vsip_rcvdiv_f(const vsip_vview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);
void vsip_crvdiv_d(const vsip_cvview_d *a, const vsip_vview_d *b, const vsip_cvview_d *r);
void vsip_crvdiv_f(const vsip_cvview_f *a, const vsip_vview_f *b, const vsip_cvview_f *r);
void vsip_mdiv_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);
void vsip_mdiv_d_con(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r); (新增接口)
void vsip_mdiv_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);
void vsip_mdiv_f_con(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r); (新增接口)
void vsip_cmdiv_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const vsip_cmview_d *r);
void vsip_cmdiv_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const vsip_cmview_f *r);
void vsip_rcmdiv_d(const vsip_mview_d *a, const vsip_cmview_d *b, const vsip_cmview_d *r);
void vsip_rcmdiv_f(const vsip_mview_f *a, const vsip_cmview_f *b, const vsip_cmview_f *r);
void vsip_crmdiv_d(const vsip_cmview_d *a, const vsip_mview_d *b, const vsip_cmview_d *r);
```

```
void vsip_crmdiv_f(const vsip_cmview_f *a, const vsip_mview_f *b, const vsip_cmview_f *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.5 vsip_dssdiv_p

按元素计算标量和向量/矩阵每个元素的商。

接口:

```
void vsip_svdiv_d(vsip_scalar_d alpha, const vsip_vview_d *b, const vsip_vview_d *r);
```

```
void vsip_svdiv_f(vsip_scalar_f alpha, const vsip_vview_f *b, const vsip_vview_f *r);
```

```
void vsip_rscvdiv_d(vsip_scalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);
```

```
void vsip_rscvdiv_f(vsip_scalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);
```

```
void vsip_csvdiv_d(vsip_cscalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);
```

```
void vsip_csvdiv_f(vsip_cscalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);
```

```
void vsip_smdiv_d(vsip_scalar_d alpha, const vsip_mview_d *b, const vsip_mview_d *r);
```

```
void vsip_smdiv_f(vsip_scalar_f alpha, const vsip_mview_f *b, const vsip_mview_f *r);
```

```
void vsip_rscmdiv_d(vsip_scalar_d alpha, const vsip_cmview_d *b, const vsip_cmview_d *r);
```

```
void vsip_rscmdiv_f(vsip_scalar_f alpha, const vsip_cmview_f *b, const vsip_cmview_f *r);
```

```
void vsip_csmdiv_d(vsip_cscalar_d alpha, const vsip_cmview_d *b, const vsip_cmview_d *r);
```

```
void vsip_csmdiv_f(vsip_cscalar_f alpha, const vsip_cmview_f *b, const vsip_cmview_f *r);
```

参数:

alpha: 输入标量。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.6 vsip_dssdiv_p (新增 4 个)

按元素计算向量/矩阵每个元素和标量的商。

接口:

```
void vsip_vsdiv_d(const vsip_vview_d *a, vsip_scalar_d beta, const vsip_vview_d *r);
```

```
void vsip_vsdiv_d_con(const vsip_vview_d *a, vsip_scalar_d beta, const vsip_vview_d *r); (新增接口)
```

```
void vsip_vsdiv_f(const vsip_vview_f *a, vsip_scalar_f beta, const vsip_vview_f *r);
```

```
void vsip_vsdiv_f_con(const vsip_vview_f *a, vsip_scalar_f beta, const vsip_vview_f *r); (新增接口)
```

```
void vsip_cvrsdiv_d(const vsip_cvview_d *a, vsip_scalar_d beta, const vsip_cvview_d *r);
```

```
void vsip_cvrsdiv_f(const vsip_cvview_f *a, vsip_scalar_f beta, const vsip_cvview_f *r);
```

```
void vsip_msdiv_d(const vsip_mview_d *a, vsip_scalar_d beta, const vsip_mview_d *r);
```

```
void vsip_msdiv_d_con(const vsip_mview_d *a, vsip_scalar_d beta, const vsip_mview_d *r); (新增接口)
```

```
void vsip_msdiv_f(const vsip_mview_f *a, vsip_scalar_f beta, const vsip_mview_f *r);
```

```
void vsip_msdiv_f_con(const vsip_mview_f *a, vsip_scalar_f beta, const vsip_mview_f *r); (新增接口)
```

```
void vsip_cmrsdiv_d(const vsip_cmview_d *a, vsip_scalar_d beta, const vsip_cmview_d *r);
```

```
void vsip_cmrsdiv_f(const vsip_cmview_f *a, vsip_scalar_f beta, const vsip_cmview_f *r);
```

参数:

a: 输入向量/矩阵视图。

beta: 输入标量。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.7 vsip_dsexpoavg_p

按元素计算两个向量/矩阵的指数加权平均值。

接口:

```
void vsip_vexpoavg_d(vsip_scalar_d alpha, const vsip_vview_d *a, const vsip_vview_d *c);
```

```
void vsip_vexpoavg_f(vsip_scalar_f alpha, const vsip_vview_f *a, const vsip_vview_f *c);
```

```
void vsip_cvexpoavg_d(vsip_scalar_d alpha, const vsip_cvview_d *a, const vsip_cvview_d *c);
```

```
void vsip_cvexpoavg_f(vsip_scalar_f alpha, const vsip_cvview_f *a, const vsip_cvview_f *c);
```

```
void vsip_mexpoavg_f(vsip_scalar_f alpha, const vsip_mview_f *a, const vsip_mview_f *r);
```

```
void vsip_cmexpoavg_f(vsip_scalar_f alpha, const vsip_cmview_f *a, const vsip_cmview_f *r);
```

```
void vsip_mexpoavg_d(vsip_scalar_d alpha, const vsip_mview_d *a, const vsip_mview_d *r);
```

```
void vsip_cmexpoavg_d(vsip_scalar_d alpha, const vsip_cmview_d *a, const vsip_cmview_d *r);
```

参数:

alpha: 标量加权因子。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.8 vsip_shypot_p

按元素计算两个输入向量/矩阵的平方和的平方根。

接口:

```
void vsip_vhypot_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
```

```
void vsip_vhypot_f(const vsip_vview_f *a, const vsip_vview_f *b, const
```

```
vsip_vview_f *r);  
void vsip_mhypot_f(const vsip_mview_f *a, const vsip_mview_f *b, const  
vsip_mview_f *r);  
void vsip_mhypot_d(const vsip_mview_d *a, const vsip_mview_d *b, const  
vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.9 vsip_ssijmul_p

按元素计算复数向量/矩阵与第二个复数向量/矩阵的共轭的乘积。

接口:

```
void vsip_cvjmul_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const  
vsip_cvview_d *r);  
void vsip_cvjmul_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const  
vsip_cvview_f *r);  
void vsip_cmjmul_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const  
vsip_cmview_f *r);  
void vsip_cmjmul_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const  
vsip_cmview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.10 vsip_dsmul_p (新增 2 个)

按元素计算两个向量/矩阵的乘积。

接口:

```
void vsip_vmul_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i  
*r);
```

```
void vsip_vmul_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_si *r);
void vsip_vmul_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_uc *r);
void vsip_vmul_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
void vsip_vmul_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);
void vsip_rcvmul_d(const vsip_vview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);
void vsip_rcvmul_f(const vsip_vview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);
void vsip_cvmul_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *r);
void vsip_cvmul_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *r);
void vsip_mmul_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);
void vsip_mmul_d_con(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r); (新增接口)
void vsip_mmul_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);
void vsip_mmul_f_con(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r); (新增接口)
void vsip_rcmmul_d(const vsip_mview_d *a, const vsip_cmview_d *b, const vsip_cmview_d *r);
void vsip_rcmmul_f(const vsip_mview_f *a, const vsip_cmview_f *b, const vsip_cmview_f *r);
void vsip_cmmul_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const vsip_cmview_d *r);
void vsip_cmmul_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const vsip_cmview_f *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.11 vsip_dssmul_p (新增 2 个)

按元素计算标量和向量/矩阵的乘积。

接口：

```
void vsip_svmul_i(vsip_scalar_i alpha, const vsip_vview_i *b, const vsip_vview_i *r);  
void vsip_svmul_si(vsip_scalar_si alpha, const vsip_vview_si *b, const vsip_vview_si *r);  
void vsip_svmul_uc(vsip_scalar_uc alpha, const vsip_vview_uc *b, const vsip_vview_uc *r);  
void vsip_svmul_d(vsip_scalar_d alpha, const vsip_vview_d *b, const vsip_vview_d *r);  
void vsip_svmul_f(vsip_scalar_f alpha, const vsip_vview_f *b, const vsip_vview_f *r);  
void vsip_rscvmul_d(vsip_scalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_rscvmul_f(vsip_scalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_csvmul_d(vsip_cscalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_csvmul_f(vsip_cscalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_smmul_d(vsip_scalar_d alpha, const vsip_mview_d *b, const vsip_mview_d *r);  
void vsip_smmul_d_con(vsip_scalar_d alpha, const vsip_mview_d *b, const vsip_mview_d *r); (新增接口)  
void vsip_smmul_f(vsip_scalar_f alpha, const vsip_mview_f *b, const vsip_mview_f *r);  
void vsip_smmul_f_con(vsip_scalar_f alpha, const vsip_mview_f *b, const vsip_mview_f *r); (新增接口)  
void vsip_rscmmul_d(vsip_scalar_d alpha, const vsip_cmview_d *b, const vsip_cmview_d *r);  
void vsip_rscmmul_f(vsip_scalar_f alpha, const vsip_cmview_f *b, const vsip_cmview_f *r);  
void vsip_csmmul_d(vsip_cscalar_d alpha, const vsip_cmview_d *b, const vsip_cmview_d *r);  
void vsip_csmmul_f(vsip_cscalar_f alpha, const vsip_cmview_f *b, const vsip_cmview_f *r);
```

参数:

alpha: 输入标量。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.12 vsip_dvdmmul_p

按元素计算向量与矩阵的行或列的乘积。

接口:

```
void vsip_vmmul_d(const vsip_vview_d *a, const vsip_mview_d *b, vsip_major
major, const vsip_mview_d *r);
```

```
void vsip_vmmul_f(const vsip_vview_f *a, const vsip_mview_f *b, vsip_major
major, const vsip_mview_f *r);
```

```
void vsip_cvmmul_d(const vsip_cvview_d *a, const vsip_cmview_d *b,
vsip_major major, const vsip_cmview_d *r);
```

```
void vsip_cvmmul_f(const vsip_cvview_f *a, const vsip_cmview_f *b, vsip_major
major, const vsip_cmview_f *r);
```

```
void vsip_rvcmmul_d(const vsip_vview_d *a, const vsip_cmview_d *b, vsip_major
major, const vsip_cmview_d *r);
```

```
void vsip_rvcmmul_f(const vsip_vview_f *a, const vsip_cmview_f *b, vsip_major
major, const vsip_cmview_f *r);
```

参数:

a: 矢量视图（按行：长度 N，按列：长度 M）。

b: 输入 MxN 矩阵视图。

major: 行或列主序。

r: 输出 MxN 矩阵视图。

返回值:

空。

4.2.4.4.13 vsip_dssub_p（新增 2 个）

按元素计算两个向量/矩阵的差。

接口:

```
void vsip_vsub_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i
*r);
```

```
void vsip_vsub_si(const vsip_vview_si *a, const vsip_vview_si *b, const
vsip_vview_si *r);
void vsip_vsub_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const
vsip_vview_uc *r);
void vsip_vsub_d(const vsip_vview_d *a, const vsip_vview_d *b, const
vsip_vview_d *r);
void vsip_vsub_f(const vsip_vview_f *a, const vsip_vview_f *b, const
vsip_vview_f *r);
void vsip_rcvsub_d(const vsip_vview_d *a, const vsip_cvview_d *b, const
vsip_cvview_d *r);
void vsip_rcvsub_f(const vsip_vview_f *a, const vsip_cvview_f *b, const
vsip_cvview_f *r);
void vsip_crvsub_d(const vsip_cvview_d *a, const vsip_vview_d *b, const
vsip_cvview_d *r);
void vsip_crvsub_f(const vsip_cvview_f *a, const vsip_vview_f *b, const
vsip_cvview_f *r);
void vsip_cvsub_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const
vsip_cvview_d *r);
void vsip_cvsub_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const
vsip_cvview_f *r);
void vsip_msub_i(const vsip_mview_i *a, const vsip_mview_i *b, const
vsip_mview_i *r);
void vsip_msub_si(const vsip_mview_si *a, const vsip_mview_si *b, const
vsip_mview_si *r);
void vsip_msub_d(const vsip_mview_d *a, const vsip_mview_d *b, const
vsip_mview_d *r);
void vsip_msub_d_con(const vsip_mview_d *a, const vsip_mview_d *b, const
vsip_mview_d *r); ( 新增接口 )
void vsip_msub_f(const vsip_mview_f *a, const vsip_mview_f *b, const
vsip_mview_f *r);
void vsip_msub_f_con(const vsip_mview_f *a, const vsip_mview_f *b, const
vsip_mview_f *r); ( 新增接口 )
void vsip_rcmsub_d(const vsip_mview_d *a, const vsip_cmview_d *b, const
vsip_cmview_d *r);
void vsip_rcmsub_f(const vsip_mview_f *a, const vsip_cmview_f *b, const
vsip_cmview_f *r);
void vsip_crmsub_d(const vsip_cmview_d *a, const vsip_mview_d *b, const
vsip_cmview_d *r);
```

```
void vsip_crmsub_f(const vsip_cmview_f *a, const vsip_mview_f *b, const vsip_cmview_f *r);  
void vsip_cmsub_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const vsip_cmview_d *r);  
void vsip_cmsub_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const vsip_cmview_f *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.14 vsip_dsssub_p

按元素计算标量和向量/矩阵的差。

接口:

```
void vsip_svsub_i(vsip_scalar_i alpha, const vsip_vview_i *b, const vsip_vview_i *r);  
void vsip_svsub_vi(vsip_scalar_vi alpha, const vsip_vview_vi *b, const vsip_vview_vi *r);  
void vsip_svsub_si(vsip_scalar_si alpha, const vsip_vview_si *b, const vsip_vview_si *r);  
void vsip_svsub_uc(vsip_scalar_uc alpha, const vsip_vview_uc *b, const vsip_vview_uc *r);  
void vsip_svsub_d(vsip_scalar_d alpha, const vsip_vview_d *b, const vsip_vview_d *r);  
void vsip_svsub_f(vsip_scalar_f alpha, const vsip_vview_f *b, const vsip_vview_f *r);  
void vsip_rscvsub_d(vsip_scalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_rscvsub_f(vsip_scalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_csvsub_d(vsip_cscalar_d alpha, const vsip_cvview_d *b, const vsip_cvview_d *r);  
void vsip_csvsub_f(vsip_cscalar_f alpha, const vsip_cvview_f *b, const vsip_cvview_f *r);  
void vsip_smsub_i(vsip_scalar_i alpha, const vsip_mview_i *b, const
```



```
vsip_mview_i *r);  
    void vsip_smsub_si(vsip_scalar_si alpha, const vsip_mview_si *b, const  
vsip_mview_si *r);  
    void vsip_smsub_d(vsip_scalar_d alpha, const vsip_mview_d *b, const  
vsip_mview_d *r);  
    void vsip_smsub_f(vsip_scalar_f alpha, const vsip_mview_f *b, const  
vsip_mview_f *r);  
    void vsip_rscmsub_d(vsip_scalar_d alpha, const vsip_cmview_d *b, const  
vsip_cmview_d *r);  
    void vsip_rscmsub_f(vsip_scalar_f alpha, const vsip_cmview_f *b, const  
vsip_cmview_f *r);  
    void vsip_csmsub_d(vsip_cscalar_d alpha, const vsip_cmview_d *b, const  
vsip_cmview_d *r);  
    void vsip_csmsub_f(vsip_cscalar_f alpha, const vsip_cmview_f *b, const  
vsip_cmview_f *r);
```

参数:

alpha: 输入标量。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.15 vsip_vssub_p (新增 3 个)

按元素计算向量/矩阵和标量的差。

接口:

```
void vsip_vssub_f(const vsip_vview_f *a, vsip_scalar_f beta, const vsip_vview_f  
*r); (新增接口)  
void vsip_vssub_d(const vsip_vview_d *a, vsip_scalar_d beta, const  
vsip_vview_d *r); (新增接口)  
void vsip_vssub_i(const vsip_vview_i *a, vsip_scalar_i beta, const vsip_vview_i  
*r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

beta: 输入标量。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.4.16 vsip_vpow_p (新增 2 个)

求幂指数。

接口：

```
void vsip_vpow_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r); (新增接口)
```

```
void vsip_vpow_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r); (新增接口)
```

参数：

a：输入底数向量。

b：输入指数向量。

r：输出乘幂结果向量。

返回值：

空。

4.2.4.4.17 vsip_svpow_p (新增 2 个)

求幂指数。

接口：

```
void vsip_svpow_f(vsip_scalar_f beta, const vsip_vview_f *a, const vsip_vview_f *r); (新增接口)
```

```
void vsip_svpow_d(vsip_scalar_d beta, const vsip_vview_d *a, const vsip_vview_d *r); (新增接口)
```

参数：

a：输入底数向量。

beta：标量指数。

r：输出乘幂结果向量。

返回值：

空。

4.2.4.5 三元运算函数

4.2.4.5.1 vsip_dvam_p

按元素计算两个向量和第三个向量的乘积之和。

接口:

```
void vsip_vam_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vam_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvam_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvam_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

b: 输入向量视图。

c: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.4.5.2 vsip_dvma_p

按元素计算两个向量与第三个向量之和的乘积。

接口:

```
void vsip_vma_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vma_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvma_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvma_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

b: 输入向量视图。

c: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.4.5.3 vsip_dvmsa_p

按元素计算两个向量的乘积和标量的总和。

接口：

```
void vsip_vmsa_d(const vsip_vview_d *a, const vsip_vview_d *b, vsip_scalar_d alpha, const vsip_vview_d *r);
```

```
void vsip_vmsa_f(const vsip_vview_f *a, const vsip_vview_f *b, vsip_scalar_f alpha, const vsip_vview_f *r);
```

```
void vsip_cvmsa_d(const vsip_cvview_d *a, const vsip_cvview_d *b, vsip_cscalar_d alpha, const vsip_cvview_d *r);
```

```
void vsip_cvmsa_f(const vsip_cvview_f *a, const vsip_cvview_f *b, vsip_cscalar_f alpha, const vsip_cvview_f *r);
```

参数：

a：输入向量视图。

b：输入向量视图。

alpha：输入标量。

r：输出向量视图。

返回值：

空。

4.2.4.5.4 vsip_dvmsb_p

按元素计算两个向量与第三个向量的差的乘积。

接口：

```
void vsip_vmsb_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vmsb_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvmsb_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvmsb_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数：

a：输入向量视图。

b：输入向量视图。

c：输入向量视图。

r：输出向量视图。

返回值:

空。

4.2.4.5.5 vsip_dvsam_p

计算向量和标量的总和，并与第二个向量的乘积，按元素。

接口:

```
void vsip_vsam_d(const vsip_vview_d *a, vsip_scalar_d beta, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vsam_f(const vsip_vview_f *a, vsip_scalar_f beta, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvsam_d(const vsip_cvview_d *a, vsip_cscalar_d beta, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvsam_f(const vsip_cvview_f *a, vsip_cscalar_f beta, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

beta: 输入标量。

c: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.4.5.6 vsip_dvsbm_p

计算两个向量的差，并按元素与第三个向量的乘积。

接口:

```
void vsip_vsbm_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vsbm_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvsbm_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvsbm_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

b: 输入向量视图。

c: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.4.5.7 vsip_dvsma_p

计算向量和标量的乘积，并按元素与第二个向量求和。

接口:

```
void vsip_vsma_d(const vsip_vview_d *a, vsip_scalar_d beta, const vsip_vview_d *c, const vsip_vview_d *r);
```

```
void vsip_vsma_f(const vsip_vview_f *a, vsip_scalar_f beta, const vsip_vview_f *c, const vsip_vview_f *r);
```

```
void vsip_cvsma_d(const vsip_cvview_d *a, vsip_cscalar_d beta, const vsip_cvview_d *c, const vsip_cvview_d *r);
```

```
void vsip_cvsma_f(const vsip_cvview_f *a, vsip_cscalar_f beta, const vsip_cvview_f *c, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

beta: 输入标量。

c: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.4.5.8 vsip_dvsmsa_p

按元素计算向量和标量的乘积，并与第二个标量求和。

接口:

```
void vsip_vsmsa_d(const vsip_vview_d *a, vsip_scalar_d beta, vsip_scalar_d gamma, const vsip_vview_d *r);
```

```
void vsip_vsmsa_f(const vsip_vview_f *a, vsip_scalar_f beta, vsip_scalar_f gamma, const vsip_vview_f *r);
```

```
void vsip_cvsmsa_d(const vsip_cvview_d *a, vsip_cscalar_d beta, vsip_cscalar_d gamma, const vsip_cvview_d *r);
```

```
void vsip_cvsmsa_f(const vsip_cvview_f *a, vsip_cscalar_f beta, vsip_cscalar_f gamma, const vsip_cvview_f *r);
```

```
gamma, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

beta: 输入标量。

gamma: 输入标量。

r: 输出向量视图。

返回值:

空。

4.2.4.6 逻辑运算函数 (新增 10 个)

4.2.4.6.1 vsip_salltrue_bl

如果向量/矩阵的所有元素均为 true, 则返回 true。

接口:

```
vsip_scalar_bl vsip_valltrue_bl(const vsip_vview_bl *a);
```

```
vsip_scalar_bl vsip_malltrue_bl(const vsip_mview_bl *a);
```

4.2.4.6.2 vsip_sanytrue_bl

如果向量/矩阵的一个或多个元素为 true, 则返回 true。

接口:

```
vsip_scalar_bl vsip_vanytrue_bl(const vsip_vview_bl *a);
```

```
vsip_scalar_bl vsip_manytrue_bl(const vsip_mview_bl *a);
```

参数:

a: 输入向量视图。

返回值:

如果任一元素为 false, 则返回 false, 否则返回 true。

4.2.4.6.3 vsip_dsleq_p (新增 4 个)

按元素计算两个向量/矩阵的“相等”的布尔比较。

接口:

```
void vsip_vleq_i(const vsip_vview_i *a, const vsip_vview_i *b, const  
vsip_vview_bl *r);
```

```
void vsip_vleq_si(const vsip_vview_si *a, const vsip_vview_si *b, const  
vsip_vview_bl *r);
```

```
void vsip_vleq_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const
vsip_vview_bl *r);
void vsip_vleq_d(const vsip_vview_d *a, const vsip_vview_d *b, const
vsip_vview_bl *r);
void vsip_vleq_f(const vsip_vview_f *a, const vsip_vview_f *b, const
vsip_vview_bl *r);
void vsip_cvleq_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const
vsip_vview_bl *r); (新增接口)
void vsip_cvleq_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const
vsip_vview_bl *r); (新增接口)
void vsip_mleq_f(const vsip_mview_f *a, const vsip_mview_f *b, const
vsip_mview_bl *r);
void vsip_mleq_d(const vsip_mview_d *a, const vsip_mview_d *b, const
vsip_mview_bl *r);
vsip_scalar_bl vsip_meq_d_con(const vsip_mview_d *a, const vsip_mview_d *b);
(新增接口)
vsip_scalar_bl vsip_meq_f_con(const vsip_mview_f *a, const vsip_mview_f *b);
(新增接口)
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空或矩阵相等返回 1、不等返回 0。

4.2.4.6.4 vsip_dslge_p

按元素计算两个向量/矩阵的“大于或等于”的布尔比较。

接口:

```
void vsip_vlge_i(const vsip_vview_i *a, const vsip_vview_i *b, const
vsip_vview_bl *r);
void vsip_vlge_si(const vsip_vview_si *a, const vsip_vview_si *b, const
vsip_vview_bl *r);
void vsip_vlge_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const
vsip_vview_bl *r);
void vsip_vlge_d(const vsip_vview_d *a, const vsip_vview_d *b, const
vsip_vview_bl *r);
void vsip_vlge_f(const vsip_vview_f *a, const vsip_vview_f *b, const
```



```
vsip_vview_bl *r);  
void vsip_mlge_f(const vsip_mview_f *a, const vsip_mview_f *b, const  
vsip_mview_bl *r);  
void vsip_mlge_d(const vsip_mview_d *a, const vsip_mview_d *b, const  
vsip_mview_bl *r);
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空。

4.2.4.6.5 vsip_dslgt_p

按元素计算两个向量/矩阵的“大于”的布尔比较。

接口:

```
void vsip_vlgt_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_bl  
*r);  
void vsip_vlgt_si(const vsip_vview_si *a, const vsip_vview_si *b, const  
vsip_vview_bl *r);  
void vsip_vlgt_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const  
vsip_vview_bl *r);  
void vsip_vlgt_d(const vsip_vview_d *a, const vsip_vview_d *b, const  
vsip_vview_bl *r);  
void vsip_vlgt_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_bl  
*r);  
void vsip_mlgt_f(const vsip_mview_f *a, const vsip_mview_f *b, const  
vsip_mview_bl *r);  
void vsip_mlgt_d(const vsip_mview_d *a, const vsip_mview_d *b, const  
vsip_mview_bl *r);
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空。

4.2.4.6.6 vsip_dsle_p

按元素计算两个向量/矩阵的“小于或等于”的布尔比较。

接口：

```
void vsip_vlle_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_bl *r);  
void vsip_vlle_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_bl *r);  
void vsip_vlle_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_bl *r);  
void vsip_vlle_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_bl *r);  
void vsip_vlle_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_bl *r);  
void vsip_mlle_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_bl *r);  
void vsip_mlle_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_bl *r);
```

参数：

- a：输入向量/矩阵视图。
- b：输入向量/矩阵视图。
- r：布尔输出向量/矩阵的视图。

返回值：

空。

4.2.4.6.7 vsip_dsllt_p

按元素计算两个向量/矩阵的“小于”的布尔比较。

接口：

```
void vsip_vllt_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_bl *r);  
void vsip_vllt_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_bl *r);  
void vsip_vllt_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_bl *r);  
void vsip_vllt_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_bl *r);
```

```
void vsip_vllt_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_bl *r);  
void vsip_mllt_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_bl *r);  
void vsip_mllt_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_bl *r);
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空。

4.2.4.6.8 vsip_dsslt_p (新增 2 个)

按元素计算标量常量与向量/矩阵的“小于”的布尔比较。

接口:

```
void vsip_svllt_d(vsip_scalar_d alpha, const vsip_vview_d *b, const vsip_vview_bl *r); (新增接口)  
void vsip_svllt_f(vsip_scalar_f alpha, const vsip_vview_f *b, const vsip_vview_bl *r); (新增接口)
```

参数:

- alpha: 标量常量。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空。

4.2.4.6.9 vsip_dslne_p (新增 4 个)

按元素计算两个向量/矩阵的“不相等”的布尔比较。

接口:

```
void vsip_vlne_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_bl *r);  
void vsip_vlne_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_bl *r);  
void vsip_vlne_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_bl *r);
```

```
vsip_vview_bl *r);
void vsip_vlne_d(const vsip_vview_d *a, const vsip_vview_d *b, const
vsip_vview_bl *r);
void vsip_vlne_f(const vsip_vview_f *a, const vsip_vview_f *b, const
vsip_vview_bl *r);
void vsip_cvlne_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const
vsip_vview_bl *r); (新增接口)
void vsip_cvlne_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const
vsip_vview_bl *r); (新增接口)
void vsip_mlne_f(const vsip_mview_f *a, const vsip_mview_f *b, const
vsip_mview_bl *r);
void vsip_mlne_d(const vsip_mview_d *a, const vsip_mview_d *b, const
vsip_mview_bl *r);
vsip_scalar_bl vsip_mineq_d_con(const vsip_mview_d *a, const vsip_mview_d
*b); (新增接口)
vsip_scalar_bl vsip_mineq_f_con(const vsip_mview_f *a, const vsip_mview_f *b);
(新增接口)
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 布尔输出向量/矩阵的视图。

返回值:

空或矩阵不相等返回 1、相等返回 0。

4.2.4.7 选择/最值运算函数 (新增 2 个)

4.2.4.7.1 vsip_sclip_p

按元素计算两个向量/矩阵的广义双剪辑。

接口:

```
void vsip_mclip_i(const vsip_mview_i *a, vsip_scalar_i t1, vsip_scalar_i t2,
vsip_scalar_i c1, vsip_scalar_i c2, const vsip_mview_i *r);
void vsip_mclip_si(const vsip_mview_si *a, vsip_scalar_si t1, vsip_scalar_si t2,
vsip_scalar_si c1, vsip_scalar_si c2, const vsip_mview_si *r);
void vsip_vclip_i(const vsip_vview_i *a, vsip_scalar_i t1, vsip_scalar_i t2,
vsip_scalar_i c1, vsip_scalar_i c2, const vsip_vview_i *r);
void vsip_vclip_si(const vsip_vview_si *a, vsip_scalar_si t1, vsip_scalar_si t2,
vsip_scalar_si c1, vsip_scalar_si c2, const vsip_vview_si *r);
```

```

void vsip_vclip_uc(const vsip_vview_uc *a, vsip_scalar_uc t1, vsip_scalar_uc t2,
vsip_scalar_uc c1, vsip_scalar_uc c2, const vsip_vview_uc *r);
void vsip_vclip_d(const vsip_vview_d *a, vsip_scalar_d t1, vsip_scalar_d t2,
vsip_scalar_d c1, vsip_scalar_d c2, const vsip_vview_d *r);
void vsip_vclip_f(const vsip_vview_f *a, vsip_scalar_f t1, vsip_scalar_f t2,
vsip_scalar_f c1, vsip_scalar_f c2, const vsip_vview_f *r);
void vsip_mclip_f(const vsip_mview_f *a, vsip_scalar_f t1, vsip_scalar_f t2,
vsip_scalar_f c1, vsip_scalar_f c2, const vsip_mview_f *r);
void vsip_mclip_d(const vsip_mview_d *a, vsip_scalar_d t1, vsip_scalar_d t2,
vsip_scalar_d c1, vsip_scalar_d c2, const vsip_mview_d *r);

```

参数:

- a: 输入向量/矩阵视图。
- t1: 阈值下限。
- t2: 阈值上限。
- c1: 下限阈值剪辑值。
- c2: 上限阈值剪辑值。
- r: 输出向量/矩阵的视图。

返回值:

空。

4.2.4.7.2 vsip_dvfirst_p

返回由元素应用的用户指定的二元标量函数返回 true 的一对向量视图对象的第一个元素的索引。

接口:

```

vsip_index vsip_vfirst_f(vsip_index j, vsip_bool
(*f)(vsip_scalar_f,vsip_scalar_f),const vsip_vview_f *x, const vsip_vview_f *y);
vsip_index vsip_vfirst_d(vsip_index j, vsip_bool
(*f)(vsip_scalar_d,vsip_scalar_d),const vsip_vview_d *x, const vsip_vview_d *y);
vsip_index vsip_vfirst_i(vsip_index j, vsip_bool
(*f)(vsip_scalar_i,vsip_scalar_i),const vsip_vview_i *x, const vsip_vview_i *y);
vsip_index vsip_vfirst_vi(vsip_index j, vsip_bool
(*f)(vsip_scalar_vi,vsip_scalar_vi),const vsip_vview_vi *x, const vsip_vview_vi *y);
vsip_index vsip_vfirst_mi(vsip_index j, vsip_bool
(*f)(vsip_scalar_mi,vsip_scalar_mi),const vsip_vview_mi *x, const vsip_vview_mi *y);

```

参数:

- j: 用户指定的搜索起始索引。

f: 用户指定的两个标量的二进制函数，返回一个布尔值。

x: x 操作数的矢量视图对象。

y: y 操作数的矢量视图对象

返回值:

此函数返回的向量索引值为:

- 1.j, 如果 j 大于向量视图参数的长度;
- 2.长度, 如果 $j \leq k \leq \text{length}$, $f(x_k, y_k)$ 为 false;
- 3.第一个向量索引, 如果 $k \geq j$, 其 $f(x_k, y_k)$ 不为 false。

4.2.4.7.3 vsip_sinvclip_p

按元素计算两个向量/矩阵的广义反向双剪辑:

$$r_j = \begin{cases} a_j, a_j < t_1 \\ c_1, a_j < t_2 \\ c_2, a_j \leq t_3 \\ a_j, \text{其它} \end{cases}$$

为矩阵时, $r_{i,j}$ 对应 r_j , $a_{i,j}$ 对应 a_j 。 $j=0,1,\dots,N-1$, $i=0,1,\dots,N-1$ 。

接口:

```
void vsip_vinvclip_i(const vsip_vview_i *a, vsip_scalar_i t1, vsip_scalar_i t2,
vsip_scalar_i t3, vsip_scalar_i c1, vsip_scalar_i c2, const vsip_vview_i *r);
```

```
void vsip_vinvclip_si(const vsip_vview_si *a, vsip_scalar_si t1, vsip_scalar_si t2,
vsip_scalar_si t3, vsip_scalar_si c1, vsip_scalar_si c2, const vsip_vview_si *r);
```

```
void vsip_vinvclip_uc(const vsip_vview_uc *a, vsip_scalar_uc t1, vsip_scalar_uc
t2, vsip_scalar_uc t3, vsip_scalar_uc c1, vsip_scalar_uc c2, const vsip_vview_uc *r);
```

```
void vsip_vinvclip_d(const vsip_vview_d *a, vsip_scalar_d t1, vsip_scalar_d t2,
vsip_scalar_d t3, vsip_scalar_d c1, vsip_scalar_d c2, const vsip_vview_d *r);
```

```
void vsip_vinvclip_f(const vsip_vview_f *a, vsip_scalar_f t1, vsip_scalar_f t2,
vsip_scalar_f t3, vsip_scalar_f c1, vsip_scalar_f c2, const vsip_vview_f *r);
```

```
void vsip_minvclip_f(const vsip_mview_f *a, vsip_scalar_f t1, vsip_scalar_f t2,
vsip_scalar_f t3, vsip_scalar_f c1, vsip_scalar_f c2, const vsip_mview_f *r);
```

```
void vsip_minvclip_d(const vsip_mview_d *a, vsip_scalar_d t1, vsip_scalar_d t2,
vsip_scalar_d t3, vsip_scalar_d c1, vsip_scalar_d c2, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

t1: 阈值下限。

t2: 中间阈值。

t3: 阈值上限。

c1: 下限阈值剪辑值。
c2: 上限阈值剪辑值。
r: 输出向量/矩阵的视图。

返回值:

空。

4.2.4.7.4 vsip_sindexbool_p

计算布尔向量/矩阵的非假元素的索引向量，并返回非假元素的数量。

接口:

```
vsip_length vsip_vindexbool( const vsip_vview_bl *b, vsip_vview_vi *index);  
vsip_length vsip_mindexbool( const vsip_mview_bl *b, vsip_vview_mi *index);
```

参数:

b: 输入布尔向量/矩阵的视图。
index: 输出向量/矩阵视图。

返回值:

返回值是非 false 元素的数量。

4.2.4.7.5 vsip_smax_p

按元素计算两个向量/矩阵的最大值。

接口:

```
void vsip_vmax_d(const vsip_vview_d *a, const vsip_vview_d *b, const  
vsip_vview_d *r);  
void vsip_vmax_f(const vsip_vview_f *a, const vsip_vview_f *b, const  
vsip_vview_f *r);  
void vsip_mmax_f(const vsip_mview_f *a, const vsip_mview_f *b, const  
vsip_mview_f *r);  
void vsip_mmax_d(const vsip_mview_d *a, const vsip_mview_d *b, const  
vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。
b: 输入向量/矩阵视图。
r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.7.6 vsip_smaxmg_p

按元素计算两个向量/矩阵的最大量值(绝对值)。

接口:

```
void vsip_vmaxmg_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
```

```
void vsip_vmaxmg_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);
```

```
void vsip_mmaxmg_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);
```

```
void vsip_mmaxmg_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.7.7 vsip_scmamaxgsq_p

按元素计算两个复数向量/矩阵的最大幅度平方。

接口:

```
void vsip_vcmamaxgsq_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_vview_d *r);
```

```
void vsip_vcmamaxgsq_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_vview_f *r);
```

```
void vsip_mcmamaxgsq_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const vsip_mview_f *r);
```

```
void vsip_mcmamaxgsq_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.7.8 vsip_scmxmsgsqval_p

返回复数向量/矩阵的元素的最大大小平方的索引和值。索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_d vsip_vcmxmsgsqval_d(const vsip_cvview_d *a, vsip_index *index);  
vsip_scalar_f vsip_vcmxmsgsqval_f(const vsip_cvview_f *a, vsip_index *index);  
vsip_scalar_f vsip_mcmxmsgsqval_f(const vsip_cmview_f *a, vsip_scalar_mi *index);  
vsip_scalar_d vsip_mcmxmsgsqval_d(const vsip_cmview_d *a, vsip_scalar_mi *index);
```

参数：

a：输入向量/矩阵的视图。

index：指向索引的指针，如果为 NULL，则不返回索引。

返回值：

返回元素的最大量级平方值。索引使用索引指针（如果为非 NULL）。

4.2.4.7.9 vsip_smaxmgval_p

返回向量/矩阵的元素的最大绝对值的索引和值。索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_d vsip_vmaxmgval_d(const vsip_vview_d *a, vsip_index *index);  
vsip_scalar_f vsip_vmaxmgval_f(const vsip_vview_f *a, vsip_index *index);  
vsip_scalar_f vsip_mmaxmgval_f(const vsip_mview_f *a, vsip_scalar_mi *index);  
vsip_scalar_d vsip_mmaxmgval_d(const vsip_mview_d *a, vsip_scalar_mi *index);
```

参数：

a：输入向量/矩阵的视图。

index：指向索引的指针，如果为 NULL，则不返回索引。

返回值：

返回元素的最大绝对值。索引使用索引指针（如果为非 NULL）。

4.2.4.7.10 vsip_smaxval_p

返回向量/矩阵的元素的最大值的索引和值。该索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_i vsip_mmaxval_i(const vsip_mview_i *a, vsip_scalar_mi *index);  
vsip_scalar_i vsip_vmaxval_i(const vsip_vview_i *a, vsip_index *index);  
vsip_scalar_si vsip_mmaxval_si(const vsip_mview_si *a, vsip_scalar_mi *index);  
vsip_scalar_si vsip_vmaxval_si(const vsip_vview_si *a, vsip_index *index);  
vsip_scalar_d vsip_vmaxval_d(const vsip_vview_d *a, vsip_index *index);  
vsip_scalar_f vsip_vmaxval_f(const vsip_vview_f *a, vsip_index *index);  
vsip_scalar_f vsip_mmaxval_f(const vsip_mview_f *a, vsip_scalar_mi *index);  
vsip_scalar_d vsip_mmaxval_d(const vsip_mview_d *a, vsip_scalar_mi *index);
```

参数:

a: 输入向量/矩阵的视图。

index: 指向索引的指针, 如果为 NULL, 则不返回索引。

返回值:

返回元素的最大值。索引使用索引指针 (如果为非 NULL) 。

4.2.4.7.11 vsip_vmaxorder_i (新增 1 个)

实现求向量中 2 为底的最大阶数。

接口:

```
vsip_scalar_i vsip_vmaxorder_i(const vsip_vview_i* a); ( 新增接口 )
```

参数:

a: 输入向量视图。

返回值:

返回向量中 2 为底的最大阶数数值。

4.2.4.7.12 vsip_vcsummgval_f (新增 1 个)

实现求复数向量元素模值总和。

接口:

```
vsip_scalar_f (vsip_vcsummgval_f)(const vsip_cvview_f* a); ( 新增接口 )
```

参数:

a: 输入向量视图。

返回值:

返回复数向量元素模值总和。

4.2.4.7.13 vsip_smin_p

按元素计算两个向量/矩阵的最小值。

接口:

```
void vsip_vmin_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
```

```
void vsip_vmin_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);
```

```
void vsip_mmin_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);
```

```
void vsip_mmin_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.7.14 vsip_sminmg_p

按元素计算两个向量/矩阵的最小量值(绝对值)。

接口:

```
void vsip_vminmg_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_vview_d *r);
```

```
void vsip_vminmg_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_vview_f *r);
```

```
void vsip_mminmg_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r);
```

```
void vsip_mminmg_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.7.15 vsip_scmminmgsq_p

按元素计算两个复数向量/矩阵的最小幅度平方。

接口：

```
void vsip_vcminmgsq_d(const vsip_cvview_d *a, const vsip_cvview_d *b, const vsip_vview_d *r);
```

```
void vsip_vcminmgsq_f(const vsip_cvview_f *a, const vsip_cvview_f *b, const vsip_vview_f *r);
```

```
void vsip_mcminmgsq_f(const vsip_cmview_f *a, const vsip_cmview_f *b, const vsip_mview_f *r);
```

```
void vsip_mcminmgsq_d(const vsip_cmview_d *a, const vsip_cmview_d *b, const vsip_mview_d *r);
```

参数：

a：输入向量/矩阵视图。

b：输入向量/矩阵视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.7.16 vsip_scmminmgsqval_p

返回复数向量/矩阵的元素的最小幅度平方的索引和值。索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_d vsip_vcminmgsqval_d(const vsip_cvview_d *a, vsip_index *index);
```

```
vsip_scalar_f vsip_vcminmgsqval_f(const vsip_cvview_f *a, vsip_index *index);
```

```
vsip_scalar_f vsip_mcminmgsqval_f(const vsip_cmview_f *a, vsip_scalar_mi *index);
```

```
vsip_scalar_d vsip_mcminmgsqval_d(const vsip_cmview_d *a, vsip_scalar_mi *index);
```

参数：

a：输入向量/矩阵的视图。

index：指向索引的指针，如果为 NULL，则不返回索引。

返回值：

返回元素的最小量级平方值。索引使用索引指针（如果为非 NULL）。

4.2.4.7.17 vsip_sminmgval_p

返回向量/矩阵的元素的最小绝对值的索引和值。索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_d vsip_vminmgval_d(const vsip_vview_d *a, vsip_index *index);  
vsip_scalar_f vsip_vminmgval_f(const vsip_vview_f *a, vsip_index *index);  
vsip_scalar_f vsip_mminmgval_f(const vsip_mview_f *a, vsip_scalar_mi *index);  
vsip_scalar_d vsip_mminmgval_d(const vsip_mview_d *a, vsip_scalar_mi *index);
```

参数：

a：输入向量/矩阵的视图。

index：指向索引的指针，如果为 NULL，则不返回索引。

返回值：

返回元素的最小绝对值。索引使用索引指针（如果为非 NULL）。

4.2.4.7.18 vsip_sminval_p

返回向量/矩阵的元素的最小值的索引和值。该索引通过引用作为参数之一返回。

接口：

```
vsip_scalar_d vsip_vminval_d(const vsip_vview_d *a, vsip_index *index);  
vsip_scalar_f vsip_vminval_f(const vsip_vview_f *a, vsip_index *index);  
vsip_scalar_f vsip_mminval_f(const vsip_mview_f *a, vsip_scalar_mi *index);  
vsip_scalar_d vsip_mminval_d(const vsip_mview_d *a, vsip_scalar_mi *index);
```

参数：

a：输入向量/矩阵的视图。

index：指向索引的指针，如果为 NULL，则不返回索引。

返回值：

返回元素的最小值。索引使用索引指针（如果为非 NULL）。

4.2.4.8 逐位布尔逻辑运算函数（新增 1 个）

4.2.4.8.1 vsip_sand_p

按元素计算两个向量/矩阵的“AND”。

接口：

```
void vsip_mand_i(const vsip_mview_i *a, const vsip_mview_i *b, const vsip_mview_i *r);
```

```
void vsip_mand_si(const vsip_mview_si *a, const vsip_mview_si *b, const
vsip_mview_si *r);
void vsip_vand_bl(const vsip_vview_bl *a, const vsip_vview_bl *b, const
vsip_vview_bl *r);
void vsip_vand_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i
*r);
void vsip_vand_si(const vsip_vview_si *a, const vsip_vview_si *b, const
vsip_vview_si *r);
void vsip_vand_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const
vsip_vview_uc *r);
```

参数:

- a: 输入向量/矩阵视图。
- b: 输入向量/矩阵视图。
- r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.8.2 vsip_snot_p

按元素计算向量/矩阵的否定 “NOT” (1 的补码)。

接口:

```
void vsip_vnot_bl(const vsip_vview_bl *a, const vsip_vview_bl *r);
void vsip_vnot_i(const vsip_vview_i *a, const vsip_vview_i *r);
void vsip_vnot_si(const vsip_vview_si *a, const vsip_vview_si *r);
void vsip_vnot_uc(const vsip_vview_uc *a, const vsip_vview_uc *r);
```

参数:

- a: 输入向量/矩阵视图。
- r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.8.3 vsip_sor_p

按元素计算两个向量/矩阵的或 “OR” 。

接口:

```
void vsip_vor_bl(const vsip_vview_bl *a, const vsip_vview_bl *b, const
vsip_vview_bl *r);
```

```
void vsip_vor_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i *r);  
void vsip_vor_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_si *r);  
void vsip_vor_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_uc *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.8.4 vsip_sxor_p

按元素计算两个向量/矩阵的异或 “XOR” 。

接口:

```
void vsip_vxor_bl(const vsip_vview_bl *a, const vsip_vview_bl *b, const vsip_vview_bl *r);  
void vsip_vxor_i(const vsip_vview_i *a, const vsip_vview_i *b, const vsip_vview_i *r);  
void vsip_vxor_si(const vsip_vview_si *a, const vsip_vview_si *b, const vsip_vview_si *r);  
void vsip_vxor_uc(const vsip_vview_uc *a, const vsip_vview_uc *b, const vsip_vview_uc *r);
```

参数:

a: 输入向量/矩阵视图。

b: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.8.5 vsip_vswapbyte_u (新增 1 个)

对向量元素实现字节反转。

接口:

```
void vsip_vswapbyte_u(const vsip_vview_u *a, const vsip_vview_u *r); (新增接口)
```

参数:

a: 输入向量/矩阵视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.9 元素生成和复制函数（新增 21 个）

4.2.4.9.1 vsip_dscopy_p_p（新增 5 个）

将源向量/矩阵复制到目标向量/矩阵。

接口:

```
void vsip_mcopy_i_f(const vsip_mview_i *x, const vsip_mview_f *y);
void vsip_mcopy_si_f(const vsip_mview_si *x, const vsip_mview_f *y);
void vsip_vcopy_d_d(const vsip_vview_d *x, const vsip_vview_d *y);
void vsip_vcopy_f_f(const vsip_vview_f *x, const vsip_vview_f *y);
void vsip_vcopy_i_d(const vsip_vview_i *x, const vsip_vview_d *y);
void vsip_vcopy_bl_bl(const vsip_vview_bl *x, const vsip_vview_bl *y);
void vsip_vcopy_bl_d(const vsip_vview_bl *x, const vsip_vview_d *y);
void vsip_vcopy_bl_f(const vsip_vview_bl *x, const vsip_vview_f *y);
void vsip_vcopy_d_i(const vsip_vview_d *x, const vsip_vview_i *y);
void vsip_vcopy_d_vi(const vsip_vview_d *x, const vsip_vview_vi *y);
void vsip_cvcopy_d_f(const vsip_cvview_d *x, const vsip_cvview_f *y);
void vsip_cvcopy_f_d(const vsip_cvview_f *x, const vsip_cvview_d *y);
void vsip_cvcopy_d_d(const vsip_cvview_d *x, const vsip_cvview_d *y);
void vsip_cvcopy_f_f(const vsip_cvview_f *x, const vsip_cvview_f *y);
void vsip_vcopy_i_uc(const vsip_vview_i *x, const vsip_vview_uc *y);
void vsip_vcopy_i_vi(const vsip_vview_i *x, const vsip_vview_vi *y);
void vsip_vcopy_i_i(const vsip_vview_i *x, const vsip_vview_i *y);
void vsip_vcopy_i_f(const vsip_vview_i *x, const vsip_vview_f *y);
void vsip_vcopy_f_i(const vsip_vview_f *x, const vsip_vview_i *y);
void vsip_vcopy_f_vi(const vsip_vview_f *x, const vsip_vview_vi *y);
void vsip_vcopy_d_bl(const vsip_vview_d *x, const vsip_vview_bl *y);
void vsip_vcopy_f_bl(const vsip_vview_f *x, const vsip_vview_bl *y);
void vsip_vcopy_si_si(const vsip_vview_si *x, const vsip_vview_si *y);
void vsip_vcopy_uc_uc(const vsip_vview_uc *x, const vsip_vview_uc *y); (新增接口)
void vsip_vcopy_d_f(const vsip_vview_d *x, const vsip_vview_f *y);
```



```
void vsip_vcopy_f_d( const vsip_vview_f *x, const vsip_vview_d *y);
void vsip_vcopy_vi_i( const vsip_vview_vi *x, const vsip_vview_i *y);
void vsip_vcopy_vi_vi( const vsip_vview_vi *x, const vsip_vview_vi *y);
void vsip_vcopy_mi_mi( const vsip_vview_mi *x, const vsip_vview_mi *y);
void vsip_vcopy_d_si( const vsip_vview_d *x, const vsip_vview_si *y);
void vsip_vcopy_d_uc( const vsip_vview_d *x, const vsip_vview_uc *y);
void vsip_vcopy_si_d( const vsip_vview_si *x, const vsip_vview_d *y);
void vsip_vcopy_si_f( const vsip_vview_si *x, const vsip_vview_f *y);
void vsip_vcopy_f_si( const vsip_vview_f *x, const vsip_vview_si *y);
void vsip_vcopy_f_uc( const vsip_vview_f *x, const vsip_vview_uc *y);
void vsip_mcopy_d_d(const vsip_mview_d *x,const vsip_mview_d *y);
void vsip_mcopy_d_d_con(const vsip_mview_d *x,const vsip_mview_d *y); ( 新增接口 )
void vsip_mcopy_f_f(const vsip_mview_f *x,const vsip_mview_f *y);
void vsip_mcopy_f_f_con(const vsip_mview_f *x,const vsip_mview_f *y); ( 新增接口 )
void vsip_mcopy_f_d(const vsip_mview_f *x,const vsip_mview_d *y);
void vsip_mcopy_d_f(const vsip_mview_d *x,const vsip_mview_f *y);
void vsip_mcopy_i_d( const vsip_mview_i *x, const vsip_mview_d *y); ( 新增接口 )
void vsip_mcopy_si_d( const vsip_mview_si *x, const vsip_mview_d *y); ( 新增接口 )
void vsip_cmcopy_f_d(const vsip_cmview_f *x,const vsip_cmview_d *y);
void vsip_cmcopy_d_f(const vsip_cmview_d *x,const vsip_cmview_f *y);
void vsip_cmcopy_d_d(const vsip_cmview_d *x,const vsip_cmview_d *y);
void vsip_cmcopy_f_f(const vsip_cmview_f *x,const vsip_cmview_f *y);
void vsip_mcopy_bl_f(const vsip_mview_bl *x,const vsip_mview_f *y);
void vsip_mcopy_bl_d(const vsip_mview_bl *x,const vsip_mview_d *y);
void vsip_mcopy_f_bl(const vsip_mview_f *x,const vsip_mview_bl *y);
void vsip_mcopy_f_i(const vsip_mview_f *x,const vsip_mview_i *y);
void vsip_mcopy_f_uc(const vsip_mview_f *x,const vsip_mview_uc *y);
void vsip_mcopy_d_bl(const vsip_mview_d *x,const vsip_mview_bl *y);
void vsip_mcopy_d_i(const vsip_mview_d *x,const vsip_mview_i *y);
void vsip_mcopy_d_uc(const vsip_mview_d *x,const vsip_mview_uc *y);
void vsip_mcopy_bl_bl(const vsip_mview_bl *x,const vsip_mview_bl *y);
```

参数:

x: 源向量/矩阵视图对象。

y: 目标向量/矩阵视图对象。

返回值:

空。

4.2.4.9.2 vsip_dscopy_p (新增 6 个)

两个向量/矩阵间的元素片段复制。

接口：

```
void vsip_vcopy_f(const vsip_vview_f *x,const vsip_vview_f *b,vsip_index ax,vsip_index bx, vsip_length m); (新增接口)
```

```
void vsip_vcopy_d(const vsip_vview_d *x,const vsip_vview_d *b,vsip_index ax,vsip_index bx, vsip_length m); (新增接口)
```

```
void vsip_vcopy_i(const vsip_vview_i *x,const vsip_vview_i *b,vsip_index ax,vsip_index bx,vsip_length m); (新增接口)
```

```
void vsip_cvcopy_f(const vsip_cvview_f *x,const vsip_cvview_f *b,vsip_index ax,vsip_index bx,vsip_length m); (新增接口)
```

```
void vsip_cvcopy_d(const vsip_cvview_d *x,const vsip_cvview_d *b,vsip_index ax,vsip_index bx,vsip_length m); (新增接口)
```

```
void vsip_cmcopy_f(const vsip_cmview_f *A,const vsip_cmview_f *B,vsip_length r_index, vsip_length c_index); (新增接口)
```

参数：

x：目标向量视图对象。

b：源向量视图对象。

ax：目标起始索引号。

bx：源复制起始索引号

m：复制向量长度即元素数。

A：源矩阵视图对象。

B：目标矩阵视图对象。

r_index：源矩阵起始复制行号。

c_index：源矩阵起始复制列号。

注：

向量接口 m 不能大于 (x 长度-ax)且不能大于 (b 长度-bx)。

矩阵接口 A 的行数不能小于 B 的行数、A 的列数不能小于 B 的列数、(A 的行数-r_index) 不能小于 B 的行数、且 (A 的列数-c_index) 不能小于 B 的列数。

返回值：

空。

4.2.4.9.3 vsip_dscopyto_user_p

将 VSIPL-FT 视图引用的数据复制到用户分配的内存中。

接口：

```
void vsip_vcopyto_user_f(const vsip_vview_f *a, vsip_scalar_f* const R);
void vsip_cvcopyto_user_f(const vsip_cvview_f *a, vsip_scalar_f* const R,
vsip_scalar_f* const I);
void vsip_mcopyto_user_f(const vsip_mview_f *a, vsip_major major,
vsip_scalar_f* const R);
void vsip_cmcopyto_user_f(const vsip_cmview_f *a, vsip_major major,
vsip_scalar_f* const R, vsip_scalar_f* const I);
void vsip_vcopyto_user_d(const vsip_vview_d *a, vsip_scalar_d* const R);
void vsip_cvcopyto_user_d(const vsip_cvview_d *a, vsip_scalar_d* const R,
vsip_scalar_d* const I);
void vsip_mcopyto_user_d(const vsip_mview_d *a, vsip_major major,
vsip_scalar_d* const R);
void vsip_cmcopyto_user_d(const vsip_cmview_d *a, vsip_major major,
vsip_scalar_d* const R, vsip_scalar_d* const I);
```

参数：

a：输入向量/矩阵视图。

major：矩阵副本的复制方向，可以按行复制，也可以按列复制。

R：指向输出内存开头的指针即复数的实部。

I：指向输出复数的虚部或 NULL 的输出的指针。

返回值：

空。

4.2.4.9.4 vsip_dscopyfrom_user_p

将数据从用户分配的内存复制到 VSIPL-FT 视图。

接口：

```
void vsip_vcopyfrom_user_f(vsip_scalar_f* const R, const vsip_vview_f *a);
void vsip_cvcopyfrom_user_f(vsip_scalar_f* const R, vsip_scalar_f* const I,
const vsip_cvview_f *a);
void vsip_mcopyfrom_user_f(vsip_scalar_f* const R, vsip_major major, const
vsip_mview_f *a);
void vsip_cmcopyfrom_user_f(vsip_scalar_f* const R, vsip_scalar_f* const I,
vsip_major major, const vsip_cmview_f *a);
```

```
void vsip_vcopyfrom_user_d(vsip_scalar_d* const R, const vsip_vview_d *a);
void vsip_cvcopyfrom_user_d( vsip_scalar_d* const R, vsip_scalar_d* const I,
const vsip_cvview_d *a);
void vsip_mcopyfrom_user_d(vsip_scalar_d* const R, vsip_major major, const
vsip_mview_d *a);
void vsip_cmcopyfrom_user_d(vsip_scalar_d* const R, vsip_scalar_d* const I,
vsip_major major, const vsip_cmview_d *a);
```

参数:

a: 输出向量/矩阵视图。

major: 矩阵副本的复制方向, 可以按行复制, 也可以按列复制。

R: 指向输入内存开头的指针即复数的实部。

I: 指向输入复数的虚部或 NULL 的输出的指针。

返回值:

空。

4.2.4.9.5 vsip_dsfill_p (新增 2 个)

用常量值填充向量/矩阵。

接口:

```
void vsip_mfill_i( vsip_scalar_i alpha, const vsip_mview_i *r);
void vsip_mfill_si( vsip_scalar_si alpha, const vsip_mview_si *r);
void vsip_vfill_i(vsip_scalar_i alpha, const vsip_vview_i *r);
void vsip_vfill_si(vsip_scalar_si alpha, const vsip_vview_si *r);
void vsip_vfill_uc(vsip_scalar_uc alpha, const vsip_vview_uc *r);
void vsip_vfill_d(vsip_scalar_d alpha, const vsip_vview_d *r);
void vsip_vfill_f(vsip_scalar_f alpha, const vsip_vview_f *r);
void vsip_cvfill_d(vsip_cscalar_d alpha, const vsip_cvview_d *r);
void vsip_cvfill_f(vsip_cscalar_f alpha, const vsip_cvview_f *r);
void vsip_mfill_f(vsip_scalar_f alpha, const vsip_mview_f *r);
void vsip_mfill_f_con(vsip_scalar_f alpha, const vsip_mview_f *r); ( 新增接口 )
void vsip_cmfill_f(vsip_cscalar_f alpha, const vsip_cmview_f *r);
void vsip_cmfill_d(vsip_cscalar_d alpha, const vsip_cmview_d *r);
void vsip_mfill_d(vsip_scalar_d alpha, const vsip_mview_d *r);
void vsip_mfill_d_con(vsip_scalar_d alpha, const vsip_mview_d *r); ( 新增接口 )
```

参数:

alpha: 标量填充值。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.9.6 vsip_cmfills_p (新增 2 个)

将 $P \times Q$ 的矩阵填充为 $M \times N$ 矩阵($M > P, N > Q$),填充后的其他位置元素为输入 alpha。

接口:

```
void vsip_cmfills_f(const vsip_cmview_f *a, const vsip_cmview_f *x, vsip_cscalar_f alpha); (新增接口)
```

```
void vsip_cmfills_d(const vsip_cmview_d *a, const vsip_cmview_d *x, vsip_cscalar_d alpha); (新增接口)
```

参数:

alpha: 标量填充值。

a: 输入 $P \times Q$ 原始矩阵视图。

x: 输出 $M \times N$ 扩充矩阵视图。

注:

扩充矩阵的行和列分别对应不能小于原始矩阵的行和列。

返回值:

空。

4.2.4.9.7 vsip_vunit_p_con (新增 2 个)

实现向量初始化为单位向量。

接口:

```
void vsip_vunit_f_con(const vsip_vview_f* a, const vsip_vview_f* u); (新增接口)
```

```
void vsip_vunit_d_con(const vsip_vview_d* a, const vsip_vview_d* u); (新增接口)
```

参数:

a: 输入向量视图。

u: 输出单位向量视图。

返回值:

空。

4.2.4.9.8 vsip_midentity3x3_p (新增 2 个)

实现 3×3 矩阵初始化为单位矩阵接口。

接口:

```
void vsip_midentity3x3_f(const vsip_mview_f *a); (新增接口)
void vsip_midentity3x3_d(const vsip_mview_d *a); (新增接口)
```

参数:

a: 输入 3x3 矩阵/输出 3x3 单位矩阵视图。

返回值:

空。

4.2.4.9.9 vsip_vramp_p (新增 2 个)

通过从初始值开始并按步长大小递增斜率来计算得到向量每个元素:

$$r_k \leftarrow \alpha + k \cdot \beta, \quad j=0,1,\dots,N-1。$$

接口:

```
void vsip_vramp_i(vsip_scalar_i alpha, vsip_scalar_i beta, const vsip_vview_i *r);
void vsip_vramp_vi(vsip_scalar_vi alpha, vsip_scalar_vi beta, const vsip_vview_vi *r);
void vsip_vramp_si(vsip_scalar_si alpha, vsip_scalar_si d beta, const vsip_vview_si *r);
void vsip_vramp_uc(vsip_scalar_uc alpha, vsip_scalar_uc d beta, const vsip_vview_uc *r);
void vsip_vramp_d(vsip_scalar_d alpha, vsip_scalar_d beta, const vsip_vview_d *r);
void vsip_vramp_f(vsip_scalar_f alpha, vsip_scalar_f d beta, const vsip_vview_f *r);
void vsip_cvrampr_d(vsip_cscalar_d alpha, vsip_cscalar_d beta, const vsip_cvview_d *r); (新增接口)
void vsip_cvrampr_f(vsip_cscalar_f alpha, vsip_cscalar_f beta, const vsip_cvview_f *r); (新增接口)
```

参数:

alpha: 矢量斜率的初始值。

beta: 斜率增量 (如果为负, 则递减)。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.10 操纵函数（新增 13 个）

4.2.4.10.1 vsip_scmplx_p（新增 2 个）

从两个实数向量/矩阵形成复数向量/矩阵。

接口：

```
void vsip_vcmplx_d(const vsip_vview_d *a, const vsip_vview_d *b, const vsip_cvview_d *r);
```

```
void vsip_mcmplx_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_cmview_d *r); (新增接口)
```

```
void vsip_vcmplx_f(const vsip_vview_f *a, const vsip_vview_f *b, const vsip_cvview_f *r);
```

```
void vsip_mcmplx_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_cmview_f *r); (新增接口)
```

参数：

a：包含实部的输入向量/矩阵的视图。

b：包含虚部的输入向量/矩阵的视图。

r：输出向量/矩阵视图。

返回值：

空。

4.2.4.10.2 vsip_cvconcatenate_f（新增 1 个）

实现两个向量拼接成新的向量。

接口：

```
void vsip_cvconcatenate_f(const vsip_cvview_f* a, const vsip_cvview_f* b, const vsip_cvview_f* r); (新增接口)
```

参数：

a：输入向量视图。

b：输入向量视图。

r：输出向量视图。

注：

r 的长度必须等于 a 与 b 的长度之和。

返回值：

空。

4.2.4.10.3 vsip_dsgather_p

聚集操作使用由索引向量提供的索引来选择源向量/矩阵的元素，所选择的元素被顺序地放置在输出向量中，使得输出向量和索引向量被索引为相同的。

接口：

```
void vsip_vgather_i(const vsip_vview_i *x, const vsip_vview_vi *index, const vsip_vview_i *y);  
void vsip_vgather_si(const vsip_vview_si *x, const vsip_vview_vi *index, const vsip_vview_si *y);  
void vsip_vgather_uc(const vsip_vview_uc *x, const vsip_vview_vi *index, const vsip_vview_uc *y);  
void vsip_vgather_d(const vsip_vview_d *x, const vsip_vview_vi *index, const vsip_vview_d *y);  
void vsip_vgather_f(const vsip_vview_f *x, const vsip_vview_vi *index, const vsip_vview_f *y);  
void vsip_cvgather_d(const vsip_cvview_d *x, const vsip_vview_vi *index, const vsip_cvview_d *y);  
void vsip_cvgather_f(const vsip_cvview_f *x, const vsip_vview_vi *index, const vsip_cvview_f *y);  
void vsip_mgather_f(const vsip_mview_f *x, const vsip_vview_mi *index, const vsip_vview_f *y);  
void vsip_cmgather_f(const vsip_cmview_f *x, const vsip_vview_mi *index, const vsip_cvview_f *y);  
void vsip_mgather_d(const vsip_mview_d *x, const vsip_vview_mi *index, const vsip_vview_d *y);  
void vsip_cmgather_d(const vsip_cmview_d *x, const vsip_vview_mi *index, const vsip_cvview_d *y);
```

参数：

x：输入源向量/矩阵的视图。

index：输入向量/矩阵索引向量的视图。

y：输出目标向量/矩阵的视图。

返回值：

空。

4.2.4.10.4 vsip_simag_p (新增 2 个)

提取复数向量/矩阵的虚部。

接口:

```
void vsip_vimag_d(const vsip_cvview_d *a, const vsip_vview_d *r);  
void vsip_mimag_d(const vsip_cmview_d *a, const vsip_mview_d *r); (新增接口)  
void vsip_vimag_f(const vsip_cvview_f *a, const vsip_vview_f *r);  
void vsip_mimag_f(const vsip_cmview_f *a, const vsip_mview_f *r); (新增接口)
```

参数:

a: 复数输入向量/矩阵的视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.10.5 vsip_spolar_p

将复数向量/矩阵从矩形转换为极坐标形式。极坐标数据由包含半径的实向量/矩阵和包含复数输入数据的自变量(角度)的对应实向量/矩阵组成。

接口:

```
void vsip_vpolar_d(const vsip_cvview_d *a, const vsip_vview_d *r, const  
vsip_vview_d *phi);  
void vsip_vpolar_f(const vsip_cvview_f *a, const vsip_vview_f *r, const  
vsip_vview_f *phi);
```

参数:

a: 输入矩形形式向量/矩阵的视图。

r: 输出半径(幅度)矢量/矩阵视图。

phi: 输出角度向量/矩阵

返回值:

空。

4.2.4.10.6 vsip_sreal_p (新增 2 个)

提取复数向量/矩阵的实部。

接口:

```
void vsip_vreal_d(const vsip_cvview_d *a, const vsip_vview_d *r);  
void vsip_mreal_d(const vsip_cmview_d *a, const vsip_mview_d *r); (新增接口)  
void vsip_vreal_f(const vsip_cvview_f *a, const vsip_vview_f *r);  
void vsip_mreal_f(const vsip_cmview_f *a, const vsip_mview_f *r); (新增接口)
```

参数:

a: 复数输入向量/矩阵的视图。

r: 输出向量/矩阵视图。

返回值:

空。

4.2.4.10.7 vsip_srect_p (新增 2 个)

将一对实数向量/矩阵从复数的极坐标转换为复数的矩形形式。

接口:

```
void vsip_vrect_d(const vsip_vview_d *r, const vsip_vview_d *phi, const vsip_cvview_d *y);
```

```
void vsip_mrect_d(const vsip_mview_d *r, const vsip_mview_d *phi, const vsip_cmview_d *y); (新增接口)
```

```
void vsip_vrect_f(const vsip_vview_f *r, const vsip_vview_f *phi, const vsip_cvview_f *y);
```

```
void vsip_mrect_f(const vsip_mview_f *r, const vsip_mview_f *phi, const vsip_cmview_f *y); (新增接口)
```

参数:

r: 输入半径 (幅度) 矢量/矩阵视图。

phi: 输入半径 (幅度) 矢量/矩阵视图。

y: 输出矩形形式向量/矩阵的视图。

返回值:

空。

4.2.4.10.8 vsip_dsscatter_p

分散操作顺序地利用源向量和索引向量的元素。向量索引的元素(矩阵索引)用于选择输出向量/矩阵中的存储位置以存储来自源向量的元素。

接口:

```
void vsip_vscatter_i(const vsip_vview_i *x, const vsip_vview_i *y, const vsip_vview_vi *index);
```

```
void vsip_vscatter_si(const vsip_vview_si *x, const vsip_vview_si *y, const vsip_vview_vi *index);
```

```
void vsip_vscatter_uc(const vsip_vview_uc *x, const vsip_vview_uc *y, const vsip_vview_vi *index);
```

```
void vsip_vscatter_d(const vsip_vview_d *x, const vsip_vview_d *y, const vsip_vview_vi *index);
```

```
void vsip_vscatter_f(const vsip_vview_f *x, const vsip_vview_f *y, const
```

```
vsip_vview_vi *index);
void vsip_cvscatter_d(const vsip_cvview_d *x, const vsip_cvview_d *y, const
vsip_vview_vi *index);
void vsip_cvscatter_f(const vsip_cvview_f *x, const vsip_cvview_f *y, const
vsip_vview_vi *index);
void vsip_mscatter_f(const vsip_vview_f *x, const vsip_mview_f *y, const
vsip_vview_mi *index);
void vsip_cmscatter_f(const vsip_cvview_f *x, const vsip_cmview_f *y, const
vsip_vview_mi *index);
void vsip_mscatter_d(const vsip_vview_d *x, const vsip_mview_d *y, const
vsip_vview_mi *index);
void vsip_cmscatter_d(const vsip_cvview_d *x, const vsip_cmview_d *y,
const vsip_vview_mi *index);
```

参数:

x: 输入源向量/矩阵的视图。

y: 输出目标向量/矩阵的视图。

index: 输入向量/矩阵索引向量的视图。

返回值:

空。

4.2.4.10.9 vsip_dsswap_p (新增1个)

在两个向量/矩阵之间交换元素。

接口:

```
void vsip_vswap_i(const vsip_vview_i *a, const vsip_vview_i *b);
void vsip_vswap_si(const vsip_vview_si *a, const vsip_vview_si *b);
void vsip_vswap_uc(const vsip_vview_uc *a, const vsip_vview_uc *b);
void vsip_vswap_d(const vsip_vview_d *a, const vsip_vview_d *b);
void vsip_vswap_f(const vsip_vview_f *a, const vsip_vview_f *b);
void vsip_cvswap_d(const vsip_cvview_d *a, const vsip_cvview_d *b);
void vsip_cvswap_f(const vsip_cvview_f *a, const vsip_cvview_f *b);
void vsip_mswap_i(const vsip_mview_i *a, const vsip_mview_i *b); (新增接口)
void vsip_mswap_d(const vsip_mview_d *a, const vsip_mview_d *b);
void vsip_mswap_f(const vsip_mview_f *a, const vsip_mview_f *b);
void vsip_cmswap_f(const vsip_cmview_f *a, const vsip_cmview_f *b);
void vsip_cmswap_d(const vsip_cmview_d *a, const vsip_cmview_d *b);
```

参数:

a: 输入向量/矩阵视图。

b: 输出向量/矩阵视图。

返回值:

空。

4.2.4.10.10 vsip_vsortip_p (新增 3 个)

对向量元素进行排序。

接口:

```
void vsip_vsortip_i(const vsip_vview_i *a,vsip_sort_mode mode,vsip_sort_dir dir,vsip_bool fill, const vsip_vview_vi *index); (新增接口)
```

```
void vsip_vsortip_d(const vsip_vview_d *a,vsip_sort_mode mode,vsip_sort_dir dir,vsip_bool fill,const vsip_vview_vi *index); (新增接口)
```

```
void vsip_vsortip_f(const vsip_vview_f *a,vsip_sort_mode mode,vsip_sort_dir dir,vsip_bool fill,const vsip_vview_vi *index); (新增接口)
```

参数:

a: 输入/输出数据的向量视图。

mode: mode=1 绝对值排序或 mode=0 数值排序。

dir: 升序 “ascending=0” or 降序“descending=1”。

fill: 指示函数是否应初始化索引向量(VSIP_TRUE); 或假定索引向量已初始化(VSIP_FALSE)。

index: 适当类型的索引向量, 用于存储排序值的原始索引, 如果无索引向量, 则为 NULL。非空索引向量索引长度的长度必须与输入视图中值的总数匹配。

返回值:

空。

4.2.5 VSIPL-FT 线性代数函数（新增 63 个）

4.2.5.1 介绍

本节对矩阵函数进行说明。以下函数对矩阵进行运算或生成矩阵结果。

4.2.5.2 矩阵和矢量运算函数（新增 30 个）

4.2.5.2.1 vsip_cmherm_p

矩阵的复埃尔米特(共轭转置)

接口:

```
void vsip_cmherm_d(const vsip_cmview_d *a, const vsip_cmview_d *r);  
void vsip_cmherm_f(const vsip_cmview_f *a, const vsip_cmview_f *r);
```

参数:

a: 输入 MxN 矩阵的视图。

r: 输出 NxM 矩阵的视图。

返回值:

空。

4.2.5.2.2 vsip_cvjdot_p

计算两个复数向量的共轭内(点)积。

接口:

```
vsip_cscalar_d vsip_cvjdot_d(const vsip_cvview_d *a, const vsip_cvview_d *r);  
vsip_cscalar_f vsip_cvjdot_f(const vsip_cvview_f *a, const vsip_cvview_f *r);
```

参数:

a: 输入向量视图。

r: 输出向量视图。

返回值:

空。

4.2.5.2.3 vsip_dgemp_p

计算两个矩阵的一般乘积并累加。

接口:

```
void vsip_cgemp_d(vsip_cscalar_d alpha, const vsip_cmview_d *A,
```

```

vsip_mat_op OpA, const vsip_cmview_d *B, vsip_mat_op OpB, vsip_cscalar_d beta,
const vsip_cmview_d *C);
    void vsip_cgemp_f(vsip_cscalar_f alpha, const vsip_cmview_f *A, vsip_mat_op
OpA, const vsip_cmview_f *B, vsip_mat_op OpB, vsip_cscalar_f beta, const
vsip_cmview_f *C);
    void vsip_gemp_d(vsip_scalar_d alpha, const vsip_mview_d *A, vsip_mat_op
OpA, const vsip_mview_d *B, vsip_mat_op OpB, vsip_scalar_d beta, const
vsip_mview_d *C);
    void vsip_gemp_f(vsip_scalar_f alpha, const vsip_mview_f *A, vsip_mat_op OpA,
const vsip_mview_f *B, vsip_mat_op OpB, vsip_scalar_f beta, const vsip_mview_f
*C);

```

参数:

alpha: (实数/复数)标量。

A: 输入矩阵 A 的视图。

OpA: 指定 op (A) 的形式。

B: 输入矩阵 B 的视图。

OpB: 指定 op (B) 的形式。

beta: (实数/复数)标量。

C: 输出 MxN 矩阵。

返回值:

空。

4.2.5.2.4 vsip_dgems_p

计算一般矩阵和。

接口:

```

void vsip_cgems_d(vsip_cscalar_d alpha, const vsip_cmview_d *A,
vsip_mat_op OpA, vsip_cscalar_d beta, const vsip_cmview_d *C);
    void vsip_cgems_f(vsip_cscalar_f alpha, const vsip_cmview_f *A, vsip_mat_op
OpA, vsip_cscalar_f beta, const vsip_cmview_f *C);
    void vsip_gems_d(vsip_scalar_d alpha, const vsip_mview_d *A, vsip_mat_op
OpA, vsip_scalar_d beta, const vsip_mview_d *C);
    void vsip_gems_f(vsip_scalar_f alpha, const vsip_mview_f *A, vsip_mat_op OpA,
vsip_scalar_f beta, const vsip_mview_f *C);

```

参数:

alpha: (实数/复数)标量。

A: 输入矩阵 A 的视图。

OpA: 指定 $op(A)$ 的形式。

beta: (实数/复数)标量。

C: 输出 $M \times N$ 矩阵。

返回值:

空。

4.2.5.2.5 vsip_dskron_p

计算两个向量或矩阵的克罗内克张量积。

接口:

```
void vsip_cmkrn_d(vsip_cscalar_d alpha,const vsip_cmview_d *x, const
vsip_cmview_d *y, const vsip_cmview_d *C);
void vsip_cmkrn_f(vsip_cscalar_f alpha,const vsip_cmview_f *x, const
vsip_cmview_f *y, const vsip_cmview_f *C);
void vsip_cvkrn_d(vsip_cscalar_d alpha,const vsip_cvview_d *x, const
vsip_cvview_d *y, const vsip_cmview_d *C);
void vsip_cvkrn_f(vsip_cscalar_f alpha,const vsip_cvview_f *x, const
vsip_cvview_f *y, const vsip_cmview_f *C);
void vsip_mkron_d(vsip_scalar_d alpha,const vsip_mview_d *x, const
vsip_mview_d *y, const vsip_mview_d *C);
void vsip_mkron_f(vsip_scalar_f alpha,const vsip_mview_f *x, const
vsip_mview_f *y, const vsip_mview_f *C);
void vsip_vkron_d(vsip_scalar_d alpha,const vsip_vview_d *x, const
vsip_vview_d *y, const vsip_mview_d *C);
void vsip_vkron_f(vsip_scalar_f alpha,const vsip_vview_f *x, const vsip_vview_f
*y, const vsip_mview_f *C);
```

参数:

alpha: (实数/复数)标量。

x: 输入向量/矩阵视图。

y: 输入向量/矩阵视图。

C: 输出矩阵视图。

返回值:

空。

4.2.5.2.6 vsip_dmprod3_p

计算 3×3 矩阵和 $3 \times N$ 矩阵的乘积。

接口:

```
void vsip_cmprod3_d(const vsip_cmview_d *A, const vsip_cmview_d *B,
const vsip_cmview_d *C);
void vsip_cmprod3_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const
vsip_cmview_f *C);
void vsip_mprod3_d(const vsip_mview_d *A, const vsip_mview_d *B, const
vsip_mview_d *C);
void vsip_mprod3_f(const vsip_mview_f *A, const vsip_mview_f *B, const
vsip_mview_f *C);
```

参数:

- A: 输入 3x3 矩阵视图。
- B: 输入 3xN 矩阵视图。
- C: 输出 3xN 矩阵视图。

返回值:

空。

4.2.5.2.7 vsip_dmprod4_p

计算 4 乘 4 矩阵和 4 乘 N 矩阵的乘积。

接口:

```
void vsip_cmprod4_d(const vsip_cmview_d *A, const vsip_cmview_d *B,
const vsip_cmview_d *C);
void vsip_cmprod4_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const
vsip_cmview_f *C);
void vsip_mprod4_d(const vsip_mview_d *A, const vsip_mview_d *B, const
vsip_mview_d *C);
void vsip_mprod4_f(const vsip_mview_f *A, const vsip_mview_f *B, const
vsip_mview_f *C);
```

参数:

- A: 输入 4x4 矩阵视图。
- B: 输入 4xN 矩阵视图。
- C: 输出 4xN 矩阵视图。

返回值:

空。

4.2.5.2.8 vsip_dmprod_p

计算两个矩阵的乘积。

接口：

```
void vsip_cmprod_d(const vsip_cmview_d *A, const vsip_cmview_d *B, const vsip_cmview_d *C);
```

```
void vsip_cmprod_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const vsip_cmview_f *C);
```

```
void vsip_mprod_d(const vsip_mview_d *A, const vsip_mview_d *B, const vsip_mview_d *C);
```

```
void vsip_mprod_f(const vsip_mview_f *A, const vsip_mview_f *B, const vsip_mview_f *C);
```

参数：

A：输入 MxP 矩阵视图。

B：输入 PxN 矩阵视图。

C：输出 MxN 矩阵视图。

返回值：

空。

4.2.5.2.9 vsip_mprod33x31_p (新增 2 个)

计算 3x3 矩阵与 3x1 矩阵 prod 乘。

接口：

```
void vsip_mprod33x31_f(const vsip_mview_f *A, const vsip_mview_f *B, const vsip_mview_f *C); (新增接口)
```

```
void vsip_mprod33x31_d(const vsip_mview_d *A, const vsip_mview_d *B, const vsip_mview_d *C); (新增接口)
```

参数：

A：输入 3x3 矩阵。

B：输入 3x1 矩阵。

C：输出 3x3 矩阵与 3x1 矩阵 prod 乘。

返回值：

空。

4.2.5.2.10 vsip_modelXY3x1_p (新增 3 个)

求矩阵或矩阵 XY 方向上模值。

接口:

```
vsip_scalar_d vsip_mmodel3x1_d(const vsip_mview_d *a); (新增接口)  
vsip_scalar_f vsip_modelXY3x1_f(const vsip_mview_f *a); (新增接口)  
vsip_scalar_d vsip_modelXY3x1_d(const vsip_mview_d *a); (新增接口)
```

参数:

a: 输入矩阵。

返回值:

返回模值。

4.2.5.2.11 vsip_mTunion3x3_p (新增 2 个)

实现 x/y/z 方向上的 3x3 转化矩阵。

接口:

```
void vsip_mTunion3x3_f(vsip_scalar_f x,const vsip_mview_f *a , vsip_scalar_i  
dir); (新增接口)  
void vsip_mTunion3x3_d(vsip_scalar_d x,const vsip_mview_d *a , vsip_scalar_i  
dir); (新增接口)
```

参数:

x: 标量常数用于转化的值。

a: 输出转化后的矩阵。

dir: 指定 x/y/z 方向 (0、1 或其他值)。

返回值:

空。

4.2.5.2.12 vsip_mfillmix_p (新增 2 个)

实现三个向量交叉填充 3x3 矩阵。

接口:

```
void vsip_mfillmix_f(const vsip_mview_f *a,const vsip_mview_f *b,const  
vsip_mview_f *c,const vsip_mview_f *r); (新增接口)  
void vsip_mfillmix_d(const vsip_mview_d *a,const vsip_mview_d *b,const  
vsip_mview_d *c,const vsip_mview_d *r); (新增接口)
```

参数:

a、b: 输入矩阵。

r: 输出矩阵。

返回值:

空。

4.2.5.2.13 vsip_dmprodh_p

计算复数矩阵和复数矩阵的乘积。

接口：

```
void vsip_cmprodh_d(const vsip_cmview_d *A, const vsip_cmview_d *B,
const vsip_cmview_d *C);
```

```
void vsip_cmprodh_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const
vsip_cmview_f *C);
```

参数：

A：输入 MxP 矩阵视图。

B：输入 NxP 矩阵视图。

C：输出 MxN 矩阵视图。

返回值：

空。

4.2.5.2.14 vsip_dmprodj_p

计算矩阵和矩阵的共轭的乘积。

接口：

```
void vsip_cmprodj_d(const vsip_cmview_d *A, const vsip_cmview_d *B, const
vsip_cmview_d *C);
```

```
void vsip_cmprodj_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const
vsip_cmview_f *C);
```

参数：

A：输入 MxP 矩阵视图。

B：输入 NxP 矩阵视图。

C：输出 MxN 矩阵视图。

返回值：

空。

4.2.5.2.15 vsip_dmprodt_p

计算矩阵的乘积和矩阵的转置。

接口：

```
void vsip_cmprodt_d(const vsip_cmview_d *A, const vsip_cmview_d *B, const
vsip_cmview_d *C);
```

```
void vsip_cmprodt_f(const vsip_cmview_f *A, const vsip_cmview_f *B, const
```

```
vsip_cmview_f *C);  
void vsip_mprodt_d(const vsip_mview_d *A, const vsip_mview_d *B, const  
vsip_mview_d *C);  
void vsip_mprodt_f(const vsip_mview_f *A, const vsip_mview_f *B, const  
vsip_mview_f *C);
```

参数:

A: 输入 $M \times P$ 矩阵视图。

B: 输入 $N \times P$ 矩阵视图。

C: 输出 $M \times N$ 矩阵视图。

返回值:

空。

4.2.5.2.16 vsip_dmvprod3_p

计算一个 3×3 矩阵和一个向量的乘积。

接口:

```
void vsip_cmvprod3_d( const vsip_cmview_d *A, const vsip_cvview_d *x,  
const vsip_cvview_d *y);  
void vsip_cmvprod3_f( const vsip_cmview_f *A, const vsip_cvview_f *x, const  
vsip_cvview_f *y);  
void vsip_mvprod3_d( const vsip_mview_d *A, const vsip_vview_d *x, const  
vsip_vview_d *y);  
void vsip_mvprod3_f( const vsip_mview_f *A, const vsip_vview_f *x, const  
vsip_vview_f *y);
```

参数:

A: 输入 3×3 矩阵视图。

x: 输入长度为 3 的向量视图。

y: 输出长度为 3 的向量视图。

返回值:

空。

4.2.5.2.17 vsip_dmvprod4_p

计算一个 4×4 矩阵和一个向量的乘积。

接口:

```
void vsip_cmvprod4_d(const vsip_cmview_d *A, const vsip_cvview_d *x,  
const vsip_cvview_d *y);
```

```
void vsip_cmvprod4_f(const vsip_cmview_f *A, const vsip_cvview_f *x, const vsip_cvview_f *y);  
void vsip_mvprod4_d(const vsip_mview_d *A, const vsip_vview_d *x, const vsip_vview_d *y);  
void vsip_mvprod4_f(const vsip_mview_f *A, const vsip_vview_f *x, const vsip_vview_f *y);
```

参数:

A: 输入 4x4 矩阵视图。

x: 输入长度为 4 的向量视图。

y: 输出长度为 4 的向量视图。

返回值:

空。

4.2.5.2.18 vsip_dmvprod_p

计算矩阵向量乘积。

接口:

```
void vsip_cmvprod_d(const vsip_cmview_d *A, const vsip_cvview_d *x, const vsip_cvview_d *y);  
void vsip_cmvprod_f(const vsip_cmview_f *A, const vsip_cvview_f *x, const vsip_cvview_f *y);  
void vsip_mvprod_d(const vsip_mview_d *A, const vsip_vview_d *x, const vsip_vview_d *y);  
void vsip_mvprod_f(const vsip_mview_f *A, const vsip_vview_f *x, const vsip_vview_f *y);
```

参数:

A: 输入 MxN 矩阵视图。

x: 输入长度为 N 的向量视图。

y: 输出长度为 M 的向量视图。

返回值:

空。

4.2.5.2.19 vsip_dmtrans_p (新增 2 个)

转置一个矩阵。

接口:

```
void vsip_cmtrans_d(const vsip_cmview_d *A, const vsip_cmview_d *C);
```

```
void vsip_cmtrans_f(const vsip_cmview_f *A, const vsip_cmview_f *C);
void vsip_mtrans_d(const vsip_mview_d *A, const vsip_mview_d *C);
void vsip_mtrans_f(const vsip_mview_f *A, const vsip_mview_f *C);
void vsip_cmtrans2D_f(vsip_cscalar_f **in, vsip_cscalar_f **out, vsip_scalar_u
row, vsip_scalar_u col); (新增接口)
void vsip_mtrans2D_f(vsip_scalar_f **in, vsip_scalar_f **out, vsip_scalar_u row,
vsip_scalar_u col); (新增接口)
```

参数:

A: 输入 MxN 矩阵视图。

C: 输出 NxM 矩阵视图。

返回值:

空。

4.2.5.2.20 vsip_dminv_p (新增 4 个)

矩阵求逆。

接口:

```
void vsip_minv_f(const vsip_mview_f *a, const vsip_mview_f *r); (新增接口)
void vsip_minv_d(const vsip_mview_d *a, const vsip_mview_d *r); (新增接口)
void vsip_cminv_f(const vsip_cmview_f *a, const vsip_cmview_f *r); (新增接口)
void vsip_cminv_d(const vsip_cmview_d *a, const vsip_cmview_d *r); (新增接口)
```

参数:

a: 输入矩阵。

r: 输出求逆后的矩阵。

返回值:

空。

4.2.5.2.21 vsip_dvdot_p (新增 4 个)

计算两个向量的内部(点)乘积。

接口:

```
vsip_cscalar_d vsip_cvdot_d(const vsip_cvview_d *a, const vsip_cvview_d *b);
vsip_cscalar_f vsip_cvdot_f(const vsip_cvview_f *a, const vsip_cvview_f *b);
vsip_cscalar_d vsip_rcvdot_d(const vsip_vview_d *a, const vsip_cvview_d *b);
(新增接口)
vsip_cscalar_f vsip_rcvdot_f(const vsip_vview_f *a, const vsip_cvview_f *b);(新增
接口)
vsip_scalar_i vsip_vdot_c(const vsip_vview_c *a, const vsip_vview_c *b); (新增接
```

口)

```
vsip_scalar_d vsip_vdot_d(const vsip_vview_d *a, const vsip_vview_d *b);  
vsip_scalar_f vsip_vdot_f(const vsip_vview_f *a, const vsip_vview_f *b);  
vsip_scalar_d vsip_mdot3x1_d(const vsip_mview_d *a, const vsip_mview_d *b);
```

(新增接口)

参数:

a: 输入向量视图。

b: 输出向量视图。

返回值:

空。

4.2.5.2.22 vsip_cvcormtx_f (新增1个)

求矩阵自相关。

接口:

```
void vsip_cvcormtx_f(vsip_cvview_f* v, vsip_length m, vsip_cmview_f* r); (新增接口)
```

口)

参数:

v: 输入复数向量。

m: 向量长度。

r: 输出矩阵。

注:

$0 \leq m < v.length$ 或矩阵大小为 $(v.length + m) \times (m + 1)$ 。

返回值:

空。

4.2.5.2.23 vsip_CrossRroduct_p or vsip_CrossProduct_p (新增4个)

计算矩阵有无参数互相关。

接口:

```
void vsip_CrossRroduct_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_mview_f *r); (新增接口)
```

```
void vsip_CrossRroduct_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_mview_d *r); (新增接口)
```

```
void vsip_CrossProduct_f(const vsip_mview_f *a, const vsip_mview_f *b, const vsip_scalar_bl bl, const vsip_mview_f *r); (新增接口)
```

```
void vsip_CrossProduct_d(const vsip_mview_d *a, const vsip_mview_d *b, const vsip_scalar_d bl, const vsip_mview_d *r); (新增接口)
```

参数:

a、b: 输入矩阵。

bl: 互相关标识, 0 做互相关, 1 不做互相关。

r: 输出矩阵。

返回值:

空。

4.2.5.2.24 vsip_dvmprod_p

计算向量-矩阵乘积。

接口:

```
void vsip_cvmprod_d(const vsip_cvview_d *x, const vsip_cmview_d *A, const vsip_cvview_d *y);
```

```
void vsip_cvmprod_f(const vsip_cvview_f *x, const vsip_cmview_f *A, const vsip_cvview_f *y);
```

```
void vsip_vmprod_d(const vsip_vview_d *x, const vsip_mview_d *A, const vsip_vview_d *y);
```

```
void vsip_vmprod_f(const vsip_vview_f *x, const vsip_mview_f *A, const vsip_vview_f *y);
```

参数:

x: 输入长度为 M 的向量视图。

A: 输入 MxN 的矩阵视图。

y: 输出长度为 N 的向量视图。

返回值:

空。

4.2.5.2.25 vsip_dvrouter_p

计算两个向量的外积。

接口:

```
void vsip_vrouter_d(vsip_scalar_d alpha, const vsip_vview_d *x, const vsip_vview_d *y, const vsip_mview_d *C);
```

```
void vsip_vrouter_f(vsip_scalar_f alpha, const vsip_vview_f *x, const vsip_vview_f *y, const vsip_mview_f *C);
```

```
void vsip_cvrouter_d(vsip_cscalar_d alpha, const vsip_cvview_d *x, const
```



```
vsip_cvview_d *y, const vsip_cmview_d *C);  
void vsip_cvouter_f(vsip_cscalar_f alpha, const vsip_cvview_f *x, const  
vsip_cvview_f *y, const vsip_cmview_f *C);
```

参数:

alpha: (实数/复数) 标量。

x: 输入长度为 M 的向量视图。

y: 输入长度为 N 的向量视图。

C: 输出 MxN 的矩阵视图。

返回值:

空。

4.2.5.2.26 vsip_dmeigenR_p (新增 4 个)

矩阵求特征值、特征向量。

接口:

```
void vsip_meigenR_f(const vsip_mview_f *a, const vsip_cmview_f *eigenvalue,  
const vsip_cmview_f *vec_feature); (新增接口)
```

```
void vsip_meigenR_d(const vsip_mview_d *a, const vsip_cmview_d *eigenvalue,  
const vsip_cmview_d *vec_feature); (新增接口)
```

```
void vsip_cmeigenR_f(const vsip_cmview_f *a, const vsip_cmview_f *eigenvalue,  
const vsip_cmview_f *vec_feature); (新增接口)
```

```
void vsip_cmeigenR_d(const vsip_cmview_d *a, const vsip_cmview_d  
*eigenvalue, const vsip_cmview_d *vec_feature); (新增接口)
```

参数:

a: 输入矩阵。

eigenvalue: 输出特征值。

vec_feature: 输出特征向量。

返回值:

空。

4.2.5.2.27 vsip_mdistance_p (新增 2 个)

计算三维坐标系两点间的距离。

接口:

```
vsip_scalar_f vsip_mdistance_f(const vsip_mview_f *a, const vsip_mview_f *b);  
(新增接口)
```

```
vsip_scalar_d vsip_mdistance_d(const vsip_mview_d *a, const vsip_mview_d *b);
```

(新增接口)

参数:

a、b: 输入矩阵。

返回值:

返回三维坐标系两点间的距离。

4.2.5.3 特殊线性系统求解器

4.2.5.3.1 vsip_dcovsol_p

求解协方差线性系统问题。

接口:

```
int vsip_ccovsol_d(const vsip_cmview_d *A, const vsip_cmview_d *XB);
int vsip_ccovsol_f(const vsip_cmview_f *A, const vsip_cmview_f *XB);
int vsip_covsol_d(const vsip_mview_d *A, const vsip_mview_d *XB);
int vsip_covsol_f(const vsip_mview_f *A, const vsip_mview_f *XB);
```

参数:

A: 输入 $M \times N$ 矩阵视图, $M > N$ 。

XB: 输出 X /输入矩阵 B 的视图, 大小为 $N \times K$ 。

返回值:

- 如果成功, 则为 0;
- 如果内存分配失败则为-1;
- 如果 A 没有满秩, 则为正, $\text{rank}(A)=N$ 。

4.2.5.3.2 vsip_dllsqsol_p

求解线性最小二乘问题。

接口:

```
int vsip_cllsqsol_d(const vsip_cmview_d *A, const vsip_cmview_d *XB);
int vsip_cllsqsol_f(const vsip_cmview_f *A, const vsip_cmview_f *XB);
int vsip_llsqsol_d(const vsip_mview_d *A, const vsip_mview_d *XB);
int vsip_llsqsol_f(const vsip_mview_f *A, const vsip_mview_f *XB);
```

参数:

A: 输入 $M \times N$ 矩阵视图, $M > N$ 。

XB: 输出 X /输入矩阵 B 的视图, 大小为 $N \times K$ 。

返回值:

- 如果成功, 则为 0;

- 如果内存分配失败则为-1;
- 如果 A 没有满秩, 则为正, $\text{rank}(A)=N$ 。

4.2.5.3.3 vsip_dtoepsol_p

求解实对称或复 Hermitian 正定 Toeplitz 线性系统。

接口:

```
int vsip_ctoepsol_d( const vsip_cvview_d *t, const vsip_cvview_d *b, const
vsip_cvview_d *w, const vsip_cvview_d *x);
int vsip_ctoepsol_f(const vsip_cvview_f *t, const vsip_cvview_f *b, const
vsip_cvview_f *w, const vsip_cvview_f *x);
int vsip_ttoepsol_d( const vsip_vview_d *t, const vsip_vview_d *b, const
vsip_vview_d *w, const vsip_vview_d *x);
int vsip_ttoepsol_f(const vsip_vview_f *t, const vsip_vview_f *b, const
vsip_vview_f *w, const vsip_vview_f *x);
```

参数:

- t: Toeplitz 矩阵 T 的第一行长度为 N 的输入向量 t 的视图。
- b: 长度为 N 的输入向量 b 的视图。
- w: 用于临时工作空间的长度为 N 的向量 w 的视图。
- x: 长度为 N 的输出向量 x 的视图。

返回值:

- 如果成功, 则为 0;
- 如果内存分配失败则为-1;
- 如果 T 不是正定, 则为正。

4.2.5.4 一般平方线性系统解算器

4.2.5.4.1 vsip_dlud_p

使用部分旋转计算方矩阵 a 的 LU 分解。

接口:

```
int vsip_lud_d(vsip_lu_d *lud, const vsip_mview_d *A);
int vsip_lud_f(vsip_lu_f *lud, const vsip_mview_f *A);
int vsip_clud_d(vsip_clu_d *lud, const vsip_cmview_d *A);
int vsip_clud_f(vsip_clu_f *lud, const vsip_cmview_f *A);
```

参数:

- lud: 指向由 vsip_dlud_create_f 创建的 LU 分解对象的指针。

A: 输入 $N \times N$ 矩阵视图。

返回值:

如果成功, 则返回零。如果 A 没有满秩, 则失败并返回非零。

4.2.5.4.2 vsip_dlud_create_p

创建一个 LU 分解对象。

接口:

```
vsip_lu_d *vsip_lud_create_d(vsip_length N);  
vsip_lu_f *vsip_lud_create_f(vsip_length N);  
vsip_clu_d *vsip_clud_create_d(vsip_length N);  
vsip_clu_f *vsip_clud_create_f(vsip_length N);
```

参数:

N: 矩阵 A 中的行数。

返回值:

返回值是指向 LU 分解对象的指针, 如果失败, 则返回 NULL。

4.2.5.4.3 vsip_dlud_destroy_p

销毁一个 LU 分解对象。

接口:

```
int vsip_lud_destroy_d(vsip_lu_d *lud);  
int vsip_lud_destroy_f(vsip_lu_f *lud);  
int vsip_clud_destroy_d(vsip_clu_d *lud);  
int vsip_clud_destroy_f(vsip_clu_f *lud);
```

参数:

lud: 指向由 vsip_dlud_create_f 创建的 LU 分解对象的指针。

返回值:

如果成功, 则返回零。

4.2.5.4.4 vsip_dlud_getattr_p

返回 LU 分解对象的属性。

接口:

```
void vsip_lud_getattr_d(const vsip_lu_d *lud, vsip_lu_attr_d *attr);  
void vsip_lud_getattr_f(const vsip_lu_f *lud, vsip_lu_attr_f *attr);  
void vsip_clud_getattr_d(const vsip_clu_d *lud, vsip_clu_attr_d *attr);  
void vsip_clud_getattr_f(const vsip_clu_f *lud, vsip_clu_attr_f *attr);
```

参数:

lud: 指向由 vsip_dlud_create_f 创建的 LU 分解对象的指针。

attr: 指向输出属性结构的指针。

返回值:

空。

4.2.5.4.5 vsip_dludsol_p

求解平方线性系统。

接口:

```
int vsip_lusol_d(const vsip_lu_d *lud, vsip_mat_op OpA, const vsip_mview_d *XB);
```

```
int vsip_lusol_f(const vsip_lu_f *lud, vsip_mat_op OpA, const vsip_mview_f *XB);
```

```
int vsip_clusol_d(const vsip_clu_d *lud, vsip_mat_op OpA, const vsip_cmview_d *XB);
```

```
int vsip_clusol_f(const vsip_clu_f *lud, vsip_mat_op OpA, const vsip_cmview_f *XB);
```

参数:

lud: 指向由 vsip_dlud_create_f 创建的 LU 分解对象的指针。

OpA: 指定 $op(A)$ 的形式。

XB: 输出 X/输入 B 的视图, 大小为 $N \times K$ 的矩阵。

返回值:

如果成功, 则返回零。

4.2.5.5 对称正定线性系统

4.2.5.5.1 vsip_dchold_p

计算对称(埃尔米特)正定矩阵的 Cholesky 分解。

接口:

```
int vsip_chold_d(vsip_chol_d *chold, const vsip_mview_d *A);
```

```
int vsip_chold_f(vsip_chol_f *chold, const vsip_mview_f *A);
```

```
int vsip_cchold_d(vsip_cchol_d *chold, const vsip_cmview_d *A);
```

```
int vsip_cchold_f(vsip_cchol_f *chold, const vsip_cmview_f *A);
```

参数:

chold: 指向由 vsip_dchold_create_f 创建的 Cholesky 分解对象的指针。

A: 输入 $N \times N$ 的矩阵。

返回值:

如果成功, 则返回零。

4.2.5.5.2 vsip_dchold_create_p

创建一个 Cholesky 分解对象。

接口:

```
vsip_chol_d *vsip_chold_create_d(const vsip_mat_uplo uplo, vsip_length N);  
vsip_chol_f *vsip_chold_create_f(const vsip_mat_uplo uplo, vsip_length N);  
vsip_cchol_d *vsip_cchold_create_d(const vsip_mat_uplo uplo, vsip_length N);  
vsip_cchol_f *vsip_cchold_create_f(const vsip_mat_uplo uplo, vsip_length N);
```

参数:

uplo: 指定是存储矩阵的上半边形还是下半边形。

N: 输入矩阵中的行数或列数。

返回值:

返回值是指向 Cholesky 分解对象的指针, 如果失败, 则返回 NULL。

4.2.5.5.3 vsip_dchold_destroy_p

销毁一个 Cholesky 分解对象。

接口:

```
int vsip_chold_destroy_d( vsip_chol_d *chold);  
int vsip_chold_destroy_f( vsip_chol_f *chold);  
int vsip_cchold_destroy_d( vsip_cchol_d *chold);  
int vsip_cchold_destroy_f( vsip_cchol_f *chold);
```

参数:

chold: 指向由 vsip_dchold_create_f 创建的 Cholesky 分解对象的指针。

返回值:

成功时返回零, 失败时返回非零。

4.2.5.5.4 vsip_dchold_getattr_p

返回 Cholesky 分解对象的属性。

接口:

```
void vsip_chold_getattr_d(const vsip_chol_d *chold, vsip_chol_attr_d *attr);  
void vsip_chold_getattr_f(const vsip_chol_f *chold, vsip_chol_attr_f *attr);  
void vsip_cchold_getattr_d( const vsip_cchol_d *chold, vsip_cchol_attr_d *attr);  
void vsip_cchold_getattr_f(const vsip_cchol_f *chold, vsip_cchol_attr_f *attr);
```

参数:

chold: 指向由 vsip_dchold_create_f 创建的 Cholesky 分解对象的指针。

attr: 指向输出属性结构的指针。

返回值:

空。

4.2.5.5.5 vsip_dcholsol_p

求解对称(hermite)正定线性系统。

接口:

```
int vsip_cholsol_d(const vsip_chol_d *chold, const vsip_mview_d *XB);
int vsip_cholsol_f(const vsip_chol_f *chold, const vsip_mview_f *XB);
int vsip_ccholsol_d(const vsip_cchol_d *chold, const vsip_cmview_d *XB);
int vsip_ccholsol_f(const vsip_cchol_f *chold, const vsip_cmview_f *XB);
```

参数:

chold: 指向由 vsip_dchold_create_f 创建的 Cholesky 分解对象的指针。

XB: 输入 B/输出 X 的视图, 大小为 NxK 的矩阵。

返回值:

成功时返回零。

4.2.5.6 超定线性系统解算器 (新增 1 个)

4.2.5.6.1 vsip_dqrd_p (新增 1 个)

计算矩阵的 QR 分解。A=QR。

接口:

```
int vsip_qrd_f(vsip_qr_f *qrd, const vsip_mview_f *A);
int vsip_qrd_d(vsip_qr_d *qrd, const vsip_mview_d *A);
int vsip_cqrd_d(vsip_cqr_d *qrd, const vsip_cmview_d *A);
int vsip_cqrd_f(vsip_cqr_f *qrd, const vsip_cmview_f *A);
int vsip_cmqr_f(const vsip_cmview_f *A, const vsip_cmview_f *r, const
vsip_cvview_f *tau); (新增接口)
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

A: 输入 MxN 矩阵 A 的视图。

注:

vsip_cmqr_f 接口要求输入矩阵实部虚部应连续交叉存储。

返回值:

成功时返回零。如果 A 列满秩，则失败并返回非零， $\text{rank}(A)=N$ 。

4.2.5.6.2 vsip_dqrd_create_p

创建 QR 分解对象。A=QR。

接口:

```
vsip_qr_d *vsip_qrd_create_d(vsip_length M,vsip_length N,vsip_qrd_qopt qopt);  
vsip_qr_f *vsip_qrd_create_f(vsip_length M,vsip_length N,vsip_qrd_qopt qopt);  
vsip_cqr_d *vsip_cqrd_create_d(vsip_length M,vsip_length N,vsip_qrd_qopt  
qopt);  
vsip_cqr_f *vsip_cqrd_create_f(vsip_length M,vsip_length N,vsip_qrd_qopt  
qopt);
```

参数:

M: 输入矩阵 A 的行数。

N: 输入矩阵 A 的列数。

qopt: 指示是否保留矩阵 Q。

返回值:

返回值是指向 QR 分解对象的指针，如果失败，则返回 NULL。

4.2.5.6.3 vsip_dqrd_destroy_p

销毁 QR 分解对象。

接口:

```
int vsip_qrd_destroy_d(vsip_qr_d *qrd);  
int vsip_qrd_destroy_f(vsip_qr_f *qrd);  
int vsip_cqrd_destroy_d(vsip_cqr_d *qrd);  
int vsip_cqrd_destroy_f(vsip_cqr_f *qrd);
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

返回值:

成功时返回零，失败时返回非零。

4.2.5.6.4 vsip_dqrd_getattr_p

返回 QR 分解对象的属性。

接口:


```
void vsip_qrd_getattr_d(const vsip_qr_d *qrd, vsip_qr_attr_d *attr);
void vsip_qrd_getattr_f(const vsip_qr_f *qrd, vsip_qr_attr_f *attr);
void vsip_cqrd_getattr_d(const vsip_cqr_d *qrd, vsip_cqr_attr_d *attr);
void vsip_cqrd_getattr_f(const vsip_cqr_f *qrd, vsip_cqr_attr_f *attr);
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

attr: 指向输出属性结构的指针。

返回值:

空。

4.2.5.6.5 vsip_dqrdprodq_p

将矩阵乘以来自 QR 分解的矩阵 Q。

接口:

```
int vsip_qrdprodq_d(const vsip_qr_d *qrd, vsip_mat_op opQ, vsip_mat_side apQ,
const vsip_mview_d *C);
```

```
int vsip_qrdprodq_f(const vsip_qr_f *qrd, vsip_mat_op opQ, vsip_mat_side apQ,
const vsip_mview_f *C);
```

```
int vsip_cqrdprodq_d(const vsip_cqr_d *qrd, vsip_mat_op opQ, vsip_mat_side
apQ, const vsip_cmview_d *C);
```

```
int vsip_cqrdprodq_f(const vsip_cqr_f *qrd, vsip_mat_op opQ, vsip_mat_side
apQ, const vsip_cmview_f *C);
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

opQ: 指定 op(Q)的形式。

apQ: 指示 op(Q)是应用于 C 的左侧还是右侧。

C: 输入时, 输入 R×S 矩阵 C 视图。在输出时, 数据以自然顺序存储在由输入矩阵视图的偏移量、行步幅和列步幅确定的块中。

返回值:

成功时返回零。

4.2.5.6.6 vsip_dqrdsolr_p

基于矩阵 R, 从矩阵 A 的 QR 分解求解线性系统。

接口:

```
int vsip_qrdsolr_d(const vsip_qr_d *qrd, vsip_mat_op opR, vsip_scalar_d alpha,
const vsip_mview_d *XB);
```

```
int vsip_qrdsolr_f(const vsip_qr_f *qrd, vsip_mat_op opR, vsip_scalar_f alpha,
const vsip_mview_f *XB);
int vsip_cqrdsolr_d(const vsip_cqr_d *qrd, vsip_mat_op opR, vsip_cscalar_d
alpha, const vsip_cmview_d *XB);
int vsip_cqrdsolr_f(const vsip_cqr_f *qrd, vsip_mat_op opR, vsip_cscalar_f
alpha, const vsip_cmview_f *XB);
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

opR: 指定 op(R)的形式。

alpha: (实数/复数)标量。

XB: 大小为 NxK 的输入/输出矩阵视图。

返回值:

成功时返回零。

4.2.5.6.7 vsip_dqrsol_p

解决线性协方差或线性最小二乘问题。

接口:

```
int vsip_qrsol_d(const vsip_qr_d *qrd, vsip_qrd_prob, const vsip_mview_d *XB);
int vsip_qrsol_f(const vsip_qr_f *qrd, vsip_qrd_prob, const vsip_mview_f *XB);
int vsip_cqrsol_d(const vsip_cqr_d *qrd, vsip_qrd_prob, const vsip_cmview_d
*XB);
int vsip_cqrsol_f(const vsip_cqr_f *qrd, vsip_qrd_prob, const vsip_cmview_f *XB);
```

参数:

qrd: 指向由 vsip_dqrd_create_f 创建的 QR 分解对象的指针。

prob: 在协方差问题和线性最小二乘问题之间进行选择。

XB: 大小为 NxK 的输入/输出矩阵视图。

返回值:

成功时返回零。

4.2.5.7 奇异值分解 (新增 32 个)

4.2.5.7.1 vsip_dsvd_p (新增 4 个)

计算矩阵的奇异值分解。

接口:

```
int vsip_svd_d(vsip_sv_d *svd, const vsip_mview_d *A, vsip_vview_d *s); (新
```

增接口)

int vsip_svd_f(vsip_sv_f *svd, const vsip_mview_f *A, vsip_vview_f *s); (新增接口)

int vsip_csvd_d(vsip_csv_d *svd, const vsip_cmview_d *A, vsip_vview_d *s); (新增接口)

int vsip_csvd_f(vsip_csv_f *svd, const vsip_cmview_f *A, vsip_vview_f *s); (新增接口)

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

A: 输入 MxN 矩阵视图。

s: 长度为 min(M,N)的实数向量, 包含 A 的输出奇异值 (按降序排列)。

返回值:

如果成功, 则返回零。如果算法无法完成, 则返回非零值。

4.2.5.7.2 vsip_dsvd_create_p (新增 4 个)

创建奇异值分解(SVD)对象。

接口:

vsip_sv_d *vsip_svd_create_d(vsip_length M, vsip_length N, vsip_svd_uv Usave, vsip_svd_uv Vsave); (新增接口)

vsip_sv_f *vsip_svd_create_f(vsip_length M, vsip_length N, vsip_svd_uv Usave, vsip_svd_uv Vsave); (新增接口)

vsip_csv_d *vsip_csvd_create_d(vsip_length M, vsip_length N, vsip_svd_uv Usave, vsip_svd_uv Vsave); (新增接口)

vsip_csv_f *vsip_csvd_create_f(vsip_length M, vsip_length N, vsip_svd_uv Usave, vsip_svd_uv Vsave); (新增接口)

参数:

M: 输入矩阵 A 的行数。

N: 输入矩阵 A 的列数。

Usave: 指定用于计算 U 的选项。

Vsave: 指定用于计算 $V^T(V^H)$ 的选项。

返回值:

返回值是指向 SVD 对象的指针, 如果失败, 则返回 NULL。

4.2.5.7.3 vsip_dsvd_destroy_p (新增 4 个)

销毁 SVD 对象。

接口:

```
int vsip_svd_destroy_d(vsip_sv_d *svd); (新增接口)
int vsip_svd_destroy_f(vsip_sv_f *svd); (新增接口)
int vsip_csvd_destroy_d(vsip_csv_d *svd); (新增接口)
int vsip_csvd_destroy_f(vsip_csv_f *svd); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

返回值:

成功时返回零, 失败时返回非零。

4.2.5.7.4 vsip_dsvd_getattr_p (新增 4 个)

返回 SVD 对象的属性。

接口:

```
void vsip_svd_getattr_d(const vsip_sv_d *svd, vsip_sv_attr_d *attr); (新增接口)
void vsip_svd_getattr_f(const vsip_sv_f *svd, vsip_sv_attr_f *attr); (新增接口)
void vsip_csvd_getattr_d(const vsip_csv_d *svd, vsip_csv_attr_d *attr); (新增接口)
void vsip_csvd_getattr_f(const vsip_csv_f *svd, vsip_csv_attr_f *attr); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

attr: 指向输出属性结构的指针。

返回值:

空。

4.2.5.7.5 vsip_dsvdprodu_p (新增 4 个)

将矩阵乘以来自奇异值分解的矩阵 U。

接口:

```
int vsip_svdprodu_d(const vsip_sv_d *svd, vsip_mat_op OpU, vsip_mat_side ApU, const vsip_mview_d *C); (新增接口)
int vsip_svdprodu_f(const vsip_sv_f *svd, vsip_mat_op OpU, vsip_mat_side ApU, const vsip_mview_f *C); (新增接口)
int vsip_csvdprodu_d(const vsip_csv_d *svd, vsip_mat_op OpU, vsip_mat_side ApU, const vsip_cmview_d *C); (新增接口)
int vsip_csvdprodu_f(const vsip_csv_f *svd, vsip_mat_op OpU, vsip_mat_side ApU, const vsip_cmview_f *C); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

OpU: 指定 op(U)的形式。

ApU: 指示 op(U)是应用于 C 的左侧还是右侧。

C: 输入时, 输入 RxS 矩阵 C 视图。在输出时, 数据以自然顺序存储在由输入矩阵视图的偏移量、行步幅和列步幅确定的块中。

返回值:

成功时返回零。

4.2.5.7.6 vsip_dsvdprodv_p (新增 4 个)

将矩阵乘以来自奇异值分解的矩阵 V。

接口:

```
int vsip_svdprodv_d(const vsip_sv_d *svd, vsip_mat_op OpV, vsip_mat_side ApV,
const vsip_mview_d *C); (新增接口)
```

```
int vsip_svdprodv_f(const vsip_sv_f *svd, vsip_mat_op OpV, vsip_mat_side ApV,
const vsip_mview_f *C); (新增接口)
```

```
int vsip_csvdprodv_d(const vsip_csv_d *svd, vsip_mat_op OpV, vsip_mat_side
ApV, const vsip_cmview_d *C); (新增接口)
```

```
int vsip_csvdprodv_f(const vsip_csv_f *svd, vsip_mat_op OpV, vsip_mat_side
ApV, const vsip_cmview_f *C); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

OpV: 指定 op(V)的形式。

ApV: 指示 op(V)是应用于 C 的左侧还是右侧。

C: 输入时, 输入 RxS 矩阵 C 视图。在输出时, 数据以自然顺序存储在由输入矩阵视图的偏移量、行步幅和列步幅确定的块中。

返回值:

成功时返回零。

4.2.5.7.7 vsip_dsvdmatu_p (新增 4 个)

从奇异值分解返回矩阵 U 中的连续列。

接口:

```
int vsip_svdmatu_d(const vsip_sv_d *svd, vsip_scalar_vi low, vsip_scalar_vi high,
const vsip_mview_d *C); (新增接口)
```

```
int vsip_svdmatu_f(const vsip_sv_f *svd, vsip_scalar_vi low, vsip_scalar_vi high,
const vsip_mview_f *C); (新增接口)
```

```
int vsip_csvdmatu_d(const vsip_csv_d *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_cmview_d *C); (新增接口)
```

```
int vsip_csvdmatu_f(const vsip_csv_f *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_cmview_f *C); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

low: 指定 U 中的第一列。

high: 指定 U 中的最后一列。

C: 在输出时, 数据以自然顺序存储在由输入矩阵视图的偏移量、行步幅和列步幅确定的块中。

返回值:

成功时返回零。

4.2.5.7.8 vsip_dsvdmatv_p (新增 4 个)

从奇异值分解返回矩阵 V 中的连续列。

接口:

```
int vsip_svdmatv_d(const vsip_sv_d *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_mview_d *C); (新增接口)
```

```
int vsip_svdmatv_f(const vsip_sv_f *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_mview_f *C); (新增接口)
```

```
int vsip_csvdmatv_d(const vsip_csv_d *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_cmview_d *C); (新增接口)
```

```
int vsip_csvdmatv_f(const vsip_csv_f *svd, vsip_scalar_vi low, vsip_scalar_vi high, const vsip_cmview_f *C); (新增接口)
```

参数:

svd: 指向由 vsip_dsvd_create_f 创建的 SVD 对象的指针。

low: 指定 V 中的第一列。

high: 指定 V 中的最后一列。

C: 在输出时, 数据以自然顺序存储在由输入矩阵视图的偏移量、行步幅和列步幅确定的块中。

返回值:

成功时返回零。

4.2.6 VSIPL-FT 信号处理函数（新增 45 个）

4.2.6.1 介绍

本节提供了 VSIPL-FT 库的 FFT、Window、卷积/相关、Filter 和其他信号处理的函数说明。

4.2.6.2 FFT 函数（新增 10 个）

4.2.6.2.1 vsip_ccfftx_p

应用复数到复数的快速傅立叶变换(FFT)。

接口：

```
void vsip_ccfftop_d(const vsip_fft_d *fft,const vsip_cvview_d *x,const vsip_cvview_d *y);
void vsip_ccfftop_f(const vsip_fft_f *fft,const vsip_cvview_f *x,const vsip_cvview_f *y);
void vsip_ccfftip_d(const vsip_fft_d *fft,const vsip_cvview_d *xy);
void vsip_ccfftip_f(const vsip_fft_f *fft,const vsip_cvview_f *xy);
```

参数：

fft：指向由 vsip_ccfftx_create_p 创建的 1D FFT 对象的指针。

x：长度为 N 的输入复向量视图。

y：长度为 N 的输出复向量视图。

xy：长度为 N 的输入/输出复向量视图。

返回值：

空。

4.2.6.2.2 vsip_crfftop_p

应用复数到实数快速傅立叶变换(FFT)。

接口：

```
void vsip_crfftop_d(const vsip_fft_d *fft,const vsip_cvview_d *x,const vsip_vview_d *y);
void vsip_crfftop_f(const vsip_fft_f *fft,const vsip_cvview_f *x,const vsip_vview_f *y);
```

参数：

fft: 指向由 vsip_crfftop_create_p 创建的 1D FFT 对象的指针。

x: 长度为 $N/2+1$ 的输入复数向量视图, 其中索引为 0 的值包含 DC (0 频率) 值, 索引为 $N/2$ 的值包含折叠频率 (一半采样率) 值。

y: 长度为 N 的输出实向量视图。

返回值:

空。

4.2.6.2.3 vsip_rcfftop_p

应用实数到复数的快速傅立叶变换(FFT)。

接口:

```
void vsip_rcfftop_d(const vsip_fft_d *fft, const vsip_vview_d *x, const vsip_cvview_d *y);
```

```
void vsip_rcfftop_f(const vsip_fft_f *fft, const vsip_vview_f *x, const vsip_cvview_f *y);
```

参数:

fft: 指向由 vsip_rcfftop_create_p 创建的 1D FFT 对象的指针。

y: 长度为 $N/2+1$ 的输出复数向量视图, 其中索引为 0 的值包含 DC (0 频率) 值, 索引为 $N/2$ 的值包含折叠频率 (一半采样率) 值。

x: 长度为 N 的输入实向量视图。

返回值:

空。

4.2.6.2.4 vsip_dfftx_create_p

创建一维 FFT 对象。

接口:

```
vsip_fft_d *vsip_ccfftop_create_d(vsip_length length, vsip_scalar_d scale, vsip_fft_dir sign, unsigned int ntimes, vsip_alg_hint hint);
```

```
vsip_fft_f *vsip_ccfftop_create_f(vsip_length length, vsip_scalar_f scale, vsip_fft_dir sign, unsigned int ntimes, vsip_alg_hint hint);
```

```
vsip_fft_d *vsip_ccfftip_create_d(vsip_length length, vsip_scalar_d scale, vsip_fft_dir sign, unsigned int ntimes, vsip_alg_hint hint);
```

```
vsip_fft_f *vsip_ccfftip_create_f(vsip_length length, vsip_scalar_f scale, vsip_fft_dir sign, unsigned int ntimes, vsip_alg_hint hint);
```

```
vsip_fft_d *vsip_crfftop_create_d(vsip_length length, vsip_scalar_d scale, unsigned int ntimes, vsip_alg_hint hint);
```



```
vsip_fft_f *vsip_crfftop_create_f(vsip_length length,vsip_scalar_f scale,unsigned
int ntimes,vsip_alg_hint hint);
vsip_fft_d *vsip_rcfftop_create_d(vsip_length length,vsip_scalar_d
scale,unsigned int ntimes,vsip_alg_hint hint);
vsip_fft_f *vsip_rcfftop_create_f(vsip_length length,vsip_scalar_f scale,unsigned
int ntimes,vsip_alg_hint hint);
```

参数:

length: FFT 长度。

scale: 执行完 FFT 后的缩放系数, 比例的典型值为 1、1/N、1/√N。

sign: 正向或反向 FFT (请注意, 该参数仅适用于复数到复数)。

ntimes: 执行 FFT 的次数。值为零被视为半无限。

hint: 帮助确定筛选方法的提示。

返回值:

返回值是指向 1D FFT 对象的指针, 如果失败, 则返回 NULL。

4.2.6.2.5 vsip_ccfftmx_p

应用多个复数到复数快速傅立叶变换(FFT)。

接口:

```
void vsip_ccfftmop_d(const vsip_fftm_d *fft,const vsip_cmview_d *x,const
vsip_cmview_d *y);
void vsip_ccfftmop_f(const vsip_fftm_f *fft,const vsip_cmview_f *x,const
vsip_cmview_f *y);
void vsip_ccfftmip_d(const vsip_fftm_d *fft,const vsip_cmview_d *xy);
void vsip_ccfftmip_f(const vsip_fftm_f *fft,const vsip_cmview_f *xy);
```

参数:

fft: 指向由 vsip_ccfftmx_create_p 创建的 1D FFT 对象的指针。

x: 输入大小为 MxN 的复矩阵视图。

y: 输出大小为 MxN 的复矩阵视图。

xy: 输入/输出大小为 MxN 的复向量视图。

返回值:

空。

4.2.6.2.6 vsip_crfftmop_p

应用多个复数到实数快速傅立叶变换(FFT)。

接口:

```
void vsip_crfftmop_d(const vsip_fftm_d *fft,const vsip_cmview_d *x,const vsip_mview_d *y);  
void vsip_crfftmop_f(const vsip_fftm_f *fft,const vsip_cmview_f *x,const vsip_mview_f *y);
```

参数:

fft: 指向由 vsip_crfftmop_create_p 创建的 1D FFT 对象的指针。

x: 输入大小为 $(M/2+1) \times N$ 或 $M \times (N/2+1)$ 的复矩阵视图。

y: 输出大小为 $M \times N$ 的复矩阵视图。

返回值:

空。

4.2.6.2.7 vsip_rcfftmop_p

应用多个实数到复数的位置快速傅立叶变换(FFT)。

接口:

```
void vsip_rcfftmop_d(const vsip_fftm_d *fft,const vsip_mview_d *x,const vsip_cmview_d *y);  
void vsip_rcfftmop_f(const vsip_fftm_f *fft,const vsip_mview_f *x,const vsip_cmview_f *y);
```

参数:

fft: 指向由 vsip_rcfftmop_create_p 创建的 1D FFT 对象的指针。

y: 输入大小为 $(M/2+1) \times N$ 或 $M \times (N/2+1)$ 的复矩阵视图。

x: 输出大小为 $M \times N$ 的复矩阵视图。

返回值:

空。

4.2.6.2.8 vsip_dfftmx_create_p

创建一个 1D 多 FFT 对象。

接口:

```
vsip_fftm_d *vsip_ccfftmip_create_d(vsip_length M, vsip_length N, vsip_scalar_d scale, vsip_fft_dir dir, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);  
vsip_fftm_f *vsip_ccfftmip_create_f(vsip_length M, vsip_length N, vsip_scalar_f scale, vsip_fft_dir dir, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);  
vsip_fftm_d *vsip_ccfftmop_create_d(vsip_length M, vsip_length N, vsip_scalar_d scale, vsip_fft_dir dir, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
```

```
vsip_fftm_f *vsip_ccfftmop_create_f(vsip_length M, vsip_length N, vsip_scalar_f
scale, vsip_fft_dir dir, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
vsip_fftm_d *vsip_crfftmop_create_d(vsip_length M, vsip_length N,
vsip_scalar_d scale, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
vsip_fftm_f *vsip_rcfftmop_create_f(vsip_length M, vsip_length N, vsip_scalar_f
scale, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
vsip_fftm_d *vsip_rcfftmop_create_d(vsip_length M, vsip_length N,
vsip_scalar_d scale, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
vsip_fftm_f *vsip_rcfftmop_create_f(vsip_length M, vsip_length N, vsip_scalar_f
scale, vsip_major major, unsigned int ntimes, vsip_alg_hint hint);
```

参数:

M: 列 FFT 的长度或行 fft 的行数。

N: 行 FFT 的长度或列 fft 的数。

scale: 实际比例因子, 比例的典型值为 1、1/N、1/√M、1/√N。

dir: 正向或反向 FFT (请注意, 该参数仅适用于复数到复数)。

major: 多个 FFT 的方向。

ntimes: 估计将调用 FFT 对象的次数。值为零被视为半无限。

hint: 帮助确定滤波方法的提示。

返回值:

返回值是指向 1D FFT 对象的指针, 如果失败, 则返回 NULL。

4.2.6.2.9 vsip_ccfft2dx_p (新增 2 个)

应用复数到复数的 2D 快速傅立叶变换(FFT)。

接口:

```
void vsip_ccfft2dop_d(const vsip_fft2d_d *Offt2d,const vsip_cmview_d *x,const
vsip_cmview_d *y); ( 新增接口 )
```

```
void vsip_ccfft2dop_f(const vsip_fft2d_f *Offt2d,const vsip_cmview_f *x,const
vsip_cmview_f *y); ( 新增接口 )
```

参数:

Offt2d: 指向由 vsip_ccfft2dx_create_p 创建的 2D FFT 对象的指针。

x: 输入大小为 MxN 的复矩阵视图。

y: 输出大小为 MxN 的复矩阵视图。

返回值:

空。

4.2.6.2.10 vsip_dfft2dx_create_p (新增 2 个)

创建 2D FFT 对象。

接口：

```
vsip_fft2d_d      *vsip_ccfft2dop_create_d(vsip_length      M,vsip_length
N,vsip_scalar_d scale,vsip_fft_dir dir,unsigned int ntimes,vsip_alg_hint hint); (新增接口)
vsip_fft2d_f      *vsip_ccfft2dop_create_f(vsip_length M,vsip_length N,vsip_scalar_f
scale,vsip_fft_dir dir,unsigned int ntimes,vsip_alg_hint hint); (新增接口)
```

参数：

M：FFT 大小为 M×N。

N：FFT 大小为 M×N。

scale：实际比例因子，比例的典型值为 1、1/N、1/√M、1/√N、1/√(M*N)。

dir：正向或反向 FFT（请注意，该参数仅适用于复数到复数）。

ntimes：估计将调用 FFT 对象的次数。值为零被视为半无限。

hint：帮助确定滤波方法的提示。

返回值：

返回值是指向 2D FFT 对象的指针，如果失败，则返回 NULL。

4.2.6.2.11 vsip_dfftn_destroy_f (新增 2 个)

销毁 FFT 对象。

接口：

```
int vsip_fft_destroy_d(vsip_fft_d *fft);
int vsip_fft_destroy_f(vsip_fft_f *fft);
int vsip_fft2d_destroy_d(vsip_fft2d_d *fft2d); (新增接口)
int vsip_fft2d_destroy_f(vsip_fft2d_f *fft2d); (新增接口)
int vsip_fftm_destroy_d(vsip_fftm_d *fft);
int vsip_fftm_destroy_f(vsip_fftm_f *fft);
```

参数：

fft、fft2d：要销毁的 FFT 对象。

返回值：

成功时返回零，失败时返回非零。

4.2.6.2.12 vsip_fftn_getattr_f

获取 FFT 对象的属性。

接口：

```
void vsip_fft_getattr_d(const vsip_fft_d *fft, vsip_fft_attr_d *attr);  
void vsip_fft_getattr_f(const vsip_fft_f *fft, vsip_fft_attr_f *attr);  
void vsip_fftm_getattr_d(const vsip_fftm_d *fft, vsip_fftm_attr_d *attr);  
void vsip_fftm_getattr_f(const vsip_fftm_f *fft, vsip_fftm_attr_f *attr);
```

参数:

fft: FFT 对象。
attr: FFT 对象的属性。

返回值:

空。

4.2.6.2.13 vsip_cvfftshift_p (新增 2 个)

将复向量以中心点位置左右两半部分进行互换。

接口:

```
void vsip_cvfftshift_d(const vsip_cvview_d* a, const vsip_cvview_d* r); (新增接口)  
void vsip_cvfftshift_f(const vsip_cvview_f* a, const vsip_cvview_f* r); (新增接口)
```

参数:

a: 输入向量视图。
b: 输出向量视图。

返回值:

空。

4.2.6.2.14 vsip_cvcycshift_p (新增 2 个)

复数向量循环移位。

接口:

```
void vsip_cvcycshift_f(const vsip_cvview_f* a, const vsip_cvview_f* r,  
vsip_vshift_dir dir, vsip_length L); (新增接口)  
void vsip_cvcycshift_d(const vsip_cvview_d* a, const vsip_cvview_d* r,  
vsip_vshift_dir dir, vsip_length L); (新增接口)
```

参数:

a: 输入向量视图。
r: 输出向量视图。
dir: 位移方向, VSIP_RIGHT 向右循环移位, VSIP_LEFT 向左循环移位。
L: 位移长度。

注:

循环移位的长度必须小于向量的长度且循环移位后的向量长度应与输入向量长度

相同。

返回值：

空。

4.2.6.3 希尔伯特变换（新增 8 个）

4.2.6.3.1 vsip_dchilbertop_p（新增 4 个）

输入复数或实数向量 x , 运算后 z 为 x 的希尔伯特变换。

接口：

`void vsip_cchilbertop_f(const vsip_cvview_f* x, const vsip_cvview_f* z);`（新增接口）

`void vsip_cchilbertop_d(const vsip_cvview_d* x, const vsip_cvview_d* z);`（新增接口）

`void vsip_rchilbertop_f(const vsip_vview_f* x, const vsip_vview_f* z);`（新增接口）

`void vsip_rchilbertop_d(const vsip_vview_d* x, const vsip_vview_d* z);`（新增接口）

参数：

x ：输入复数或实数向量 x 。

z ：输出希尔伯特变换 z 。

返回值：

空。

4.2.6.3.2 vsip_dchilbertip_p（新增 4 个）

输入复数或实数向量 x , 运算后 x 为源 x 的希尔伯特变换。

接口：

`void vsip_cchilbertip_f(const vsip_cvview_f* x);`（新增接口）

`void vsip_cchilbertip_d(const vsip_cvview_d* x);`（新增接口）

`void vsip_rchilbertip_f(const vsip_vview_f* x);`（新增接口）

`void vsip_rchilbertip_d(const vsip_vview_d* x);`（新增接口）

参数：

x ：输入复数或实数向量/输出希尔伯特变换。

返回值：

空。

4.2.6.4 卷积/相关函数（新增 7 个）

4.2.6.4.1 vsip_dconv1d_create_p（新增 1 个）

创建一个抽取的一维卷积滤波器对象。

接口：

```
vsip_conv1d_d *vsip_conv1d_create_d(const vsip_vview_d *a, vsip_symmetry  
symm, vsip_length N, int D, vsip_support_region support, unsigned int  
ntimes, vsip_alg_hint hint);
```

```
vsip_conv1d_f *vsip_conv1d_create_f(const vsip_vview_f *a,  
vsip_symmetry, vsip_length N, int D, vsip_support_region support, unsigned int  
ntimes, vsip_alg_hint hint);
```

```
vsip_cconv1d_f *vsip_cconv1d_create_f(const vsip_cvview_f *h, vsip_symmetry  
symm, vsip_length N, int D, vsip_support_region support, unsigned int  
ntimes, vsip_alg_hint hint); (新增接口)
```

参数：

a：指向滤波器系数的矢量视图对象的指针，

- 非对称：长度 M

- 对称：长度 $[M/2]$

symm：滤波器对称性，包括长度对称性{偶数、奇数}。

N：输入矢量视图的长度。

D：抽取因子 (≥ 1)。

support：支持的输出区域（指示要计算的输出点）。

ntimes：估计将应用多少卷积。零值被视为半无限。

hint：帮助确定算法方法的提示。

返回值：

返回指向 1D 卷积滤波器对象的指针，如果创建失败，则返回 NULL。

4.2.6.4.2 vsip_dconv1d_destroy_p（新增 1 个）

销毁一维卷积对象并释放任何关联的内存。

接口：

```
int vsip_conv1d_destroy_d(vsip_conv1d_d *r);
```

```
int vsip_conv1d_destroy_f(vsip_conv1d_f *r);
```

```
int vsip_cconv1d_destroy_f(vsip_cconv1d_f *r); (新增接口)
```

参数：

r: 指向 1D 卷积对象的指针。

返回值:

成功时返回零, 失败时返回非零。

4.2.6.4.3 vsip_dconv1d_getattr_p (新增 1 个)

返回一维卷积对象的属性。

接口:

```
void vsip_conv1d_getattr_d( const vsip_conv1d_d *a, vsip_conv1d_attr *attr);  
void vsip_conv1d_getattr_f( const vsip_conv1d_f *a, vsip_conv1d_attr *attr);  
void vsip_cconv1d_getattr_f(const vsip_cconv1d_f *conv, vsip_conv1d_attr *attr);
```

(新增接口)

参数:

a: 指向 1D 卷积对象的指针。

attr: 指向 vsip_conv1d_attr_p 结构的指针。

返回值:

空。

4.2.6.4.4 vsip_dconvolve1d_p (新增 1 个)

计算两个向量的抽取实数一维(1D)卷积。

接口:

```
void vsip_convolve1d_d( const vsip_conv1d_d *a, const vsip_vview_d *x, const  
vsip_vview_d *y);  
void vsip_convolve1d_f( const vsip_conv1d_f *a, const vsip_vview_f *x, const  
vsip_vview_f *y);  
void vsip_cconvolve1d_f(const vsip_cconv1d_f *conv, const vsip_cvview_f  
*x, const vsip_cvview_f *y); (新增接口)
```

参数:

a: 指向 1D 卷积对象的指针。

x: 长度为 N 的实输入数据向量视图。

y: 实目标向量视图, 长度为:

- 满: $\lfloor (N + M - 2)/D \rfloor + 1$
- 相同: $\lfloor (N - 1)/D \rfloor + 1$
- 最低: $\lfloor (N - 1)/D \rfloor - \lfloor (M - 1)/D \rfloor + 1$

返回值:

空。

4.2.6.4.5 vsip_dconv2d_create_p (新增 1 个)

创建抽取 2D 卷积对象。

接口：

```
vsip_conv2d_f *vsip_cconv2d_create_f(const vsip_cmview_f *h, vsip_symmetry  
symm, vsip_length P, vsip_length Q, int D, vsip_support_region support, unsigned int  
ntimes, vsip_alg_hint hint); (新增接口)
```

参数：

h：指向卷积系数矩阵视图对象的指针，

- 非对称：长度 $M \times N$
- 对称：长度 $[M/2] \times [N/2]$

symm：卷积行和列对称性，包括长度对称性{偶数、奇数}。对称性适用于卷积的两个方向。

P：输入数据矩阵中的行数。

Q：输入数据矩阵中的列数。

D：抽取因子 (≥ 1)。

support：支持的输出区域（指示要计算的输出点）。

ntimes：估计将应用多少卷积。零值被视为半无限。

hint：帮助确定算法方法的提示。

注：

窄卷积(Narrow convolution)即 support=VSIP_SUPPORT_MIN 在 VSIP 中未实现。

返回值：

返回指向 2D 卷积滤波器对象的指针，如果创建失败，则返回 NULL。

4.2.6.4.6 vsip_dconv2d_destroy_f (新增 1 个)

销毁 2D 卷积对象。

接口：

```
int vsip_conv2d_destroy_f(vsip_conv2d_f *conv); (新增接口)
```

参数：

conv：指向 2D 卷积对象的指针。

返回值：

成功时返回零，失败时返回非零。

4.2.6.4.7 vsip_convolve2d_f (新增 1 个)

计算两个矩阵的抽取实数二维(2D)卷积。

接口:

```
void vsip_cconvolve2d_f(const vsip_conv2d_f *conv, const vsip_cmview_f *x, const vsip_cmview_f *y); (新增接口)
```

参数:

conv: 指向 2D 卷积对象的指针。

x: 实输入 PxQ 矩阵视图。

y: 实际目标矩阵视图, 规模为:

- 完整: $(\lfloor (P + M - 2)/D \rfloor + 1) \times (\lfloor (Q + N - 2)/D \rfloor + 1)$
- 相同: $(\lfloor (P - 1)/D \rfloor + 1) \times (\lfloor (Q - 1)/D \rfloor + 1)$
- 最小值: $(\lfloor (P - 1)/D \rfloor - \lfloor (M - 1)/D \rfloor + 1) \times (\lfloor (Q - 1)/D \rfloor - \lfloor (N - 1)/D \rfloor + 1)$

返回值:

空。

4.2.6.4.8 vsip_dcorr1d_create_p

创建一维关联对象。

接口:

```
vsip_corr1d_d *vsip_corr1d_create_d(vsip_length M, vsip_length N, vsip_support_region support, unsigned int ntimes, vsip_alg_hint hint);  
vsip_corr1d_f *vsip_corr1d_create_f(vsip_length M, vsip_length N, vsip_support_region support, unsigned int ntimes, vsip_alg_hint hint);  
vsip_ccorr1d_d *vsip_ccorr1d_create_d(vsip_length M, vsip_length N, vsip_support_region support, unsigned int ntimes, vsip_alg_hint hint);  
vsip_ccorr1d_f *vsip_ccorr1d_create_f(vsip_length M, vsip_length N, vsip_support_region support, unsigned int ntimes, vsip_alg_hint hint);
```

参数:

M: 输入参考向量视图 r 的长度。

N: 输入数据向量视图 x 的长度。

support: 支持的输出区域 (指示要计算的输出点)。

ntimes: 估计将应用多少卷积。值为零被视为半无限。

hint: 帮助确定算法方法的提示。

返回值:

返回指向 1D 关联对象的指针, 如果创建失败, 则返回 NULL。

4.2.6.4.9 vsip_dcorr1d_destroy_p

销毁一维相关对象。

接口：

```
int vsip_corr1d_destroy_d( vsip_corr1d_d *r);
int vsip_corr1d_destroy_f( vsip_corr1d_f *r);
int vsip_ccorr1d_destroy_d( vsip_ccorr1d_d *r);
int vsip_ccorr1d_destroy_f( vsip_ccorr1d_f *r);
```

参数：

r: 指向 1D 相关对象的指针。

返回值：

成功时返回零，失败时返回非零。

4.2.6.4.10 vsip_dcorr1d_getattr_p

返回一维相关对象的属性。

接口：

```
void vsip_corr1d_getattr_d( const vsip_corr1d_d *a, vsip_corr1d_attr *r);
void vsip_corr1d_getattr_f( const vsip_corr1d_f *a, vsip_corr1d_attr *r);
void vsip_ccorr1d_getattr_d( const vsip_ccorr1d_d *a, vsip_corr1d_attr *r);
void vsip_ccorr1d_getattr_f( const vsip_ccorr1d_f *a, vsip_corr1d_attr *r);
```

参数：

a: 指向 1D 相关对象的指针。

r: 指向 vsip_dcorr1d_attr_p 结构的指针。

返回值：

空。

4.2.6.4.11 vsip_dcorrelate_p

计算两个向量的实际一维（1D）相关性。

接口：

```
void vsip_correlate1d_d( const vsip_corr1d_d *a, vsip_bias bias, const
vsip_vview_d *ref, const vsip_vview_d *x, const vsip_vview_d *r);
void vsip_correlate1d_f( const vsip_corr1d_f *a, vsip_bias bias, const vsip_vview_f *ref,
const vsip_vview_f *x, const vsip_vview_f *r);
void vsip_ccorrelate1d_d( const vsip_ccorr1d_d *a, vsip_bias bias, const
vsip_cvview_d *ref, const vsip_cvview_d *x, const vsip_cvview_d *r);
```

```
void vsip_ccorrelate1d_f( const vsip_ccorr1d_f *a, vsip_bias, const vsip_cvview_f *  
ref, const vsip_cvview_f *x, const vsip_cvview_f *r);
```

参数:

a: 指向相关对象的指针。

bias: 选择有偏或无偏的相关性估计。

ref: 长度为 M 的实数 (复数) 参考数据向量视图。

x: 长度为 N 的实数 (复数) 输入数据向量视图。

y: 长度为 (N+M-1)、N 或 (N-M+1) (完整、相同或最小值) 的实数 (复数) 滞后向量视图。

返回值:

空。

4.2.6.5 Window 函数 (新增 6 个)

4.2.6.5.1 vsip_vcreate_blackman_p

创建一个具有 Blackman 窗口权重的向量。

接口:

```
vsip_vview_d *vsip_vcreate_blackman_d( vsip_length N, vsip_memory_hint hint);
```

```
vsip_vview_f *vsip_vcreate_blackman_f( vsip_length N, vsip_memory_hint hint);
```

参数:

N: 窗口向量的长度。

hint: 内存运用方式 (通常为 VSIP_MEM_CONST) 。

返回值:

返回指向矢量视图对象的指针, 或在失败时返回 NULL。

4.2.6.5.2 vsip_vcreate_bartlett_p (新增 2 个)

创建 bartlett 窗函数。

接口:

```
vsip_vview_f* vsip_vcreate_bartlett_f( vsip_length N, vsip_memory_hint h); ( 新增  
接口 )
```

```
vsip_vview_d* vsip_vcreate_bartlett_d( vsip_length N, vsip_memory_hint h); ( 新增  
接口 )
```

参数:

N: 窗口向量的长度。

hint: 内存运用方式 (通常为 VSIP_MEM_CONST) 。

返回值:

返回指向矢量视图对象的指针，或在失败时返回 NULL。

4.2.6.5.3 vsip_vcreate_cheby_p

创建一个具有 Dolph-Chebyshev 窗口权重的向量。

接口:

```
vsip_vview_d *vsip_vcreate_cheby_d(vsip_length N, vsip_scalar_d ripple,  
vsip_memory_hint hint);
```

```
vsip_vview_f *vsip_vcreate_cheby_f(vsip_length N, vsip_scalar_f ripple,  
vsip_memory_hint hint);
```

参数:

N: 窗口向量的长度。

ripple: 窗口纹波（单位为 db）。

hint: 内存运用方式（通常为 VSIP_MEM_CONST）。

返回值:

返回指向矢量视图对象的指针，或在失败时返回 NULL。

4.2.6.5.4 vsip_vcreate_hann_p（新增 2 个）

用于 hann 窗系数生成。

接口:

```
vsip_vview_d *vsip_vcreate_hann_d(vsip_length N, vsip_memory_hint hint);
```

```
vsip_vview_f *vsip_vcreate_hann_f(vsip_length N, vsip_memory_hint hint);
```

参数:

N: 窗口向量的长度。

hint: 内存类型提示（通常为 VSIP_MEM_CONST）。

返回值:

返回指向矢量视图对象的指针，或在失败时返回 NULL。

4.2.6.5.5 vsip_vcreate_hanning_p

创建一个具有 Hanning 窗口权重的向量。

接口:

```
vsip_vview_d *vsip_vcreate_hanning_d(vsip_length N, vsip_memory_hint hint);
```

```
vsip_vview_f *vsip_vcreate_hanning_f(vsip_length N, vsip_memory_hint hint);
```

参数:

N: 窗口向量的长度。

hint: 内存类型提示 (通常为 VSIP_MEM_CONST)。

返回值:

返回指向矢量视图对象的指针, 或在失败时返回 NULL。

4.2.6.5.6 vsip_vcreate_hamming_p (新增 2 个)

创建 hamming 窗口。

接口:

vsip_vview_f* vsip_vcreate_hamming_f(vsip_length,vsip_memory_hint); (新增接口)

vsip_vview_d* vsip_vcreate_hamming_d(vsip_length,vsip_memory_hint); (新增接口)

参数:

N: 窗口向量的长度。

hint: 内存类型提示 (通常为 VSIP_MEM_CONST)。

返回值:

返回指向矢量视图对象的指针, 或在失败时返回 NULL。

4.2.6.5.7 vsip_vcreate_kaiser_p

创建具有 Kaiser 窗口权重的矢量。

接口:

vsip_vview_d *vsip_vcreate_kaiser_d(vsip_length N,vsip_scalar_d beta,vsip_memory_hint hint);

vsip_vview_f *vsip_vcreate_kaiser_f(vsip_length N,vsip_scalar_f beta,vsip_memory_hint hint);

参数:

N: 窗口向量的长度。

beta: 实标量, 过渡宽度参数。

hint: 内存运用方式 (通常为 VSIP_MEM_CONST)。

返回值:

返回指向矢量视图对象的指针, 或在失败时返回 NULL。

4.2.6.6 Filter 函数（新增 10 个）

4.2.6.6.1 vsip_dfir_create_p

创建抽取的 FIR 滤波器对象。

接口：

```
vsip_fir_d *vsip_fir_create_d(const vsip_vview_d *a, vsip_symmetry symm,
vsip_length N, vsip_length D, vsip_obj_state state, unsigned int ntimes, vsip_alg_hint
hint);
```

```
vsip_fir_f *vsip_fir_create_f(const vsip_vview_f *a, vsip_symmetry symm,
vsip_length N, vsip_length D, vsip_obj_state state, unsigned int ntimes, vsip_alg_hint
hint);
```

```
vsip_cfir_d *vsip_cfir_create_d(const vsip_cvview_d *a, vsip_symmetry symm,
vsip_length N, vsip_length D, vsip_obj_state state, unsigned int ntimes, vsip_alg_hint
hint);
```

```
vsip_cfir_f *vsip_cfir_create_f(const vsip_cvview_f *a, vsip_symmetry symm,
vsip_length N, vsip_length D, vsip_obj_state state, unsigned int ntimes, vsip_alg_hint
hint);
```

参数：

a：指向非冗余 FIR 滤波器系数的矢量视图对象的指针，

- 非对称：长度 $M+1$ ；
- （共轭）对称：长度 $\lceil (M+1)/2 \rceil$ 。

symm：核对称性，包括长度对称性{偶数、奇数}。

N：输入数据段的长度。

D：抽取因子。

state：对象状态历史记录要求。如果对象将用于过滤多个向量设置为 VSIP_STATE_SAVE。如果对象将用于单次调用滤波，请将 state 设置为 VSIP_STATE_NO_SAVE。

ntimes：估计将过滤的区段数。值为零被视为半无穷大。

hint：帮助确定算法方法的提示。

返回值：

返回指向 FIR 滤波器对象的指针，如果创建失败，则返回 NULL。

4.2.6.6.2 vsip_dfir_destroy_p

销毁 FIR 滤波器对象。

接口:

```
int vsip_fir_destroy_d(vsip_fir_d *filt);
int vsip_fir_destroy_f(vsip_fir_f *filt);
int vsip_cfir_destroy_d(vsip_cfir_d *filt);
int vsip_cfir_destroy_f(vsip_cfir_f *filt);
```

参数:

filt: 指向 FIR 滤波器对象的指针。

返回值:

成功时返回零, 失败时返回非零。

4.2.6.6.3 vsip_dfirflt_p

FIR 对输入序列进行滤波并抽取输出。

接口:

```
int vsip_firflt_d(vsip_fir_d *a, const vsip_vview_d *x, const vsip_vview_d *y);
int vsip_firflt_f(vsip_fir_f *a, const vsip_vview_f *x, const vsip_vview_f *y);
int vsip_cfirflt_d(vsip_cfir_d *a, const vsip_cvview_d *x, const vsip_cvview_d *y);
int vsip_cfirflt_f(vsip_cfir_f *a, const vsip_cvview_f *x, const vsip_cvview_f *y);
```

参数:

a: 指向 FIR 滤波器对象的指针。

x: 指向输入序列的指针 (矢量视图对象) 。

y: 指向输出序列的指针 (矢量视图对象) 。

返回值:

返回值是计算的输出样本数。

4.2.6.6.4 vsip_dfir_getattr_p

返回 FIR 滤波器对象的属性。

接口:

```
void vsip_fir_getattr_d(const vsip_fir_d *a, vsip_fir_attr *r);
void vsip_fir_getattr_f(const vsip_fir_f *a, vsip_fir_attr *r);
void vsip_cfir_getattr_d(const vsip_cfir_d *a, vsip_cfir_attr *r);
void vsip_cfir_getattr_f(const vsip_cfir_f *a, vsip_cfir_attr *r);
```

参数:

a: 指向 FIR 滤波器对象的指针。

r: 指向 vsip_dfir_attr_p 结构的指针。

返回值:

空。

4.2.6.6.5 vsip_dfir_reset_p

重置抽取的 FIR 滤波器对象的状态。

接口：

```
void vsip_fir_reset_d(vsip_fir_d *filt);  
void vsip_fir_reset_f(vsip_fir_f *filt);  
void vsip_cfir_reset_d(vsip_cfir_d *filt);  
void vsip_cfir_reset_f(vsip_cfir_f *filt);
```

参数：

filt：指向 FIR 滤波器对象的指针。

返回值：

空。

4.2.6.6.6 vsip_firgenbandpass_p (新增 2 个)

FIR 带通滤波器。

接口：

```
void vsip_firgenbandpass_d( vsip_scalar_d freq_low, vsip_scalar_d freq_high,  
const vsip_vview_d* taps, const vsip_vview_d* windows, vsip_bool doNormal);( 新增接  
口 )
```

```
void vsip_firgenbandpass_f( vsip_scalar_f freq_low, vsip_scalar_f freq_high,  
const vsip_vview_f* taps, const vsip_vview_f* windows, vsip_bool doNormal);( 新增接  
口 )
```

参数：

freq_low：低截至频率(0,0.5)且小于 freqHigh。

freq_high：高截至频率(0,0.5)且大于 freqLow。

taps：输出指向存储计算出的 tap 值向量的非空指针。

windows：指定计算中使用的窗口类型（HMPP_WIN_BARTLETT、HMPP_WIN_BLACKMAN、HMPP_WIN_HAMMING 或 HMPP_WIN_HANN）。

doNormal：指定计算过滤系数是否进行归一化（HMPP_TRUE 或 HMPP_FALSE）。

返回值：

空。

4.2.6.6.7 vsip_firgenhighpass_p (新增 2 个)

FIR 高通滤波器。

接口：

```
void vsip_firgenhighpass_d( vsip_scalar_d freq, const vsip_vview_d* taps, const vsip_vview_d* windows, vsip_bool doNormal); (新增接口)
```

```
void vsip_firgenhighpass_f( vsip_scalar_f freq, const vsip_vview_f* taps, const vsip_vview_f* windows, vsip_bool doNormal); (新增接口)
```

参数：

freq：截至频率(0,0.5)。

taps：输出指向存储计算出的 tap 值向量的非空指针。

windows：指定计算中使用的窗口类型（HMPP_WIN_BARTLETT、HMPP_WIN_BLACKMAN、HMPP_WIN_HAMMING 或 HMPP_WIN_HANN）。

doNormal：指定计算过滤系数是否进行归一化（HMPP_TRUE 或 HMPP_FALSE）。

返回值：

空。

4.2.6.6.8 vsip_firgenlowpass_p (新增 2 个)

FIR 低通滤波器。

接口：

```
void vsip_firgenlowpass_d(vsip_scalar_d freq,const vsip_vview_d* taps,const vsip_vview_d* windows,vsip_bool doNormal); (新增接口)
```

```
void vsip_firgenlowpass_f(vsip_scalar_f freq,const vsip_vview_f* taps,const vsip_vview_f* windows,vsip_bool doNormal); (新增接口)
```

参数：

freq：截至频率(0,0.5)。

taps：输出指向存储计算出的 tap 值向量的非空指针。

windows：指定计算中使用的窗口类型（HMPP_WIN_BARTLETT、HMPP_WIN_BLACKMAN、HMPP_WIN_HAMMING 或 HMPP_WIN_HANN）。

doNormal：指定计算过滤系数是否进行归一化（HMPP_TRUE 或 HMPP_FALSE）。

返回值：

空。

4.2.6.6.9 vsip_firgenbandstop_p (新增 2 个)

有限脉冲响应滤波器 FIR 带阻滤波器系数生成接口。

接口：

```
void vsip_firgenbandstop_d(vsip_scalar_d freq_low, vsip_scalar_d freq_high, const vsip_vview_d* taps, const vsip_vview_d* windows, vsip_bool doNormal); (新增接口)
```

```
void vsip_firgenbandstop_f(vsip_scalar_f freq_low, vsip_scalar_f freq_high, const vsip_vview_f* taps, const vsip_vview_f* windows, vsip_bool doNormal); (新增接口)
```

参数：

freq_low：低截至频率(0,0.5)且小于 freqHigh。

freq_high：高截至频率(0,0.5)且大于 freqLow。

taps：输出指向存储计算出的 tap 值向量的非空指针。

windows：指定计算中使用的窗口类型（HMPP_WIN_BARTLETT、HMPP_WIN_BLACKMAN、HMPP_WIN_HAMMING 或 HMPP_WIN_HANN）。

doNormal：指定计算过滤系数是否进行归一化（HMPP_TRUE 或 HMPP_FALSE）。

返回值：

空。

4.2.6.6.10 vsip_filter_p (新增 2 个)

用于 FIR 滤波。

接口：

```
vsip_vview_f* vsip_filter_f(const vsip_vview_f *h,const vsip_vview_f *x,vsip_symmetry symm,vsip_length N,int D,vsip_obj_state state,unsigned int ntimes,vsip_alg_hint hint); (新增接口)
```

```
vsip_vview_d* vsip_filter_d(const vsip_vview_d *h,const vsip_vview_d *x,vsip_symmetry symm,vsip_length N,int D,vsip_obj_state state,unsigned int ntimes,vsip_alg_hint hint); (新增接口)
```

参数：

h：系统响应信号。

x：输入信号。

N：输入信号长度。

其余参数同 vsip_dfir_create_p 类接口。

返回值：

返回滤波后信号。

4.2.6.7 其他信号处理函数（新增 4 个）

4.2.6.7.1 vsip_shisto_p

计算向量（矩阵）的直方图。

接口：

```
void vsip_mhisto_d(const vsip_mview_d *a, vsip_scalar_d min_bin,
vsip_scalar_d max_bin, vsip_hist_opt opt, const vsip_vview_d *r);
void vsip_mhisto_f(const vsip_mview_f *a, vsip_scalar_f min_bin, vsip_scalar_f
max_bin, vsip_hist_opt opt, const vsip_vview_f *r);
void vsip_mhisto_i(const vsip_mview_i *a, vsip_scalar_i min_bin, vsip_scalar_i
max_bin, vsip_hist_opt opt, const vsip_vview_i *r);
void vsip_mhisto_si(const vsip_mview_si *a, vsip_scalar_si min_bin,
vsip_scalar_si max_bin, vsip_hist_opt opt, const vsip_vview_si *r);
void vsip_vhisto_d(const vsip_vview_d *a, vsip_scalar_d min_bin, vsip_scalar_d
max_bin, vsip_hist_opt opt, const vsip_vview_d *r);
void vsip_vhisto_f(const vsip_vview_f *a, vsip_scalar_f min_bin, vsip_scalar_f
max_bin, vsip_hist_opt opt, const vsip_vview_f *r);
void vsip_vhisto_i(const vsip_vview_i *a, vsip_scalar_i min_bin, vsip_scalar_i
max_bin, vsip_hist_opt opt, const vsip_vview_i *r);
void vsip_vhisto_si(const vsip_vview_si *a, vsip_scalar_si min_bin, vsip_scalar_si
max_bin, vsip_hist_opt opt, const vsip_vview_si *r);
```

参数：

a：源向量（矩阵）视图。

min_bin：最小 bin 的阈值。

max_bin：最大 bin 的阈值。

opt：枚举类型，用于确定输出直方图是首先初始化为零，还是累积在以前的数据之上。

r：直方图向量视图。

返回值：

空。

4.2.6.7.2 vsip_dsfreqswap_p (新增 4 个)

交换向量的两半或矩阵的象限，以将零频率从原点重新映射到中间。

接口：

```
void vsip_cvfreqswap_d(const vsip_cvview_d *r); (新增接口)
```

```
void vsip_cvfreqswap_f(const vsip_cvview_f *r); (新增接口)
```

```
void vsip_vfreqswap_d(const vsip_vview_d *r); (新增接口)
```

```
void vsip_vfreqswap_f(const vsip_vview_f *r); (新增接口)
```

参数：

r：指向输入/输出向量视图对象的指针。

返回值：

空。

4.2.7 VSIPL-FT 插值函数（新增 9 个）

4.2.7.1 函数介绍

本节介绍 VSIPL-FT 的插值函数。对于 VSIPL-FT 插值，一组初始数据点 (x, y) 是通过测量过程（例如从一组传感器收集数据）获得的，或者是从某种功能关系中计算出来的。每个 y 数据点都与一个 x 数据点相关联。 x 个数据点按从小到大的顺序排列。术语“节点”用于描述特定的 (x, y) 对。

数据 x 存储在向量 x 中，数据 y 存储在向量 y 中。如果 x 和 y 的长度为 n ，则 x 的范围为 $[x(0), x(n-1)]$ 。 x 中包含的节点值必须按从小到大的顺序排列，并且不能相同。这意味着 $x[i]$ 严格小于 $x[i+1]$ 并且不相等。插值的有效性取决于插值方法的选择和插值的数据。

4.2.7.2 插值函数（新增 9 个）

4.2.7.2.1 vsip_vlagr_d（新增 1 个）

vlagr 拉格朗日插值多项式

接口：

```
vsip_scalar_d vsip_vlagr_d(vsip_scalar_d x,const vsip_vview_d *a,const  
vsip_vview_d *b,vsip_index low,vsip_index high); (新增接口)
```

参数：

x ：待求节点值。

a ：保存已知节点（向量）。

b ：保存对应已知节点的函数值（向量）。

low 、 $high$ ：索引，用来选取节点及对应函数值（ $high-low$ 为对应的阶数）。

返回值：

返回待求节点的函数值。

4.2.7.2.2 vsip_dinterp_nearest_p（新增 2 个）

最邻近插值

接口：

```
void vsip_vinterp_nearest_f(const vsip_vview_f *x0,const vsip_vview_f *y0,const  
vsip_vview_f *x,const vsip_vview_f *y); (新增接口)
```

```
void vsip_minterp_nearest_f(const vsip_vview_f *x0, const vsip_mview_f
```

*y0, vsip_major dim, const vsip_vview_f *x, const vsip_mview_f *y); (新增接口)

参数:

- x0: 已知 x 数据点的向量。
- y0: 已知 y 数据点的向量/矩阵。
- dim: 指示按行或按列插值。
- x: 请求插值 y 值的 x 值的输入向量。
- y: 输出与 x 长度相同的 y 向量/矩阵。

返回值:

空。

4.2.7.2.3 vsip_dinterp_linear_p (新增 2 个)

线性插值

接口:

void vsip_vinterp_linear_f(const vsip_vview_f *x0, const vsip_vview_f *y0, const vsip_vview_f *x, const vsip_vview_f *y); (新增接口)

void vsip_minterp_linear_f(const vsip_vview_f *x0, const vsip_mview_f *y0, vsip_major dim, const vsip_vview_f *x, const vsip_mview_f *y); (新增接口)

参数:

- x0: 已知 x 数据点的向量。
- y0: 已知 y 数据点的向量。
- dim: 指示按行或按列插值。
- x: 计算插值 y 值的 x 值的输入向量。此向量按从小到大的顺序排列。
- y: 输出与 x 长度相同的 y 向量/矩阵。

返回值:

空。

4.2.7.2.4 vsip_dinterp_spline_p (新增 2 个)

三次样条插值

接口:

void vsip_vinterp_spline_f(const vsip_vview_f *x0, const vsip_vview_f *y0, vsip_spline_f *spl, const vsip_vview_f *x, const vsip_vview_f *y); (新增接口)

void vsip_minterp_spline_f(const vsip_vview_f *x0, const vsip_mview_f *y0, vsip_spline_f *spl, vsip_major dim, const vsip_vview_f *x, const vsip_mview_f *y); (新增接口)

参数:

x0: 输入节点值的有序（从小到大）向量。

y0: 已知 y 数据点的向量/矩阵。

spl: Spline 对象。

x: 请求插值 y 值的 x 值的输入向量。

y: 输出与 x 长度相同的 y 向量/矩阵。

返回值:

空。

4.2.7.2.5 vsip_spline_create_p (新增 1 个)

创建三次样条对象。

接口:

vsip_spline_f*vsip_spline_create_f(vsip_length max); (新增接口)

参数:

max: 已知输入数据点的最大数量。N 必须大于 2。

返回值:

指向已创建样条对象的指针，或在创建失败时为 NULL。

4.2.7.2.6 vsip_spline_destroy_p (新增 1 个)

销毁三次样条对象。

接口:

int vsip_spline_destroy_f(vsip_spline_f*spl); (新增接口)

参数:

spl: 要销毁的样条对象或 NULL 对象。

返回值:

空。

4.2.8 VSIPL-FT 置换函数（新增 7 个）

4.2.8.1 函数介绍

本节介绍 VSIPL-FT 的置换函数。置换函数定义对 VSIPL-FT 矩阵进行置换操作的功能，按行或按列（以 “in place” 方式）完成置换操作。

4.2.8.2 置换函数（新增 7 个）

4.2.8.2.1 vsip_dmpermute_create_p（新增 1 个）

创建 permute 对象。

接口：

`vsip_permute *vsip_mpermute_create_f(vsip_length m, vsip_length n, vsip_major major);`（新增接口）

参数：

m：矩阵中要置换的行数。

n：矩阵中要置换的列数。

major：置换方向。VSIP_ROW 表示行是置换的，VSIP_COL 表示列是置换的。

返回值：

指向置换对象的指针，或在失败时为 NULL。

4.2.8.2.2 vsip_permute_init（新增 1 个）

初始化 permute 对象

接口：

`vsip_permute *vsip_permute_init(vsip_permute *perm, const vsip_vview_vi *vi);`（新增接口）

参数：

perm：Permute 对象。

vi：要置换的行或列的索引向量。

返回值：

指向置换对象的便利指针。

4.2.8.2.3 vsip_permute_destroy（新增 1 个）

销毁 permute 对象

接口:

```
void vsip_permute_destroy(vsip_permute *perm); (新增接口)
```

参数:

perm: 要销毁的有效置换对象或 NULL。

返回值:

空。

4.2.8.2.4 vsip_dmpermute_p (新增 2 个)

矩阵置换

接口:

```
void vsip_mpermute_f( const vsip_mview_f *in, const vsip_permute *perm, const vsip_mview_f *out); (新增接口)
```

```
void vsip_cmpermute_f( const vsip_cmview_f *in, const vsip_permute *perm, const vsip_cmview_f *out); (新增接口)
```

参数:

in: 输入矩阵的矩阵视图。

perm: 符合使用 vsip_dmpermute_create_p 函数创建的 permute 对象。

out: 输出矩阵的矩阵视图。

返回值:

空。

4.2.8.2.5 vsip_dmpermute_once_p (新增 2 个)

无需创建置换对象 perm 实现矩阵置换

接口:

```
void vsip_mpermute_once_f(const vsip_mview_f *in, vsip_major major, const vsip_vview_vi *p, const vsip_mview_f *out); (新增接口)
```

```
void vsip_cmpermute_once_f(const vsip_cmview_f *in, vsip_major major, const vsip_vview_vi *p, const vsip_cmview_f *out); (新增接口)
```

参数:

in: 输入矩阵的矩阵视图。

major: 置换方向。VSIP_ROW 表示行是置换的, VSIP_COL 表示列是置换的。

p: 要置换的行或列的索引向量。

out: 输出矩阵的矩阵视图。

返回值:

空。

4.2.9 VSIPL-FT 直接数据访问 (DDA) 函数 (新增 8 个)

4.2.9.1 函数介绍

本节介绍 VSIPL-FT 的直接数据访问 DDA 函数。直接数据访问 (DDA) 提供了通过原始指针访问视图数据的方法，与视图保存数据的方式无关。此直接数据访问可用于在本地桥接需要原始指针的代码，而不会破坏视图对象。

4.2.9.2 DDA 函数 (新增 8 个)

4.2.9.2.1 vsip_dda_dkcost_p (新增 1 个)

计算提供对视图的直接数据访问的成本

接口：

```
int vsip_dda_vcost_f(vsip_vview_f *view, vsip_datalayout *layout, vsip_dda_sync_policy sync_policy); (新增接口)
```

参数：

view：此参数提供将为其计算 DDA 成本的视图。

layout：此参数提供在计算 DDA 成本时要使用的布局属性。NULL 布局属性指示函数在计算成本时应使用视图的布局。

sync_policy：此参数提供在计算 DDA 开销时要使用的同步策略。

返回值：

此函数返回一个数值，该值表示通过 DDA 访问视图的成本。

4.2.9.2.2 vsip_dda_dkrequired_buffer_size_p (新增 1 个)

获取直接数据所需的缓冲区的大小 (以元素数为单位)

接口：

```
vsip_length vsip_dda_vrequired_buffer_size_f(vsip_vview_f *view, vsip_datalayout *layout, vsip_dda_sync_policy sync_policy); (新增接口)
```

参数：

view：此参数提供在计算 DDA 缓冲区大小时使用的视图，不能为 NULL。

layout：此参数提供在计算 DDA 成本时要使用的布局属性。NULL 布局属性指示函数在计算成本时应使用视图的布局。

sync_policy：此参数提供在计算 DDA 缓冲区大小时要使用的同步策略，为 1 或 2 或 3 或 4。

返回值:

此函数返回通过 DDA 访问视图数据时保存视图数据所需的 DDA 缓冲区的大小(以元素数为单位)。

4.2.9.2.3 vsip_dda_dkdatacreate_p (新增 1 个)

创建 DDA 数据对象。

接口:

```
vsip_dda_vdata_f *vsip_dda_vdatacreate_f(vsip_vview_f *view, vsip_datalayout
*layout, vsip_dda_sync_policy sync_policy, vsip_scalar_f *buffer); (新增接口)
```

参数:

view: 此参数提供将为其创建数据对象的视图。

layout: 此参数提供要用于数据对象的布局属性。NULL 布局属性指示函数应使用视图的布局来创建数据对象

sync_policy: 此参数提供用于数据对象的同步策略。

buffer: 此参数提供预分配的用户空间内存, 供实现用作 DDA 缓冲区。对于实际视图, 此内存必须至少与 vsip_dda_dkrequired_buffer_size_p()报告的大小一样大。对于复数视图, 此内存必须至少为 vsip_dda_dkrequired_buffer_size_p()报告大小的两倍。如果提供 NULL, 则将使用 VSIPL-FT 空间 DDA 缓冲区, 或者提供对视图内存的直接访问, 具体取决于实现和同步策略。

返回值:

此函数返回视图的数据对象, 或在失败时返回 NULL。如果在同步策略中设置了 VSIP_DDA_SYNC_POLICY_IN 标志, 则关联的 DDA 缓冲区(如果有)将从视图内存同步。

4.2.9.2.4 vsip_dda_dkdatadestroy_p (新增 1 个)

销毁 DDA 数据对象。

接口:

```
int vsip_dda_vdatadestroy_f(vsip_dda_vdata_f *data); (新增接口)
```

参数:

data: 此参数提供要销毁的数据对象。

返回值:

此函数在成功时返回零, 在失败时返回非零值。如果在同步策略中设置了 VSIP_DDA_SYNC_POLICY_OUT 标志, 则(如果有)关联的 DDA 缓冲区将同步到视图内存。

4.2.9.2.5 vsip_dda_dkdatagetattrib_p (新增 1 个)

获取数据对象的属性。

接口：

```
void vsip_dda_vdatagetattrib_f(vsip_dda_vdata_f *data, vsip_dda_vdataattr_f *attr); (新增接口)
```

参数：

data：此参数提供数据对象。

attr：此参数包含数据对象属性。

返回值：

此函数在成功完成时使用与数据对象关联的值填充 attributes 参数，并且在失败时不进行任何更改。

4.2.9.2.6 vsip_dda_kptr_p (新增 1 个)

获取 DDA 的指针。

接口：

```
vsip_scalar_f *vsip_dda_vptr_f(vsip_dda_vdata_f *data); (新增接口)
```

参数：

data：此参数提供将返回其 DDA 指针的数据对象。

返回值：

此函数在成功时返回 DDA 的指针，或在失败时返回 NULL 的指针。

4.2.9.2.7 vsip_dda_dksync_in_p (新增 1 个)

将视图内存中的数据对象同步到 DDA 缓冲区。

接口：

```
int vsip_dda_vsync_in_f(vsip_dda_vdata_f *data); (新增接口)
```

参数：

data：此参数提供要同步的数据对象。

返回值：

此函数在数据对象成功同步时返回零，在失败时返回非零值。

4.2.9.2.8 vsip_dda_dksync_out_p (新增 1 个)

将数据对象从 DDA 缓冲区同步到视图内存。

接口：

```
int vsip_dda_vsync_out_f(vsip_dda_vdata_f *data); (新增接口)
```

参数:

data: 此参数提供要同步的数据对象。

返回值:

此函数在数据对象成功同步时返回零，在失败时返回非零值。

5 总结

飞腾信号处理库符合 vsipl 规范，提供向量运算、矩阵运算、快速傅里叶变换、滤波函数、希尔伯特变换、卷积等高性能计算接口，可广泛应用于信号处理、航空航天、生产制造等领域。